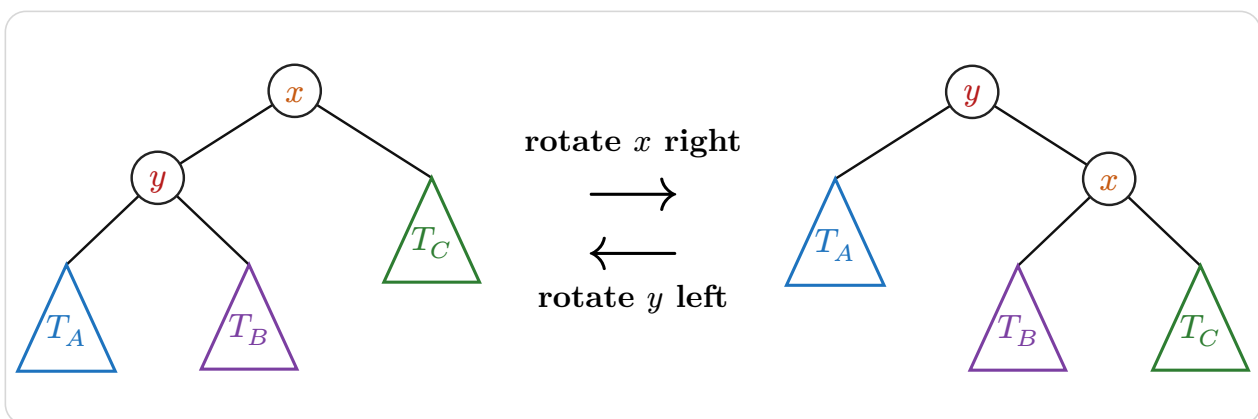


# typed-dsa

Declarative data-structure diagrams for Typst

## User Guide



Version 0.4.0

# Contents

---

|      |                                                                                                             |     |
|------|-------------------------------------------------------------------------------------------------------------|-----|
| 1    | Introduction .....                                                                                          | 3   |
| 2    | Getting started .....                                                                                       | 4   |
| 2.1  | Installation and import .....                                                                               | 4   |
| 2.2  | Your first diagram .....                                                                                    | 7   |
| 3    | Structures .....                                                                                            | 8   |
| 3.1  | <code>bst()</code> and <code>avl()</code> .....                                                             | 8   |
| 3.2  | <code>min-heap()</code> and <code>max-heap()</code> .....                                                   | 15  |
| 3.3  | <code>linked-list()</code> .....                                                                            | 19  |
| 3.4  | <code>doubly-linked-list()</code> .....                                                                     | 22  |
| 3.5  | <code>stack()</code> .....                                                                                  | 26  |
| 3.6  | <code>queue()</code> .....                                                                                  | 29  |
| 3.7  | <code>skip-list()</code> .....                                                                              | 34  |
| 3.8  | <code>array-view()</code> and <code>matrix()</code> .....                                                   | 38  |
| 3.9  | <code>hash-table()</code> .....                                                                             | 42  |
| 3.10 | Sorting algorithms .....                                                                                    | 44  |
| 3.11 | <code>graph()</code> .....                                                                                  | 57  |
| 3.12 | <code>labels</code> and <code>positions</code> .....                                                        | 67  |
| 3.13 | <code>edge-customizations</code> .....                                                                      | 68  |
| 3.14 | Graph algorithm traces .....                                                                                | 70  |
| 4    | Hand-composed trees .....                                                                                   | 73  |
| 5    | Operation transitions .....                                                                                 | 83  |
| 5.1  | Tree operations: <code>tree-insert()</code> , <code>tree-delete()</code> , <code>tree-search()</code> ..... | 88  |
| 5.2  | Heap operations: <code>heap-insert()</code> and <code>heap-extract</code> .....                             | 91  |
| 6    | Objects and operations .....                                                                                | 92  |
| 6.1  | The object model .....                                                                                      | 92  |
| 6.2  | Chaining operations .....                                                                                   | 93  |
| 6.3  | Wrapped operation sequences .....                                                                           | 95  |
| 6.4  | Custom arrows with <code>op-arrow()</code> .....                                                            | 98  |
| 7    | Localization .....                                                                                          | 99  |
| 7.1  | The three resolution layers .....                                                                           | 99  |
| 7.2  | <code>language:</code> .....                                                                                | 99  |
| 7.3  | <code>messages:</code> and the <code>messages(...)</code> helper .....                                      | 100 |
| 7.4  | <code>step-label:</code> has the last word .....                                                            | 102 |
| 7.5  | Structural default labels .....                                                                             | 102 |
| 7.6  | Message-key reference .....                                                                                 | 102 |
| 8    | Styling .....                                                                                               | 105 |
| 8.1  | Named themes and typography roles .....                                                                     | 105 |
| 8.2  | Autocomplete-friendly style builders .....                                                                  | 106 |
| 8.3  | Where keys are documented .....                                                                             | 107 |
| 8.4  | Diff-highlight styling .....                                                                                | 108 |
| 9    | Worked example: Last Stone Weight .....                                                                     | 110 |
| 10   | Limitations .....                                                                                           | 111 |
| 11   | Quick reference .....                                                                                       | 112 |
| 11.1 | Structure builders .....                                                                                    | 112 |
| 11.2 | Graph algorithms .....                                                                                      | 112 |
| 11.3 | Sorting algorithms .....                                                                                    | 113 |
| 11.4 | Hand-composed trees .....                                                                                   | 113 |
| 11.5 | Transitions and operations .....                                                                            | 113 |

11.6 Localization ..... 116

11.7 Styling keys ..... 117

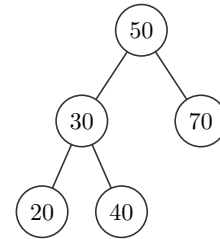
11.8 Styling helpers ..... 119

# 1 Introduction

typed-dsa renders data-structure diagrams from a declarative description inside Typst documents. It is built on top of [CeTZ](#). The package targets CS coursework: lecture notes, problem sets, exam solutions, where you need to show how an **operation** changes a structure, not just draw it once.

This guide covers every builder, argument, and operation, each with a runnable example: source on the left, rendered output on the right.

```
1 #bst(50, 30, 70, 20, 40).diagram
```

 Typst

## 2 Getting started

### 2.1 Installation and import

Import the package from the Typst preview namespace. A wildcard import gives you every public symbol:

```
1 #import "@preview/typed-dsa:0.5.0": *
```

 Typst

Or import only what you need:

```
1 #import "@preview/typed-dsa:0.5.0": bst, avl, min-heap, max-heap
```

 Typst

**Note.** A wildcard import shadows Typst's built-in `stack` function. Use `std.stack` for layout, or import `typed-dsa` selectively to avoid the name clash.

The package exports these symbols:

| Symbol                                                                                                                                                                                                | Purpose                                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| <code>bst</code> , <code>avl</code>                                                                                                                                                                   | Binary search tree / AVL tree builders. |
| <code>min-heap</code> , <code>max-heap</code>                                                                                                                                                         | Binary heap builders.                   |
| <code>linked-list</code> , <code>doubly-linked-list</code> , <code>stack</code> , <code>queue</code>                                                                                                  | Linear-structure builders.              |
| <code>array-view</code> , <code>matrix</code>                                                                                                                                                         | Static array and matrix/grid builders.  |
| <code>merge-sort</code> , <code>merge-operation</code> , <code>partition-step</code> , <code>quick-sort</code> , <code>bubble-sort</code> , <code>insertion-sort</code> , <code>selection-sort</code> | Automatic                               |

|                                                      |                                                                                                                                   |
|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
|                                                      | sort-<br>ing<br>traces.                                                                                                           |
| <code>graph</code>                                   | Graph<br>builder,<br>from<br>an<br>ad-<br>ja-<br>cency<br>dict.                                                                   |
| <code>tree , node , subtree</code>                   | Hand-<br>com-<br>posed<br>ab-<br>stract<br>trees.                                                                                 |
| <code>transition</code>                              | One-<br>shot<br>be-<br>fore<br>→<br>af-<br>ter<br>op-<br>er-<br>a-<br>tion<br>di-<br>a-<br>grams<br>for<br>trees<br>and<br>heaps. |
| <code>tree-insert , tree-delete , tree-search</code> | Tree<br>op-<br>er-<br>a-<br>tion<br>con-<br>struc-<br>tors,<br>for<br><code>tran-<br/>si-<br/>tion</code> .                       |

|                                                      |
|------------------------------------------------------|
| <code>heap-insert</code> , <code>heap-extract</code> |
| <code>sequence</code>                                |
| <code>op-arrow</code>                                |
| <code>theme</code> , <code>resolve</code>            |

Heap  
op-  
er-  
a-  
tion  
con-  
struc-  
tors,  
for  
tran-  
si-  
tion .

Wrapped  
lay-  
out  
for  
op-  
er-  
a-  
tion-  
step  
se-  
quences.

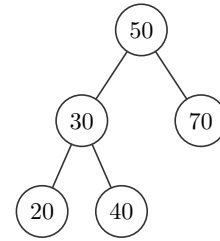
Reusable  
la-  
beled  
ar-  
row,  
for  
cus-  
tom  
op-  
er-  
a-  
tion  
lay-  
outs.

The  
de-  
fault  
style  
dic-  
tio-  
nary  
and  
its  
merge  
helper.

## 2.2 Your first diagram

Every builder takes keys and returns an object; `.diagram` is its drawing.

```
1 #bst(50, 30, 70, 20, 40).diagram
```

[Typst](#)

## 3 Structures

Every structure builder returns a **unified object**: a dictionary that is both the drawing ( `.diagram` ) and the thing you operate on. Calling an operation field returns a **step**; [Section 6](#) covers that model in full.

This section documents each builder's own arguments and methods. [Section 6](#) covers the step shape they share.

### 3.1 `bst()` and `avl()`

Full signatures:

```
1 #bst(..keys, style: (:), edge-customizations: (), node-customizations: (),
  node-labels: (:), language: "en", messages: (:))
2 #avl(..keys, style: (:), edge-customizations: (), node-customizations: (), node-labels:
  (:), language: "en", messages: (:))
```

 Typst

Both build a binary tree by inserting `keys` one at a time, in the order given. `bst` is a plain binary search tree; `avl` additionally rebalances after every insertion and deletion, so the tree stays height-balanced.

| Argument                         | Type                | Default    | Description                                                                                                                    |
|----------------------------------|---------------------|------------|--------------------------------------------------------------------------------------------------------------------------------|
| <code>..keys</code>              | int<br>content      | (required) | Keys inserted into the tree in order, one BST/AVL insert per key. Duplicate keys are ignored.                                  |
| <code>style</code>               | dictionary          | (:)        | Per-call style override merged over the defaults. See <a href="#">Section 8</a> .                                              |
| <code>edge-customizations</code> | array               | ()         | <code>(parent, child, options)</code> tuples restyling individual tree edges by node value. See <a href="#">Section 3.13</a> . |
| <code>node-customizations</code> | array               | ()         | <code>(node, options)</code> tuples restyling individual tree nodes by value.                                                  |
| <code>node-labels</code>         | array<br>dictionary | (:)        | Labels drawn outside individual tree nodes.                                                                                    |

#### Nested keys for `style`

| Path                           | Type   | Default               | Description                                                              |
|--------------------------------|--------|-----------------------|--------------------------------------------------------------------------|
| <code>style.node-radius</code> | float  | 0.34                  | Circle radius, or base size for other node shapes, in canvas units.      |
| <code>style.node-shape</code>  | str    | "circle"              | "circle" , "square" , "rounded" , "capsule" , "diamond" , or "hexagon" . |
| <code>style.x-gap</code>       | float  | 1.05                  | Horizontal spacing between in-order columns.                             |
| <code>style.y-gap</code>       | float  | 1.2                   | Vertical spacing between depths.                                         |
| <code>style.node-stroke</code> | stroke | 0.6pt +<br>"#333333") | Node outline.                                                            |
| <code>style.node-fill</code>   | color  | white                 | Default node fill.                                                       |
| <code>style.edge-stroke</code> | stroke | 0.6pt +<br>"#333333") | Parent-child edge stroke.                                                |

|                                        |                                |                                     |                                                                                                                                        |
|----------------------------------------|--------------------------------|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <code>style.edge-arrow</code>          | <code>none / bool / str</code> | <code>none</code>                   | Edge arrowheads: <code>none / false</code> , <code>"end"</code> or <code>true</code> , <code>"start"</code> , or <code>"both"</code> . |
| <code>style.edge-pattern</code>        | <code>str</code>               | <code>"normal"</code>               | <code>"normal"</code> , <code>"dashed"</code> , <code>"dotted"</code> , or <code>"wavy"</code> .                                       |
| <code>style.edge-wave-amplitude</code> | <code>float</code>             | <code>0.07</code>                   | Wave height in canvas units.                                                                                                           |
| <code>style.edge-wave-step</code>      | <code>float</code>             | <code>0.14</code>                   | Approximate wavelength in canvas units.                                                                                                |
| <code>style.node-text</code>           | <code>dictionary</code>        | <code>(size: 9pt)</code>            | Node text dictionary. See nested keys below.                                                                                           |
| <code>style.value-text</code>          | <code>dictionary</code>        | inherits <code>node-text</code>     | Value text overrides.                                                                                                                  |
| <code>style.label-text</code>          | <code>dictionary</code>        | <code>(fill: rgb("#555555"))</code> | Annotation text dictionary. See nested keys below.                                                                                     |
| <code>style.edge-label-text</code>     | <code>dictionary</code>        | inherits <code>label-text</code>    | Edge-label typography.                                                                                                                 |
| <code>style.operation-text</code>      | <code>dictionary</code>        | <code>(size: 8pt)</code>            | Operation-arrow caption typography.                                                                                                    |
| <code>style.scale</code>               | <code>float</code>             | <code>1.0</code>                    | Uniform scale on <code>.diagram</code> , <code>.before</code> , and <code>.after</code> .                                              |
| <code>style.diff-colors</code>         | <code>bool</code>              | <code>true</code>                   | <code>false</code> keeps operation marks but uses normal fills.                                                                        |

#### Nested keys for `style.node-text`

| Path                                  | Type                     | Default           | Description                                    |
|---------------------------------------|--------------------------|-------------------|------------------------------------------------|
| <code>style.node-text.size</code>     | <code>length</code>      | inherits          | Text size.                                     |
| <code>style.node-text.color</code>    | <code>color</code>       | inherits          | Text color.                                    |
| <code>style.node-text.rotation</code> | <code>angle</code>       | <code>0deg</code> | Text rotation applied by the diagram renderer. |
| <code>style.node-text.font</code>     | <code>str / array</code> | inherits          | Typst text font selection.                     |
| <code>style.node-text.weight</code>   | <code>str / int</code>   | inherits          | Typst text weight.                             |

#### Nested keys for `style.label-text`

| Path                                   | Type                     | Default                                  | Description                                                                                          |
|----------------------------------------|--------------------------|------------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>style.label-text.size</code>     | <code>length</code>      | 85% of <code>style.node-text.size</code> | Label text size.                                                                                     |
| <code>style.label-text.color</code>    | <code>color</code>       | <code>rgb("#555555")</code>              | Label text color.                                                                                    |
| <code>style.label-text.rotation</code> | <code>angle / str</code> | <code>0deg</code>                        | Label rotation. Tree and graph edge labels also accept <code>"edge"</code> to follow the edge angle. |
| <code>style.label-text.font</code>     | <code>str / array</code> | inherits                                 | Typst text font selection.                                                                           |
| <code>style.label-text.weight</code>   | <code>str / int</code>   | inherits                                 | Typst text weight.                                                                                   |

#### Nested keys for diff-highlight dictionaries

| Path                                   | Type               | Default                        | Description                                                                                                                                                                         |
|----------------------------------------|--------------------|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>style.new-style</code>           | color / dictionary | <code>rgb("#C8E6C9")</code>    | Node/cell added by an operation. A color is shorthand for <code>(fill: color)</code> .                                                                                              |
| <code>style.path-style</code>          | color / dictionary | <code>rgb("#FFE9A8")</code>    | Traversal or sift path.                                                                                                                                                             |
| <code>style.remove-style</code>        | color / dictionary | <code>rgb("#FFCDD2")</code>    | Node/cell removed by an operation.                                                                                                                                                  |
| <code>style.rotate-style</code>        | color / dictionary | <code>rgb("#BBDEFB")</code>    | AVL rotation nodes and visible subtree roots.                                                                                                                                       |
| <code>style.*-style.fill</code>        | color              | mark default                   | Fill for the marked node/cell.                                                                                                                                                      |
| <code>style.*-style.shape</code>       | str                | <code>style.node-shape</code>  | Marked tree/heap node shape: <code>"circle"</code> , <code>"square"</code> , <code>"rounded"</code> , <code>"capsule"</code> , <code>"diamond"</code> , or <code>"hexagon"</code> . |
| <code>style.*-style.stroke</code>      | stroke             | normal stroke                  | Marked node/cell outline.                                                                                                                                                           |
| <code>style.*-style.node-radius</code> | float              | <code>style.node-radius</code> | Marked tree/heap node radius.                                                                                                                                                       |
| <code>style.*-style.text</code>        | dictionary         | <code>style.node-text</code>   | Marked node/cell text overrides. Accepts <code>size</code> , <code>color</code> , <code>font</code> , and <code>weight</code> .                                                     |

### Nested keys for `edge-customizations`

| Path                                               | Type              | Default               | Description                                                                                                                                      |
|----------------------------------------------------|-------------------|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>edge-customizations[]</code>                 | tuple             | n/a                   | One <code>(from, to, options)</code> tuple. For trees, <code>from</code> / <code>to</code> are parent and child values; for graphs, node labels. |
| <code>edge-customizations[].from</code>            | content           | required              | Source node identity.                                                                                                                            |
| <code>edge-customizations[].to</code>              | content           | required              | Target node identity.                                                                                                                            |
| <code>edge-customizations[].options</code>         | dictionary        | required              | Per-edge options.                                                                                                                                |
| <code>edge-customizations[].options.stroke</code>  | stroke            | none                  | Full stroke override. Wins over <code>color</code> if both are set.                                                                              |
| <code>edge-customizations[].options.color</code>   | color             | none                  | Paint only, at the default thickness.                                                                                                            |
| <code>edge-customizations[].options.pattern</code> | str               | <code>"normal"</code> | <code>"normal"</code> , <code>"dashed"</code> , <code>"dotted"</code> , or <code>"wavy"</code> .                                                 |
| <code>edge-customizations[].options.arrow</code>   | none / bool / str | none                  | Per-edge arrowheads: <code>none</code> , <code>"end"</code> / <code>true</code> , <code>"start"</code> , or <code>"both"</code> .                |
| <code>edge-customizations[].options.bend</code>    | str / bool        | false                 | <code>"left"</code> or <code>"right"</code> curves the edge relative to its travel direction. <code>false</code> keeps it straight.              |
| <code>edge-customizations[].options.angle</code>   | angle             | 25deg                 | How sharply a bent edge curves. Ignored when <code>bend</code> is <code>false</code> .                                                           |

|                                                  |                                                |                   |                                                                                                                       |
|--------------------------------------------------|------------------------------------------------|-------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>edge-customizations[].options.label</code> | <code>content</code> / <code>dictionary</code> | <code>none</code> | Tree edge label content, or graph edge-label text overrides. A dictionary can include <code>content</code> for trees. |
|--------------------------------------------------|------------------------------------------------|-------------------|-----------------------------------------------------------------------------------------------------------------------|

#### Nested keys for `edge-customizations[].options.label`

| Path                                                      | Type                                  | Default                                | Description                                                                                          |
|-----------------------------------------------------------|---------------------------------------|----------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>edge-customizations[].options.label.size</code>     | <code>length</code>                   | <code>style.label-text.size</code>     | Edge-label size.                                                                                     |
| <code>edge-customizations[].options.label.content</code>  | <code>content</code>                  | <code>tree only</code>                 | Tree edge label content. Graph edge labels come from adjacency entries such as (" <b>B</b> ", [4]) . |
| <code>edge-customizations[].options.label.color</code>    | <code>color</code>                    | <code>style.label-text.color</code>    | Edge-label text color.                                                                               |
| <code>edge-customizations[].options.label.rotation</code> | <code>angle</code> / <code>str</code> | <code>style.label-text.rotation</code> | An angle, or " <code>edge</code> " to follow the tree/graph edge angle.                              |
| <code>edge-customizations[].options.label.font</code>     | <code>str</code> / <code>array</code> | <code>style.label-text.font</code>     | Typst text font selection.                                                                           |
| <code>edge-customizations[].options.label.weight</code>   | <code>str</code> / <code>int</code>   | <code>style.label-text.weight</code>   | Typst text weight.                                                                                   |

#### Nested keys for `node-customizations`

| Path                                                   | Type                    | Default                        | Description                                                                                                                                                                           |
|--------------------------------------------------------|-------------------------|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>node-customizations[]</code>                     | <code>tuple</code>      | <code>n/a</code>               | One ( <code>node</code> , <code>options</code> ) tuple.                                                                                                                               |
| <code>node-customizations[].node</code>                | <code>content</code>    | <code>required</code>          | For <code>bst</code> / <code>avl</code> , the inserted key.                                                                                                                           |
| <code>node-customizations[].options</code>             | <code>dictionary</code> | <code>required</code>          | Per-node drawing options.                                                                                                                                                             |
| <code>node-customizations[].options.fill</code>        | <code>color</code>      | <code>normal fill</code>       | Node fill.                                                                                                                                                                            |
| <code>node-customizations[].options.stroke</code>      | <code>stroke</code>     | <code>normal stroke</code>     | Node outline.                                                                                                                                                                         |
| <code>node-customizations[].options.shape</code>       | <code>str</code>        | <code>style.node-shape</code>  | Trees/graphs only: " <code>circle</code> " , " <code>square</code> " , " <code>rounded</code> " , " <code>capsule</code> " , " <code>diamond</code> " , or " <code>hexagon</code> " . |
| <code>node-customizations[].options.node-radius</code> | <code>float</code>      | <code>style.node-radius</code> | Trees/graphs only. Shape size.                                                                                                                                                        |
| <code>node-customizations[].options.text</code>        | <code>dictionary</code> | <code>style.node-text</code>   | Text style merged into the node text.                                                                                                                                                 |

#### Nested keys for `node-labels`

| Path                                     | Type                                           | Default                               | Description                                                                                             |
|------------------------------------------|------------------------------------------------|---------------------------------------|---------------------------------------------------------------------------------------------------------|
| <code>node-labels[]</code>               | <code>tuple</code>                             | <code>n/a</code>                      | One ( <code>node</code> , <code>label</code> ) tuple.                                                   |
| <code>node-labels[].node</code>          | <code>content</code>                           | <code>required</code>                 | For <code>bst</code> / <code>avl</code> , the inserted key.                                             |
| <code>node-labels[].label</code>         | <code>content</code> / <code>dictionary</code> | <code>required</code>                 | Bare content for quick labels, or a dictionary with <code>content</code> plus style and placement keys. |
| <code>node-labels[].label.content</code> | <code>content</code>                           | <code>required for dict labels</code> | Label body. <code>body</code> is also accepted.                                                         |

|                                           |                          |                                        |                                                                                                        |
|-------------------------------------------|--------------------------|----------------------------------------|--------------------------------------------------------------------------------------------------------|
| <code>node-labels[].label.position</code> | <code>str / angle</code> | <code>"right"</code>                   | <code>"right"</code> , <code>"left"</code> , <code>"top"</code> , <code>"bottom"</code> , or an angle. |
| <code>node-labels[].label.offset</code>   | <code>point</code>       | <code>(0, 0)</code>                    | Extra <code>(dx, dy)</code> shift after the position is applied.                                       |
| <code>node-labels[].label.size</code>     | <code>length</code>      | <code>style.label-text.size</code>     | Node-label size.                                                                                       |
| <code>node-labels[].label.color</code>    | <code>color</code>       | <code>style.label-text.color</code>    | Node-label text color.                                                                                 |
| <code>node-labels[].label.rotation</code> | <code>angle</code>       | <code>style.label-text.rotation</code> | Rotation of the label text itself.                                                                     |
| <code>node-labels[].label.font</code>     | <code>str / array</code> | <code>style.label-text.font</code>     | Typst text font selection.                                                                             |
| <code>node-labels[].label.weight</code>   | <code>str / int</code>   | <code>style.label-text.weight</code>   | Typst text weight.                                                                                     |

### Nested keys for `style.node-labels`

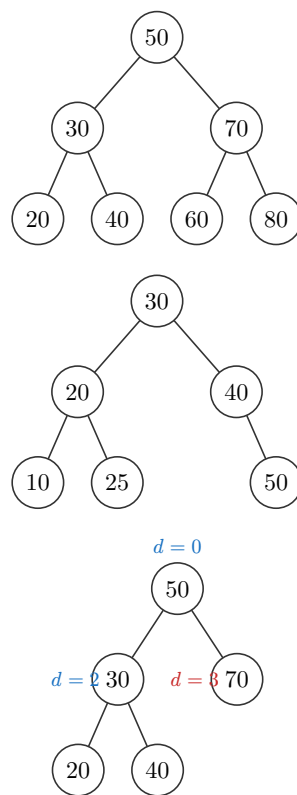
| Path                                    | Type                     | Default                                | Description                                                    |
|-----------------------------------------|--------------------------|----------------------------------------|----------------------------------------------------------------|
| <code>style.node-labels.position</code> | <code>str / angle</code> | <code>"right"</code>                   | Default position for all node labels in this diagram.          |
| <code>style.node-labels.offset</code>   | <code>point</code>       | <code>(0, 0)</code>                    | Default extra <code>(dx, dy)</code> shift for all node labels. |
| <code>style.node-labels.gap</code>      | <code>float</code>       | <code>0.22</code>                      | Default distance from the node boundary in canvas units.       |
| <code>style.node-labels.size</code>     | <code>length</code>      | <code>style.label-text.size</code>     | Default node-label text size.                                  |
| <code>style.node-labels.color</code>    | <code>color</code>       | <code>style.label-text.color</code>    | Default node-label text color.                                 |
| <code>style.node-labels.font</code>     | <code>str / array</code> | <code>inherits</code>                  | Typst text font selection.                                     |
| <code>style.node-labels.weight</code>   | <code>str / int</code>   | <code>inherits</code>                  | Typst text weight.                                             |
| <code>style.node-labels.rotation</code> | <code>angle</code>       | <code>style.label-text.rotation</code> | Default rotation of the label text itself.                     |

| Field                 | Call                           | Effect                                                                                                  |
|-----------------------|--------------------------------|---------------------------------------------------------------------------------------------------------|
| <code>.diagram</code> | <code>n/a</code>               | The rendered tree.                                                                                      |
| <code>.insert</code>  | <code>(obj.insert)(key)</code> | Insert <code>key</code> . Returns a step (Section 6).                                                   |
| <code>.delete</code>  | <code>(obj.delete)(key)</code> | Delete <code>key</code> . AVL deletes rebalance when removal makes a subtree too heavy. Returns a step. |
| <code>.search</code>  | <code>(obj.search)(key)</code> | Highlight the traversal path to <code>key</code> . Returns a step.                                      |

```

1  #bst(50, 30, 70, 20, 40, 60, 80).diagram
2  #avl(10, 20, 30, 40, 50, 25).diagram
3
4  #bst(
5    50, 30, 70, 20, 40,
6    node-labels: (
7      (50, (content: [$d=0$], position: "top")),
8      (30, [$d=2$]),
9      (70, (content: [$d=3$], color: rgb("#C92A2A"))),
10   ),
11   style: (node-labels: (position: "left", color: rgb("#1971C2"))),
12 ).diagram

```

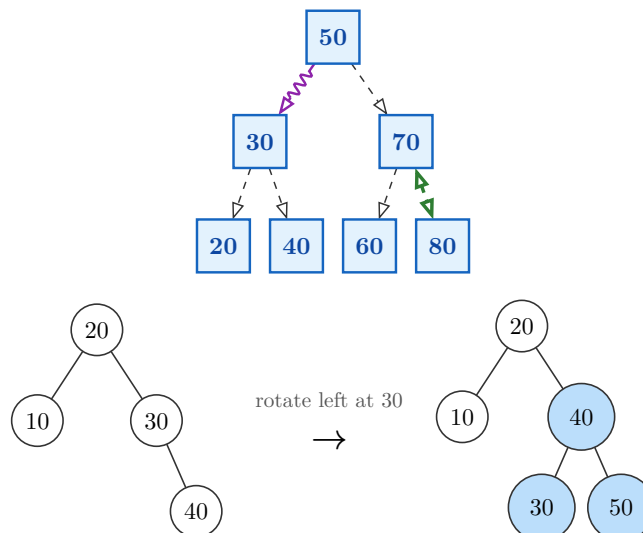
 Typst


Argument coverage: this example combines tree-level styling, nested `node-text`, edge arrows/patterns, per-edge customizations, and diff-highlight dictionaries.

```

1  #std.stack(spacing: 1em,
2    bst(
3      50, 30, 70, 20, 40, 60, 80,
4      style: (
5        node-shape: "square",
6        node-radius: 0.38,
7        node-fill: rgb("#E3F2FD"),
8        node-stroke: 1pt + rgb("#1565C0"),
9        node-text: (size: 10pt, weight: "bold", color: rgb("#0D47A1")),
10       edge-arrow: "end",
11       edge-pattern: "dashed",
12       scale: 0.92,
13       y-gap : 1.5,
14     ),
15     edge-customizations: (
16       (50, 30, (pattern: "wavy", color: rgb("#8E24AA"))),
17       (70, 80, (stroke: 1.4pt + rgb("#2E7D32"), arrow: "both")),
18     ),
19   ).diagram,
20   transition(
21     "avl",
22     (20, 10, 30, 40),
23     tree-insert(50),
24     style: (
25       new-style: (shape: "square", stroke: 1.5pt + rgb("#2E7D32")),
26       path-style: (stroke: 1.2pt + rgb("#F9A825")),
27       rotate-style: (node-radius: 0.44),
28     ),
29   ),
30 )

```



### 3.2 min-heap() and max-heap()

Full signatures:

```
1 #min-heap(..keys, style: (:), language: "en", messages: (:))
2 #max-heap(..keys, style: (:), language: "en", messages: (:))
```

**t** Typst

A heap is an array underneath. Index  $i$ 's children live at  $2i+1$  and  $2i+2$ . Each key sifts up as it's inserted, and the diagram draws the complete binary tree that array shape forms. `min-heap` keeps the smallest key at the root; `max-heap` keeps the largest.

| Argument            | Type           | Default    | Description                                                                       |
|---------------------|----------------|------------|-----------------------------------------------------------------------------------|
| <code>..keys</code> | int<br>content | (required) | Keys inserted into the heap in order, one sift-up per key.                        |
| <code>style</code>  | dictionary     | (:)        | Per-call style override merged over the defaults. See <a href="#">Section 8</a> . |

#### Nested keys for style

| Path                                   | Type              | Default                 | Description                                                              |
|----------------------------------------|-------------------|-------------------------|--------------------------------------------------------------------------|
| <code>style.node-radius</code>         | float             | 0.34                    | Circle radius, or base size for other node shapes, in canvas units.      |
| <code>style.node-shape</code>          | str               | "circle"                | "circle" , "square" , "rounded" , "capsule" , "diamond" , or "hexagon" . |
| <code>style.x-gap</code>               | float             | 1.05                    | Horizontal spacing between in-order columns.                             |
| <code>style.y-gap</code>               | float             | 1.2                     | Vertical spacing between depths.                                         |
| <code>style.node-stroke</code>         | stroke            | 0.6pt + rgb("#333333")  | Node outline.                                                            |
| <code>style.node-fill</code>           | color             | white                   | Default node fill.                                                       |
| <code>style.edge-stroke</code>         | stroke            | 0.6pt + rgb("#333333")  | Parent-child edge stroke.                                                |
| <code>style.edge-arrow</code>          | none / bool / str | none                    | Edge arrowheads: none / false , "end" or true , "start" , or "both" .    |
| <code>style.edge-pattern</code>        | str               | "normal"                | "normal" , "dashed" , "dotted" , or "wavy" .                             |
| <code>style.edge-wave-amplitude</code> | float             | 0.07                    | Wave height in canvas units.                                             |
| <code>style.edge-wave-step</code>      | float             | 0.14                    | Approximate wavelength in canvas units.                                  |
| <code>style.node-text</code>           | dictionary        | (size: 9pt)             | Node text dictionary. See nested keys below.                             |
| <code>style.value-text</code>          | dictionary        | inherits node-text      | Value text overrides.                                                    |
| <code>style.label-text</code>          | dictionary        | (fill: "#555555") rgb() | Annotation text dictionary. See nested keys below.                       |
| <code>style.edge-label-text</code>     | dictionary        | inherits label-text     | Edge-label typography.                                                   |
| <code>style.operation-text</code>      | dictionary        | (size: 8pt)             | Operation-arrow caption typography.                                      |

|                                |       |      |                                                                                           |
|--------------------------------|-------|------|-------------------------------------------------------------------------------------------|
| <code>style.scale</code>       | float | 1.0  | Uniform scale on <code>.diagram</code> , <code>.before</code> , and <code>.after</code> . |
| <code>style.diff-colors</code> | bool  | true | <code>false</code> keeps operation marks but uses normal fills.                           |

#### Nested keys for `style.node-text`

| Path                                  | Type        | Default  | Description                                    |
|---------------------------------------|-------------|----------|------------------------------------------------|
| <code>style.node-text.size</code>     | length      | inherits | Text size.                                     |
| <code>style.node-text.color</code>    | color       | inherits | Text color.                                    |
| <code>style.node-text.rotation</code> | angle       | 0deg     | Text rotation applied by the diagram renderer. |
| <code>style.node-text.font</code>     | str / array | inherits | Typst text font selection.                     |
| <code>style.node-text.weight</code>   | str / int   | inherits | Typst text weight.                             |

#### Nested keys for `style.label-text`

| Path                                   | Type        | Default                                  | Description                                                                                          |
|----------------------------------------|-------------|------------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>style.label-text.size</code>     | length      | 85% of <code>style.node-text.size</code> | Label text size.                                                                                     |
| <code>style.label-text.color</code>    | color       | <code>rgb("#555555")</code>              | Label text color.                                                                                    |
| <code>style.label-text.rotation</code> | angle / str | 0deg                                     | Label rotation. Tree and graph edge labels also accept <code>"edge"</code> to follow the edge angle. |
| <code>style.label-text.font</code>     | str / array | inherits                                 | Typst text font selection.                                                                           |
| <code>style.label-text.weight</code>   | str / int   | inherits                                 | Typst text weight.                                                                                   |

#### Nested keys for diff-highlight dictionaries

| Path                                   | Type               | Default                        | Description                                                                                                                                                                         |
|----------------------------------------|--------------------|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>style.new-style</code>           | color / dictionary | <code>rgb("#C8E6C9")</code>    | Node/cell added by an operation. A color is shorthand for <code>(fill: color)</code> .                                                                                              |
| <code>style.path-style</code>          | color / dictionary | <code>rgb("#FFE9A8")</code>    | Traversal or sift path.                                                                                                                                                             |
| <code>style.remove-style</code>        | color / dictionary | <code>rgb("#FFCDD2")</code>    | Node/cell removed by an operation.                                                                                                                                                  |
| <code>style.rotate-style</code>        | color / dictionary | <code>rgb("#BBDEFB")</code>    | AVL rotation nodes and visible subtree roots.                                                                                                                                       |
| <code>style.*-style.fill</code>        | color              | mark default                   | Fill for the marked node/cell.                                                                                                                                                      |
| <code>style.*-style.shape</code>       | str                | <code>style.node-shape</code>  | Marked tree/heap node shape: <code>"circle"</code> , <code>"square"</code> , <code>"rounded"</code> , <code>"capsule"</code> , <code>"diamond"</code> , or <code>"hexagon"</code> . |
| <code>style.*-style.stroke</code>      | stroke             | normal stroke                  | Marked node/cell outline.                                                                                                                                                           |
| <code>style.*-style.node-radius</code> | float              | <code>style.node-radius</code> | Marked tree/heap node radius.                                                                                                                                                       |

|                                 |            |                              |                                                                                                                                    |
|---------------------------------|------------|------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <code>style.*-style.text</code> | dictionary | <code>style.node-text</code> | Marked node/cell text overrides.<br>Accepts <code>size</code> , <code>color</code> , <code>font</code> , and <code>weight</code> . |
|---------------------------------|------------|------------------------------|------------------------------------------------------------------------------------------------------------------------------------|

| Field                 | Call                           | Effect                                                                             |
|-----------------------|--------------------------------|------------------------------------------------------------------------------------|
| <code>.diagram</code> | n/a                            | The rendered heap.                                                                 |
| <code>.insert</code>  | <code>(obj.insert)(key)</code> | Append <code>key</code> and sift it up. Returns a step.                            |
| <code>.extract</code> | <code>(obj.extract)()</code>   | Remove the root (no key), move the last leaf up, and sift it down. Returns a step. |

1 `#min-heap(50, 30, 70, 20, 40, 60, 80).diagram`

2 `#max-heap(50, 30, 70, 20, 40, 60, 80).diagram`

t

Typst

```
graph TD
    20((20)) --- 30((30))
    20 --- 60((60))
    30 --- 50((50))
    30 --- 40((40))
    60 --- 70((70))
    60 --- 80((80))

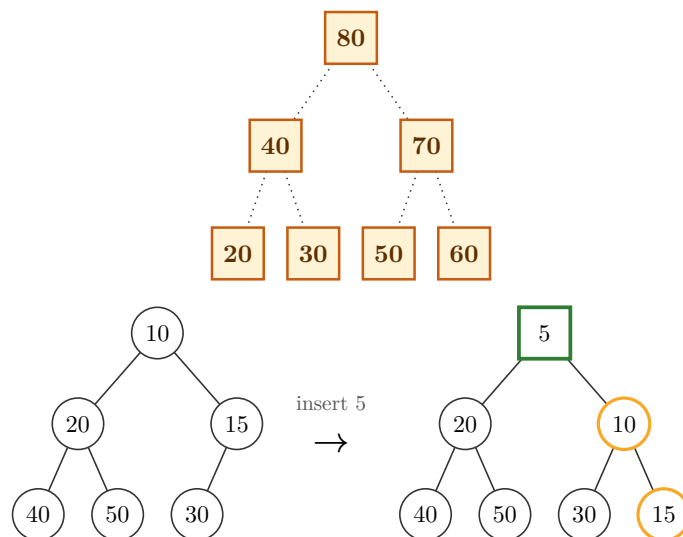
    80_2((80)) --- 40_2((40))
    80_2 --- 70_2((70))
    40_2 --- 20_2((20))
    40_2 --- 30_2((30))
    70_2 --- 50_2((50))
    70_2 --- 60_2((60))
```

Argument coverage: heaps reuse tree styling, and heap operations use the same diff-highlight dictionaries.

```

1  #std.stack(spacing: 1em,
2      max-heap(
3          50, 30, 70, 20, 40, 60, 80,
4          style: (
5              node-shape: "square",
6              node-radius: 0.36,
7              y-gap: 1.5,
8              node-fill: rgb("#FFF3D6"),
9              node-stroke: 1pt + rgb("#C75B12"),
10             node-text: (size: 10pt, color: rgb("#5D2F00"), weight: "bold"),
11             edge-pattern: "dotted",
12             scale: 0.95,
13         ),
14     ).diagram,
15     transition(
16         "min-heap",
17         (10, 20, 15, 40, 50, 30),
18         heap-insert(5),
19         style: (
20             diff-colors: false,
21             new-style: (shape: "square", stroke: 1.4pt + rgb("#2E7D32")),
22             path-style: (stroke: 1.2pt + rgb("#F9A825")),
23         ),
24     ),
25 )

```



**Note.** A binary heap supports inserting and pulling out the root efficiently. It has no key-based `delete` or `search` the way `bst` / `avl` do: that's not a gap in typed-dsa, it's what a heap is.

### 3.3 linked-list()

Full signature:

```
1 #linked-list(
2   ..vals,
3   style: (:),
4   pointer: false,
5   addresses: none,
6   head: false,
7   language: "en",
8   messages: (:),
9 )
```

 Typst

| Argument               | Type           | Default    | Description                                                                            |
|------------------------|----------------|------------|----------------------------------------------------------------------------------------|
| <code>..vals</code>    | int<br>content | (required) | Values in the list, head first.                                                        |
| <code>style</code>     | dictionary     | (:)        | Per-call style override merged over the defaults. See <a href="#">Section 8</a> .      |
| <code>pointer</code>   | bool           | false      | Draw each node as a <code>data   next</code> cell pair instead of a single value cell. |
| <code>addresses</code> | none<br>array  | none       | With <code>pointer: true</code> , one address label drawn under each node, in order.   |
| <code>head</code>      | bool           | false      | Draw a “Head” arrow pointing at the first node.                                        |

#### Nested keys for `style`

| Path                             | Type       | Default                | Description                                                                     |
|----------------------------------|------------|------------------------|---------------------------------------------------------------------------------|
| <code>style.box-w</code>         | float      | 0.95                   | Cell width.                                                                     |
| <code>style.box-h</code>         | float      | 0.7                    | Cell height.                                                                    |
| <code>style.box-shape</code>     | str        | "square"               | "square", "rounded", or "capsule".                                              |
| <code>style.box-gap</code>       | float      | 0.55                   | Gap between cells or list nodes.                                                |
| <code>style.box-stroke</code>    | stroke     | 0.6pt + rgb("#333333") | Cell outline.                                                                   |
| <code>style.box-fill</code>      | color      | white                  | Default cell fill.                                                              |
| <code>style.ptr-fill</code>      | color      | rgb("#D7ECC9")         | Next-pointer cell fill for <code>linked-list(pointer: true)</code> .            |
| <code>style.prev-ptr-fill</code> | color      | rgb("#D7ECC9")         | Previous-pointer cell fill for <code>doubly-linked-list(pointer: true)</code> . |
| <code>style.next-ptr-fill</code> | color      | rgb("#D7ECC9")         | Next-pointer cell fill for <code>doubly-linked-list(pointer: true)</code> .     |
| <code>style.node-text</code>     | dictionary | (size: 9pt)            | Cell text dictionary. See nested keys below.                                    |
| <code>style.value-text</code>    | dictionary | inherits node-text     | Value text overrides.                                                           |

|                                   |            |                                                  |                                                                                           |
|-----------------------------------|------------|--------------------------------------------------|-------------------------------------------------------------------------------------------|
| <code>style.label-text</code>     | dictionary | <code>(fill: "#555555")</code> <code>rgb(</code> | Head, address, front, and rear label text dictionary. See nested keys below.              |
| <code>style.pointer-text</code>   | dictionary | <code>inherits</code> <code>label-text</code>    | Pointer, head, address, front, and rear typography.                                       |
| <code>style.operation-text</code> | dictionary | <code>(size: 8pt)</code>                         | Operation-arrow caption typography.                                                       |
| <code>style.scale</code>          | float      | 1.0                                              | Uniform scale on <code>.diagram</code> , <code>.before</code> , and <code>.after</code> . |
| <code>style.diff-colors</code>    | bool       | true                                             | <code>false</code> keeps operation marks but uses normal fills.                           |

#### Nested keys for `style.node-text`

| Path                                  | Type        | Default  | Description                                    |
|---------------------------------------|-------------|----------|------------------------------------------------|
| <code>style.node-text.size</code>     | length      | inherits | Text size.                                     |
| <code>style.node-text.color</code>    | color       | inherits | Text color.                                    |
| <code>style.node-text.rotation</code> | angle       | 0deg     | Text rotation applied by the diagram renderer. |
| <code>style.node-text.font</code>     | str / array | inherits | Typst text font selection.                     |
| <code>style.node-text.weight</code>   | str / int   | inherits | Typst text weight.                             |

#### Nested keys for `style.label-text`

| Path                                   | Type        | Default                                  | Description                                                                                          |
|----------------------------------------|-------------|------------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>style.label-text.size</code>     | length      | 85% of <code>style.node-text.size</code> | Label text size.                                                                                     |
| <code>style.label-text.color</code>    | color       | <code>rgb("#555555")</code>              | Label text color.                                                                                    |
| <code>style.label-text.rotation</code> | angle / str | 0deg                                     | Label rotation. Tree and graph edge labels also accept <code>"edge"</code> to follow the edge angle. |
| <code>style.label-text.font</code>     | str / array | inherits                                 | Typst text font selection.                                                                           |
| <code>style.label-text.weight</code>   | str / int   | inherits                                 | Typst text weight.                                                                                   |

#### Nested keys for diff-highlight dictionaries

| Path                            | Type               | Default                     | Description                                                                            |
|---------------------------------|--------------------|-----------------------------|----------------------------------------------------------------------------------------|
| <code>style.new-style</code>    | color / dictionary | <code>rgb("#C8E6C9")</code> | Node/cell added by an operation. A color is shorthand for <code>(fill: color)</code> . |
| <code>style.path-style</code>   | color / dictionary | <code>rgb("#FFE9A8")</code> | Traversal or sift path.                                                                |
| <code>style.remove-style</code> | color / dictionary | <code>rgb("#FFCDD2")</code> | Node/cell removed by an operation.                                                     |
| <code>style.rotate-style</code> | color / dictionary | <code>rgb("#BBDEFB")</code> | AVL rotation nodes and visible subtree roots.                                          |

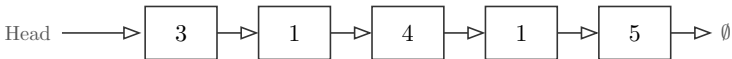
|                                        |                         |                                |                                                                                                                                 |
|----------------------------------------|-------------------------|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <code>style.*-style.fill</code>        | <code>color</code>      | mark default                   | Fill for the marked node/cell.                                                                                                  |
| <code>style.*-style.shape</code>       | <code>str</code>        | <code>style.node-shape</code>  | Marked tree/heap node shape: "circle" , "square" , "rounded" , "capsule" , "diamond" , or "hexagon" .                           |
| <code>style.*-style.stroke</code>      | <code>stroke</code>     | normal stroke                  | Marked node/cell outline.                                                                                                       |
| <code>style.*-style.node-radius</code> | <code>float</code>      | <code>style.node-radius</code> | Marked tree/heap node radius.                                                                                                   |
| <code>style.*-style.text</code>        | <code>dictionary</code> | <code>style.node-text</code>   | Marked node/cell text overrides. Accepts <code>size</code> , <code>color</code> , <code>font</code> , and <code>weight</code> . |

| Field                   | Call                                      | Effect                                                                                                  |
|-------------------------|-------------------------------------------|---------------------------------------------------------------------------------------------------------|
| <code>.diagram</code>   | n/a                                       | The rendered list.                                                                                      |
| <code>.prepend</code>   | <code>(obj.prepend)(v)</code>             | Insert <code>v</code> at the head. Returns a step.                                                      |
| <code>.insert</code>    | <code>(obj.insert)(v, index: none)</code> | Append <code>v</code> by default, or insert at an index from <code>0</code> through the current length. |
| <code>.delete</code>    | <code>(obj.delete)(v)</code>              | Remove the first node equal to <code>v</code> . Returns a step.                                         |
| <code>.delete-at</code> | <code>(obj.delete-at)(index)</code>       | Remove the node at <code>index</code> . Returns a step.                                                 |
| <code>.search</code>    | <code>(obj.search)(v)</code>              | Highlight the traversed prefix. The step also exposes <code>.found</code> and <code>.index</code> .     |

1

`#linked-list(3, 1, 4, 1, 5, head: true).diagram`

Typst



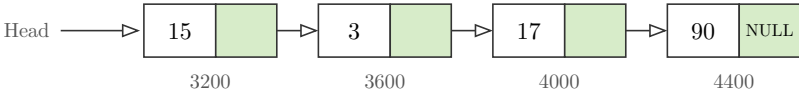
1

`#linked-list(15, 3, 17, 90, pointer: true, head: true,`

Typst

2

`addresses: ("3200", "3600", "4000", "4400")).diagram`

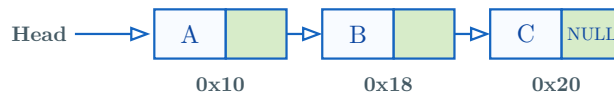


Argument coverage: `pointer` , `addresses` , `head` , label text, pointer-cell fills, scaling, and operation mark styling.

```

1  #let xs = linked-list(
2    [A], [B], [C],
3    pointer: true,
4    head: true,
5    addresses: ("0x10", "0x18", "0x20"),
6    style: (
7      box-w: 0.9,
8      box-h: 0.62,
9      box-gap: 0.45,
10     box-fill: rgb("#F8FBFF"),
11     box-stroke: 0.8pt + rgb("#1565C0"),
12     ptr-fill: rgb("#D7ECC9"),
13     node-text: (size: 9pt, color: rgb("#0D47A1")),
14     label-text: (size: 7.5pt, color: rgb("#455A64"), weight: "bold"),
15     scale: 1.05,
16     new-style: (fill: rgb("#C8E6C9"), stroke: 1pt + rgb("#2E7D32")),
17     remove-style: (fill: rgb("#FFCDD2"), stroke: 1pt + rgb("#C62828")),
18   ),
19 )
20 #xs.diagram

```



### 3.4 doubly-linked-list()

Full signature:

```

1  #doubly-linked-list(
2    ..vals,
3    style: (:),
4    pointer: false,
5    addresses: none,
6    head: false,
7    language: "en",
8    messages: (:),
9  )

```

| Argument            | Type           | Default    | Description                                                                       |
|---------------------|----------------|------------|-----------------------------------------------------------------------------------|
| <code>..vals</code> | int<br>content | (required) | Values in the list, head first.                                                   |
| <code>style</code>  | dictionary     | (:)        | Per-call style override merged over the defaults. See <a href="#">Section 8</a> . |

|                        |                                        |                    |                                                                                                                                                                                    |
|------------------------|----------------------------------------|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>pointer</code>   | <code>bool</code>                      | <code>false</code> | When <code>false</code> , draw one value cell per node with a forward arrow above and a backward arrow below. When <code>true</code> , draw <code>prev   data   next</code> cells. |
| <code>addresses</code> | <code>none</code> / <code>array</code> | <code>none</code>  | With <code>pointer: true</code> , one address label drawn under each node, in order.                                                                                               |
| <code>head</code>      | <code>bool</code>                      | <code>false</code> | Draw a “Head” arrow pointing at the first node.                                                                                                                                    |

### Nested keys for `style`

| Path                              | Type                    | Default                             | Description                                                                               |
|-----------------------------------|-------------------------|-------------------------------------|-------------------------------------------------------------------------------------------|
| <code>style.box-w</code>          | <code>float</code>      | <code>0.95</code>                   | Cell width.                                                                               |
| <code>style.box-h</code>          | <code>float</code>      | <code>0.7</code>                    | Cell height.                                                                              |
| <code>style.box-shape</code>      | <code>str</code>        | <code>"square"</code>               | <code>"square"</code> , <code>"rounded"</code> , or <code>"capsule"</code> .              |
| <code>style.box-gap</code>        | <code>float</code>      | <code>0.55</code>                   | Gap between cells or list nodes.                                                          |
| <code>style.box-stroke</code>     | <code>stroke</code>     | <code>0.6pt + rgb("#333333")</code> | Cell outline.                                                                             |
| <code>style.box-fill</code>       | <code>color</code>      | <code>white</code>                  | Default cell fill.                                                                        |
| <code>style.ptr-fill</code>       | <code>color</code>      | <code>rgb("#D7ECC9")</code>         | Next-pointer cell fill for <code>linked-list(pointer: true)</code> .                      |
| <code>style.prev-ptr-fill</code>  | <code>color</code>      | <code>rgb("#D7ECC9")</code>         | Previous-pointer cell fill for <code>doubly-linked-list(pointer: true)</code> .           |
| <code>style.next-ptr-fill</code>  | <code>color</code>      | <code>rgb("#D7ECC9")</code>         | Next-pointer cell fill for <code>doubly-linked-list(pointer: true)</code> .               |
| <code>style.node-text</code>      | <code>dictionary</code> | <code>(size: 9pt)</code>            | Cell text dictionary. See nested keys below.                                              |
| <code>style.value-text</code>     | <code>dictionary</code> | <code>inherits node-text</code>     | Value text overrides.                                                                     |
| <code>style.label-text</code>     | <code>dictionary</code> | <code>(fill: rgb("#555555"))</code> | Head, address, front, and rear label text dictionary. See nested keys below.              |
| <code>style.pointer-text</code>   | <code>dictionary</code> | <code>inherits label-text</code>    | Pointer, head, address, front, and rear typography.                                       |
| <code>style.operation-text</code> | <code>dictionary</code> | <code>(size: 8pt)</code>            | Operation-arrow caption typography.                                                       |
| <code>style.scale</code>          | <code>float</code>      | <code>1.0</code>                    | Uniform scale on <code>.diagram</code> , <code>.before</code> , and <code>.after</code> . |
| <code>style.diff-colors</code>    | <code>bool</code>       | <code>true</code>                   | <code>false</code> keeps operation marks but uses normal fills.                           |

### Nested keys for `style.node-text`

| Path                               | Type                | Default               | Description |
|------------------------------------|---------------------|-----------------------|-------------|
| <code>style.node-text.size</code>  | <code>length</code> | <code>inherits</code> | Text size.  |
| <code>style.node-text.color</code> | <code>color</code>  | <code>inherits</code> | Text color. |

|                                       |             |          |                                                |
|---------------------------------------|-------------|----------|------------------------------------------------|
| <code>style.node-text.rotation</code> | angle       | 0deg     | Text rotation applied by the diagram renderer. |
| <code>style.node-text.font</code>     | str / array | inherits | Typst text font selection.                     |
| <code>style.node-text.weight</code>   | str / int   | inherits | Typst text weight.                             |

#### Nested keys for `style.label-text`

| Path                                   | Type        | Default                                  | Description                                                                                          |
|----------------------------------------|-------------|------------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>style.label-text.size</code>     | length      | 85% of <code>style.node-text.size</code> | Label text size.                                                                                     |
| <code>style.label-text.color</code>    | color       | <code>rgb("#555555")</code>              | Label text color.                                                                                    |
| <code>style.label-text.rotation</code> | angle / str | 0deg                                     | Label rotation. Tree and graph edge labels also accept <code>"edge"</code> to follow the edge angle. |
| <code>style.label-text.font</code>     | str / array | inherits                                 | Typst text font selection.                                                                           |
| <code>style.label-text.weight</code>   | str / int   | inherits                                 | Typst text weight.                                                                                   |

#### Nested keys for `diff-highlight` dictionaries

| Path                                   | Type               | Default                        | Description                                                                                                                                                                         |
|----------------------------------------|--------------------|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>style.new-style</code>           | color / dictionary | <code>rgb("#C8E6C9")</code>    | Node/cell added by an operation. A color is shorthand for <code>(fill: color)</code> .                                                                                              |
| <code>style.path-style</code>          | color / dictionary | <code>rgb("#FFE9A8")</code>    | Traversal or sift path.                                                                                                                                                             |
| <code>style.remove-style</code>        | color / dictionary | <code>rgb("#FFCDD2")</code>    | Node/cell removed by an operation.                                                                                                                                                  |
| <code>style.rotate-style</code>        | color / dictionary | <code>rgb("#BBDEFB")</code>    | AVL rotation nodes and visible subtree roots.                                                                                                                                       |
| <code>style.*-style.fill</code>        | color              | mark default                   | Fill for the marked node/cell.                                                                                                                                                      |
| <code>style.*-style.shape</code>       | str                | <code>style.node-shape</code>  | Marked tree/heap node shape: <code>"circle"</code> , <code>"square"</code> , <code>"rounded"</code> , <code>"capsule"</code> , <code>"diamond"</code> , or <code>"hexagon"</code> . |
| <code>style.*-style.stroke</code>      | stroke             | normal stroke                  | Marked node/cell outline.                                                                                                                                                           |
| <code>style.*-style.node-radius</code> | float              | <code>style.node-radius</code> | Marked tree/heap node radius.                                                                                                                                                       |
| <code>style.*-style.text</code>        | dictionary         | <code>style.node-text</code>   | Marked node/cell text overrides. Accepts <code>size</code> , <code>color</code> , <code>font</code> , and <code>weight</code> .                                                     |

| Field                 | Call                                      | Effect                                                          |
|-----------------------|-------------------------------------------|-----------------------------------------------------------------|
| <code>.diagram</code> | n/a                                       | The rendered list.                                              |
| <code>.prepend</code> | <code>(obj.prepend)(v)</code>             | Insert <code>v</code> at the head. Returns a step.              |
| <code>.insert</code>  | <code>(obj.insert)(v, index: none)</code> | Append by default, or insert <code>v</code> at an index.        |
| <code>.delete</code>  | <code>(obj.delete)(v)</code>              | Remove the first node equal to <code>v</code> . Returns a step. |

|                         |                                     |                                                                                       |
|-------------------------|-------------------------------------|---------------------------------------------------------------------------------------|
| <code>.delete-at</code> | <code>(obj.delete-at)(index)</code> | Remove the node at <code>index</code> . Returns a step.                               |
| <code>.search</code>    | <code>(obj.search)(v)</code>        | Highlight the traversed prefix and expose <code>.found</code> / <code>.index</code> . |

```
1 #doubly-linked-list(3, 1, 4,
  head: true).diagram
```

```
1 #doubly-linked-list(15, 3, 17, 90, pointer: true, head: true,
2   addresses: ("3200", "3600", "4000", "4400"),
3   style: (
4     prev-ptr-fill: rgb("#FDE2E4"),
5     next-ptr-fill: rgb("#D7ECC9"),
6   )).diagram
```

Argument coverage: doubly linked pointer cells have separate previous and next pointer fills, plus the same address, head, label, and mark styling as `linked-list` .

```
1 #let xs = doubly-linked-list(
2   15, 3, 17,
3   pointer: true,
4   head: true,
5   addresses: ("3200", "3600", "4000"),
6   style: (
7     box-w: 0.82,
8     box-h: 0.62,
9     box-gap: 0.5,
10    box-fill: rgb("#FFDF7"),
11    box-stroke: 0.8pt + rgb("#6D4C41"),
12    prev-ptr-fill: rgb("#FDE2E4"),
13    next-ptr-fill: rgb("#D7ECC9"),
14    node-text: (size: 8.5pt, weight: "bold"),
15    label-text: (size: 7pt, color: rgb("#6D4C41")),
16    path-style: (fill: rgb("#FFE9A8"), stroke: 1pt + rgb("#F9A825")),
17    remove-style: (fill: rgb("#FFCDD2"), stroke: 1pt + rgb("#C62828")),
18    scale : 0.8,
19  ),
20 )
21 #(xs.delete)(3).diagram
```

### 3.5 `stack()`

Full signature:

```
1 #stack(..vals, style: (:), top-label: auto, language: "en", messages: (:))
```

**t** Typst

| Argument               | Type            | Default    | Description                                                                                                             |
|------------------------|-----------------|------------|-------------------------------------------------------------------------------------------------------------------------|
| <code>..vals</code>    | int<br>content  | (required) | Values in the stack; the <b>first</b> argument is the top.                                                              |
| <code>style</code>     | dictionary      | (:)        | Per-call style override merged over the defaults. See <a href="#">Section 8</a> .                                       |
| <code>top-label</code> | content<br>auto | auto       | Label beside the top cell. <b>auto</b> uses the localized <code>stack.top</code> default ( <a href="#">Section 7</a> ). |

#### Nested keys for `style`

| Path                              | Type       | Default                 | Description                                                                               |
|-----------------------------------|------------|-------------------------|-------------------------------------------------------------------------------------------|
| <code>style.box-w</code>          | float      | 0.95                    | Cell width.                                                                               |
| <code>style.box-h</code>          | float      | 0.7                     | Cell height.                                                                              |
| <code>style.box-shape</code>      | str        | "square"                | "square" , "rounded" , or "capsule" .                                                     |
| <code>style.box-gap</code>        | float      | 0                       | Gap between cells or list nodes.                                                          |
| <code>style.box-stroke</code>     | stroke     | 0.6pt + rgb("#333333")  | Cell outline.                                                                             |
| <code>style.box-fill</code>       | color      | white                   | Default cell fill.                                                                        |
| <code>style.ptr-fill</code>       | color      | rgb("#D7ECC9")          | Next-pointer cell fill for <code>linked-list(pointer: true)</code> .                      |
| <code>style.prev-ptr-fill</code>  | color      | rgb("#D7ECC9")          | Previous-pointer cell fill for <code>doubly-linked-list(pointer: true)</code> .           |
| <code>style.next-ptr-fill</code>  | color      | rgb("#D7ECC9")          | Next-pointer cell fill for <code>doubly-linked-list(pointer: true)</code> .               |
| <code>style.node-text</code>      | dictionary | (size: 9pt)             | Cell text dictionary. See nested keys below.                                              |
| <code>style.value-text</code>     | dictionary | inherits node-text      | Value text overrides.                                                                     |
| <code>style.label-text</code>     | dictionary | (fill: "#555555") rgb() | Head, address, front, and rear label text dictionary. See nested keys below.              |
| <code>style.pointer-text</code>   | dictionary | inherits label-text     | Pointer, head, address, front, and rear typography.                                       |
| <code>style.operation-text</code> | dictionary | (size: 8pt)             | Operation-arrow caption typography.                                                       |
| <code>style.scale</code>          | float      | 1.0                     | Uniform scale on <code>.diagram</code> , <code>.before</code> , and <code>.after</code> . |
| <code>style.diff-colors</code>    | bool       | true                    | <b>false</b> keeps operation marks but uses normal fills.                                 |

#### Nested keys for `style.node-text`

| Path                                  | Type        | Default  | Description                                    |
|---------------------------------------|-------------|----------|------------------------------------------------|
| <code>style.node-text.size</code>     | length      | inherits | Text size.                                     |
| <code>style.node-text.color</code>    | color       | inherits | Text color.                                    |
| <code>style.node-text.rotation</code> | angle       | 0deg     | Text rotation applied by the diagram renderer. |
| <code>style.node-text.font</code>     | str / array | inherits | Typst text font selection.                     |
| <code>style.node-text.weight</code>   | str / int   | inherits | Typst text weight.                             |

#### Nested keys for `style.label-text`

| Path                                   | Type        | Default                                  | Description                                                                                          |
|----------------------------------------|-------------|------------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>style.label-text.size</code>     | length      | 85% of <code>style.node-text.size</code> | Label text size.                                                                                     |
| <code>style.label-text.color</code>    | color       | <code>rgb("#555555")</code>              | Label text color.                                                                                    |
| <code>style.label-text.rotation</code> | angle / str | 0deg                                     | Label rotation. Tree and graph edge labels also accept <code>"edge"</code> to follow the edge angle. |
| <code>style.label-text.font</code>     | str / array | inherits                                 | Typst text font selection.                                                                           |
| <code>style.label-text.weight</code>   | str / int   | inherits                                 | Typst text weight.                                                                                   |

#### Nested keys for diff-highlight dictionaries

| Path                                   | Type               | Default                        | Description                                                                                                                                                                         |
|----------------------------------------|--------------------|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>style.new-style</code>           | color / dictionary | <code>rgb("#C8E6C9")</code>    | Node/cell added by an operation. A color is shorthand for <code>(fill: color)</code> .                                                                                              |
| <code>style.path-style</code>          | color / dictionary | <code>rgb("#FFE9A8")</code>    | Traversal or sift path.                                                                                                                                                             |
| <code>style.remove-style</code>        | color / dictionary | <code>rgb("#FFCDD2")</code>    | Node/cell removed by an operation.                                                                                                                                                  |
| <code>style.rotate-style</code>        | color / dictionary | <code>rgb("#BBDEFB")</code>    | AVL rotation nodes and visible subtree roots.                                                                                                                                       |
| <code>style.*-style.fill</code>        | color              | mark default                   | Fill for the marked node/cell.                                                                                                                                                      |
| <code>style.*-style.shape</code>       | str                | <code>style.node-shape</code>  | Marked tree/heap node shape: <code>"circle"</code> , <code>"square"</code> , <code>"rounded"</code> , <code>"capsule"</code> , <code>"diamond"</code> , or <code>"hexagon"</code> . |
| <code>style.*-style.stroke</code>      | stroke             | normal stroke                  | Marked node/cell outline.                                                                                                                                                           |
| <code>style.*-style.node-radius</code> | float              | <code>style.node-radius</code> | Marked tree/heap node radius.                                                                                                                                                       |
| <code>style.*-style.text</code>        | dictionary         | <code>style.node-text</code>   | Marked node/cell text overrides. Accepts <code>size</code> , <code>color</code> , <code>font</code> , and <code>weight</code> .                                                     |

| Field                 | Call | Effect              |
|-----------------------|------|---------------------|
| <code>.diagram</code> | n/a  | The rendered stack. |

|                    |                            |                                                   |
|--------------------|----------------------------|---------------------------------------------------|
| <code>.push</code> | <code>(obj.push)(v)</code> | Push <code>v</code> onto the top. Returns a step. |
| <code>.pop</code>  | <code>(obj.pop)()</code>   | Pop the top value (no argument). Returns a step.  |

1

`#stack(9, 7, 2).diagram`

Typst

9

7

2

top

1

`#stack(9, 7, 2, top-label: [Peek]).diagram`

Typst

9

7

2

Peek

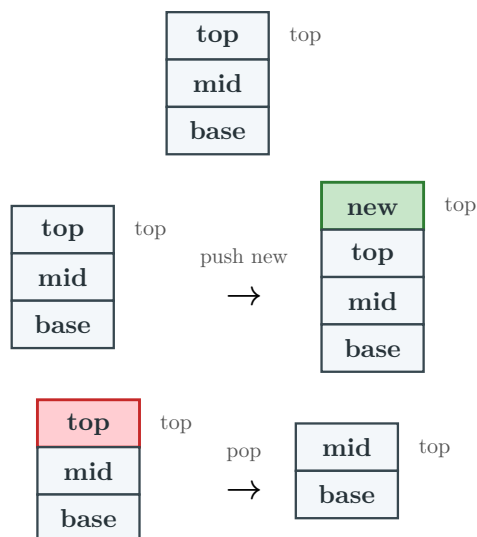
Argument coverage: stacks use the linear [style](#) keys and operation marks for `.push` / `.pop` .

```

1  #let s = stack(
2    [top], [mid], [base],
3    style: (
4      box-w: 1.25,
5      box-h: 0.58,
6      box-gap: 0,
7      box-fill: rgb("#F3F7FA"),
8      box-stroke: 0.8pt + rgb("#37474F"),
9      node-text: (size: 9pt, color: rgb("#263238"), weight: "bold"),
10     scale: 1.08,
11     new-style: (fill: rgb("#C8E6C9"), stroke: 1pt + rgb("#2E7D32")),
12     remove-style: (fill: rgb("#FFCDD2"), stroke: 1pt + rgb("#C62828")),
13   ),
14 )
15 #std.stack(spacing: 1em, s.diagram, (s.push)([new]).diagram, (s.pop)().diagram)

```

**t** Typst



### 3.6 queue()

Full signature:

```

1  #queue(
2    ..vals,
3    style: (:),
4    enqueue: none,
5    dequeue: none,
6    front-label: auto,
7    rear-label: auto,
8    language: "en",
9    messages: (:),
10 )

```

**t** Typst

| Argument                 | Type                                                        | Default           | Description                                                                                                                                                                                 |
|--------------------------|-------------------------------------------------------------|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>..vals</code>      | <code>int</code> / <code>content</code>                     | (required)        | Values in the queue; the <b>first</b> argument is the front.                                                                                                                                |
| <code>style</code>       | <code>dictionary</code>                                     | <code>(:)</code>  | Per-call style override merged over the defaults. See <a href="#">Section 8</a> .                                                                                                           |
| <code>enqueue</code>     | <code>none</code> / <code>int</code> / <code>content</code> | <code>none</code> | Draw one extra element entering at the rear, with an arrow, in the same frame.                                                                                                              |
| <code>dequeue</code>     | <code>none</code> / <code>int</code> / <code>content</code> | <code>none</code> | Draw the front element leaving to the left, with an arrow, in the same frame.                                                                                                               |
| <code>front-label</code> | <code>content</code> / <code>auto</code>                    | <code>auto</code> | Label over the front cell; <code>auto</code> uses the localized <code>queue.front</code> default ( <a href="#">Section 7</a> ). Combined with <code>rear-label</code> for a one-cell queue. |
| <code>rear-label</code>  | <code>content</code> / <code>auto</code>                    | <code>auto</code> | Label over the rear cell; <code>auto</code> uses the localized <code>queue.rear</code> default ( <a href="#">Section 7</a> ). Combined with <code>front-label</code> for a one-cell queue.  |

### Nested keys for `style`

| Path                              | Type                    | Default                             | Description                                                                     |
|-----------------------------------|-------------------------|-------------------------------------|---------------------------------------------------------------------------------|
| <code>style.box-w</code>          | <code>float</code>      | <code>0.95</code>                   | Cell width.                                                                     |
| <code>style.box-h</code>          | <code>float</code>      | <code>0.7</code>                    | Cell height.                                                                    |
| <code>style.box-shape</code>      | <code>str</code>        | <code>"square"</code>               | <code>"square"</code> , <code>"rounded"</code> , or <code>"capsule"</code> .    |
| <code>style.box-gap</code>        | <code>float</code>      | <code>0.55</code>                   | Gap between cells or list nodes.                                                |
| <code>style.box-stroke</code>     | <code>stroke</code>     | <code>0.6pt + rgb("#333333")</code> | Cell outline.                                                                   |
| <code>style.box-fill</code>       | <code>color</code>      | <code>white</code>                  | Default cell fill.                                                              |
| <code>style.ptr-fill</code>       | <code>color</code>      | <code>rgb("#D7ECC9")</code>         | Next-pointer cell fill for <code>linked-list(pointer: true)</code> .            |
| <code>style.prev-ptr-fill</code>  | <code>color</code>      | <code>rgb("#D7ECC9")</code>         | Previous-pointer cell fill for <code>doubly-linked-list(pointer: true)</code> . |
| <code>style.next-ptr-fill</code>  | <code>color</code>      | <code>rgb("#D7ECC9")</code>         | Next-pointer cell fill for <code>doubly-linked-list(pointer: true)</code> .     |
| <code>style.node-text</code>      | <code>dictionary</code> | <code>(size: 9pt)</code>            | Cell text dictionary. See nested keys below.                                    |
| <code>style.value-text</code>     | <code>dictionary</code> | <code>inherits node-text</code>     | Value text overrides.                                                           |
| <code>style.label-text</code>     | <code>dictionary</code> | <code>(fill: "#555555") rgb(</code> | Head, address, front, and rear label text dictionary. See nested keys below.    |
| <code>style.pointer-text</code>   | <code>dictionary</code> | <code>inherits label-text</code>    | Pointer, head, address, front, and rear typography.                             |
| <code>style.operation-text</code> | <code>dictionary</code> | <code>(size: 8pt)</code>            | Operation-arrow caption typography.                                             |

|                                |       |      |                                                                                           |
|--------------------------------|-------|------|-------------------------------------------------------------------------------------------|
| <code>style.scale</code>       | float | 1.0  | Uniform scale on <code>.diagram</code> , <code>.before</code> , and <code>.after</code> . |
| <code>style.diff-colors</code> | bool  | true | <code>false</code> keeps operation marks but uses normal fills.                           |

#### Nested keys for `style.node-text`

| Path                                  | Type        | Default  | Description                                    |
|---------------------------------------|-------------|----------|------------------------------------------------|
| <code>style.node-text.size</code>     | length      | inherits | Text size.                                     |
| <code>style.node-text.color</code>    | color       | inherits | Text color.                                    |
| <code>style.node-text.rotation</code> | angle       | 0deg     | Text rotation applied by the diagram renderer. |
| <code>style.node-text.font</code>     | str / array | inherits | Typst text font selection.                     |
| <code>style.node-text.weight</code>   | str / int   | inherits | Typst text weight.                             |

#### Nested keys for `style.label-text`

| Path                                   | Type        | Default                                  | Description                                                                                          |
|----------------------------------------|-------------|------------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>style.label-text.size</code>     | length      | 85% of <code>style.node-text.size</code> | Label text size.                                                                                     |
| <code>style.label-text.color</code>    | color       | <code>rgb("#555555")</code>              | Label text color.                                                                                    |
| <code>style.label-text.rotation</code> | angle / str | 0deg                                     | Label rotation. Tree and graph edge labels also accept <code>"edge"</code> to follow the edge angle. |
| <code>style.label-text.font</code>     | str / array | inherits                                 | Typst text font selection.                                                                           |
| <code>style.label-text.weight</code>   | str / int   | inherits                                 | Typst text weight.                                                                                   |

#### Nested keys for diff-highlight dictionaries

| Path                                   | Type               | Default                        | Description                                                                                                                                                                         |
|----------------------------------------|--------------------|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>style.new-style</code>           | color / dictionary | <code>rgb("#C8E6C9")</code>    | Node/cell added by an operation. A color is shorthand for <code>(fill: color)</code> .                                                                                              |
| <code>style.path-style</code>          | color / dictionary | <code>rgb("#FFE9A8")</code>    | Traversal or sift path.                                                                                                                                                             |
| <code>style.remove-style</code>        | color / dictionary | <code>rgb("#FFCDD2")</code>    | Node/cell removed by an operation.                                                                                                                                                  |
| <code>style.rotate-style</code>        | color / dictionary | <code>rgb("#BBDEFB")</code>    | AVL rotation nodes and visible subtree roots.                                                                                                                                       |
| <code>style.*-style.fill</code>        | color              | mark default                   | Fill for the marked node/cell.                                                                                                                                                      |
| <code>style.*-style.shape</code>       | str                | <code>style.node-shape</code>  | Marked tree/heap node shape: <code>"circle"</code> , <code>"square"</code> , <code>"rounded"</code> , <code>"capsule"</code> , <code>"diamond"</code> , or <code>"hexagon"</code> . |
| <code>style.*-style.stroke</code>      | stroke             | normal stroke                  | Marked node/cell outline.                                                                                                                                                           |
| <code>style.*-style.node-radius</code> | float              | <code>style.node-radius</code> | Marked tree/heap node radius.                                                                                                                                                       |

|                                 |            |                              |                                                                                                                                 |
|---------------------------------|------------|------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <code>style.*-style.text</code> | dictionary | <code>style.node-text</code> | Marked node/cell text overrides. Accepts <code>size</code> , <code>color</code> , <code>font</code> , and <code>weight</code> . |
|---------------------------------|------------|------------------------------|---------------------------------------------------------------------------------------------------------------------------------|

| Field                 | Call                          | Effect                                                 |
|-----------------------|-------------------------------|--------------------------------------------------------|
| <code>.diagram</code> | n/a                           | The rendered queue.                                    |
| <code>.enqueue</code> | <code>(obj.enqueue)(v)</code> | Enqueue <code>v</code> at the rear. Returns a step.    |
| <code>.dequeue</code> | <code>(obj.dequeue)()</code>  | Dequeue the front value (no argument). Returns a step. |

1 `#queue(3, 8, 5, 1).diagram`

Typst

FrontRear

|   |   |   |   |
|---|---|---|---|
| 3 | 8 | 5 | 1 |
|---|---|---|---|

`enqueue` / `dequeue` draw a single-frame arrow, independent of the object's own `.enqueue` / `.dequeue` methods:

1 `#queue(3, 4, 5, 6, 7, 8, enqueue: 9, dequeue: 2,`

Typst

2 `front-label: [Front / Head], rear-label: [Back / Tail / Rear]).diagram`

Front / HeadBack / Tail / Rear

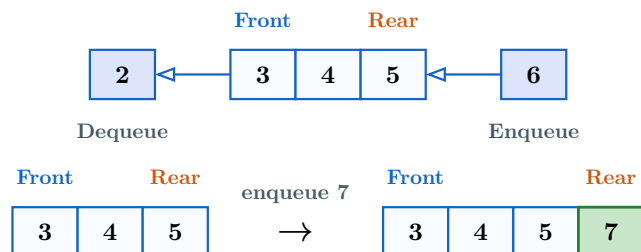
|         |   |   |   |   |   |   |   |   |         |
|---------|---|---|---|---|---|---|---|---|---------|
| 2       | ← | 3 | 4 | 5 | 6 | 7 | 8 | ← | 9       |
| Dequeue |   |   |   |   |   |   |   |   | Enqueue |

Argument coverage: queue examples can combine the single-frame `enqueue` / `dequeue` arrows, custom front/rear labels, label text styling, and operation mark styles.

```

1  #let q = queue(
2    3, 4, 5,
3    enqueue: 6,
4    dequeue: 2,
5    front-label: text(fill: rgb("#1565C0"))[*Front*],
6    rear-label: text(fill: rgb("#C75B12"))[*Rear*],
7    style: (
8      box-w: 0.82,
9      box-h: 0.62,
10     box-gap: 0.35,
11     box-fill: rgb("#F8FBFF"),
12     box-stroke: 0.8pt + rgb("#1565C0"),
13     node-text: (size: 9pt, weight: "bold"),
14     label-text: (size: 7.5pt, color: rgb("#455A64"), weight: "bold"),
15     scale: 1.05,
16     new-style: (fill: rgb("#C8E6C9"), stroke: 1pt + rgb("#2E7D32")),
17     remove-style: (fill: rgb("#FFCDD2"), stroke: 1pt + rgb("#C62828")),
18   ),
19 )
20 #std.stack(spacing: 1em, q.diagram, (q.enqueue)(7).diagram)

```



### 3.7 skip-list()

Full signature:

```
1 #skip-list(
2   ..vals,
3   style: (:),
4   decision-fn: default-decision-fn,
5   level-spacing: 1.4,
6   max-level: 4,
7   language: "en",
8   messages: (:),
9 )
```

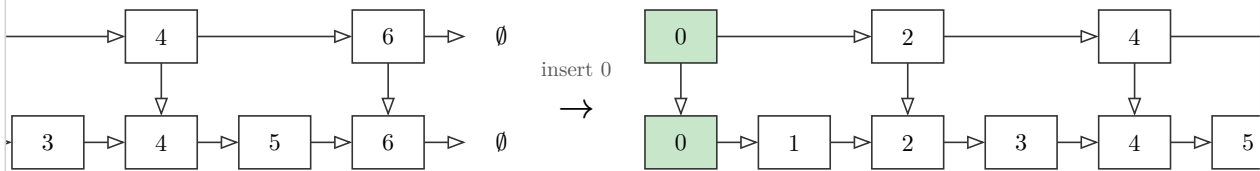
 Typst

`..vals` must be strictly ascending, with no duplicate values; `skip-list` validates this but does not sort them for you. Values must be all numbers (integers and floats may be mixed) or all strings. Levels above the base row are express lanes: a node's height is decided once, when it is built or inserted, and never changes afterward, so later inserts/deletes elsewhere never reshuffle it.

The smallest value is always drawn spanning every level, exactly like the sentinel head node in a standard skip list — it's what search starts from at the top level, so it must always have a way in. Its own `decision-fn`-assigned height still exists underneath; it only takes over once `insert` gives it a smaller value to make way for.

```
1 #let l = skip-list(1, 2, 3, 4, 5, 6)
2 #(l.insert)(0).diagram
```

 Typst



| Argument                 | Type                                                      | Default                          | Description                                                                                                                                                                                                                                                                                                            |
|--------------------------|-----------------------------------------------------------|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>..vals</code>      | <code>int</code><br><code>float</code> / <code>str</code> | (required)                       | Values in strictly ascending order, without duplicates. All values must be numbers (integers and floats may be mixed) or all strings. <code>insert</code> compares values to place them, so arbitrary <code>content</code> isn't supported here.                                                                       |
| <code>style</code>       | dictionary                                                | <code>(:)</code>                 | Per-call style override merged over the defaults. See <a href="#">Section 8</a> .                                                                                                                                                                                                                                      |
| <code>decision-fn</code> | function                                                  | <code>default-decision-fn</code> | <code>(level, value) =&gt; bool</code> , called with <code>level</code> starting at 1 and increasing; a node keeps climbing while this returns <code>true</code> . Typst has no RNG, so the default hashes <code>value</code> deterministically instead of flipping a coin — same value, same height, every recompile. |

|                            |                          |     |                                                                                                                                                                     |
|----------------------------|--------------------------|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>level-spacing</code> | <code>int / float</code> | 1.4 | Positive vertical distance between level rows, in canvas units.                                                                                                     |
| <code>max-level</code>     | <code>int</code>         | 4   | Non-negative upper bound on the height <code>decision-fn</code> can assign automatically. <code>insert(..., level:)</code> can still exceed it when set explicitly. |

### Nested keys for `style`

| Path                              | Type                    | Default                              | Description                                                                               |
|-----------------------------------|-------------------------|--------------------------------------|-------------------------------------------------------------------------------------------|
| <code>style.box-w</code>          | <code>float</code>      | 0.95                                 | Cell width.                                                                               |
| <code>style.box-h</code>          | <code>float</code>      | 0.7                                  | Cell height.                                                                              |
| <code>style.box-shape</code>      | <code>str</code>        | "square"                             | "square" , "rounded" , or "capsule" .                                                     |
| <code>style.box-gap</code>        | <code>float</code>      | 0.55                                 | Gap between cells or list nodes.                                                          |
| <code>style.box-stroke</code>     | <code>stroke</code>     | 0.6pt + <code>rgb("#333333")</code>  | Cell outline.                                                                             |
| <code>style.box-fill</code>       | <code>color</code>      | white                                | Default cell fill.                                                                        |
| <code>style.ptr-fill</code>       | <code>color</code>      | <code>rgb("#D7ECC9")</code>          | Next-pointer cell fill for <code>linked-list(pointer: true)</code> .                      |
| <code>style.prev-ptr-fill</code>  | <code>color</code>      | <code>rgb("#D7ECC9")</code>          | Previous-pointer cell fill for <code>doubly-linked-list(pointer: true)</code> .           |
| <code>style.next-ptr-fill</code>  | <code>color</code>      | <code>rgb("#D7ECC9")</code>          | Next-pointer cell fill for <code>doubly-linked-list(pointer: true)</code> .               |
| <code>style.node-text</code>      | <code>dictionary</code> | (size: 9pt)                          | Cell text dictionary. See nested keys below.                                              |
| <code>style.value-text</code>     | <code>dictionary</code> | inherits <code>node-text</code>      | Value text overrides.                                                                     |
| <code>style.label-text</code>     | <code>dictionary</code> | (fill: <code>rgb("#555555")</code> ) | Head, address, front, and rear label text dictionary. See nested keys below.              |
| <code>style.pointer-text</code>   | <code>dictionary</code> | inherits <code>label-text</code>     | Pointer, head, address, front, and rear typography.                                       |
| <code>style.operation-text</code> | <code>dictionary</code> | (size: 8pt)                          | Operation-arrow caption typography.                                                       |
| <code>style.scale</code>          | <code>float</code>      | 1.0                                  | Uniform scale on <code>.diagram</code> , <code>.before</code> , and <code>.after</code> . |
| <code>style.diff-colors</code>    | <code>bool</code>       | true                                 | <code>false</code> keeps operation marks but uses normal fills.                           |

### Nested keys for `style.node-text`

| Path                                  | Type                | Default  | Description                                    |
|---------------------------------------|---------------------|----------|------------------------------------------------|
| <code>style.node-text.size</code>     | <code>length</code> | inherits | Text size.                                     |
| <code>style.node-text.color</code>    | <code>color</code>  | inherits | Text color.                                    |
| <code>style.node-text.rotation</code> | <code>angle</code>  | 0deg     | Text rotation applied by the diagram renderer. |

|                                     |             |          |                            |
|-------------------------------------|-------------|----------|----------------------------|
| <code>style.node-text.font</code>   | str / array | inherits | Typst text font selection. |
| <code>style.node-text.weight</code> | str / int   | inherits | Typst text weight.         |

### Nested keys for `style.label-text`

| Path                                   | Type        | Default                                  | Description                                                                                          |
|----------------------------------------|-------------|------------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>style.label-text.size</code>     | length      | 85% of <code>style.node-text.size</code> | Label text size.                                                                                     |
| <code>style.label-text.color</code>    | color       | <code>rgb("#555555")</code>              | Label text color.                                                                                    |
| <code>style.label-text.rotation</code> | angle / str | 0deg                                     | Label rotation. Tree and graph edge labels also accept <code>"edge"</code> to follow the edge angle. |
| <code>style.label-text.font</code>     | str / array | inherits                                 | Typst text font selection.                                                                           |
| <code>style.label-text.weight</code>   | str / int   | inherits                                 | Typst text weight.                                                                                   |

### Nested keys for diff-highlight dictionaries

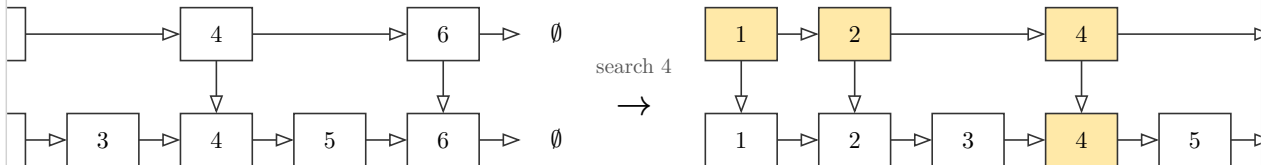
| Path                                   | Type               | Default                        | Description                                                                                                                                                                         |
|----------------------------------------|--------------------|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>style.new-style</code>           | color / dictionary | <code>rgb("#C8E6C9")</code>    | Node/cell added by an operation. A color is shorthand for <code>(fill: color)</code> .                                                                                              |
| <code>style.path-style</code>          | color / dictionary | <code>rgb("#FFE9A8")</code>    | Traversal or sift path.                                                                                                                                                             |
| <code>style.remove-style</code>        | color / dictionary | <code>rgb("#FFCDD2")</code>    | Node/cell removed by an operation.                                                                                                                                                  |
| <code>style.rotate-style</code>        | color / dictionary | <code>rgb("#BBDEFB")</code>    | AVL rotation nodes and visible subtree roots.                                                                                                                                       |
| <code>style.*-style.fill</code>        | color              | mark default                   | Fill for the marked node/cell.                                                                                                                                                      |
| <code>style.*-style.shape</code>       | str                | <code>style.node-shape</code>  | Marked tree/heap node shape: <code>"circle"</code> , <code>"square"</code> , <code>"rounded"</code> , <code>"capsule"</code> , <code>"diamond"</code> , or <code>"hexagon"</code> . |
| <code>style.*-style.stroke</code>      | stroke             | normal stroke                  | Marked node/cell outline.                                                                                                                                                           |
| <code>style.*-style.node-radius</code> | float              | <code>style.node-radius</code> | Marked tree/heap node radius.                                                                                                                                                       |
| <code>style.*-style.text</code>        | dictionary         | <code>style.node-text</code>   | Marked node/cell text overrides. Accepts <code>size</code> , <code>color</code> , <code>font</code> , and <code>weight</code> .                                                     |

| Field                 | Call                                          | Effect                                                                                                                                                                                                       |
|-----------------------|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>.diagram</code> | n/a                                           | The rendered skip list.                                                                                                                                                                                      |
| <code>.search</code>  | <code>(obj.search)(key)</code>                | Highlight the top-down search path to <code>key</code> , or its predecessor when absent. Returns a step with <code>found</code> and <code>index</code> ( <code>false</code> / <code>none</code> for a miss). |
| <code>.insert</code>  | <code>(obj.insert)(value, level: auto)</code> | Insert a new <code>value</code> in sorted position. The value must be compatible with the existing values and not already present. <code>level: auto</code> assigns height with <code>decision-fn</code> an  |

|                      |                                  |                                                                                                         |
|----------------------|----------------------------------|---------------------------------------------------------------------------------------------------------|
|                      |                                  | explicit non-negative integer forces a specific tower height instead. Returns a step.                   |
| <code>.delete</code> | <code>(obj.delete)(value)</code> | Remove <code>value</code> from every level it spans at once. The value must be present. Returns a step. |

```
1 #let l = skip-list(1, 2, 3, 4, 5, 6)
2 #(l.search)(4).diagram
```

**t** Typst



A search miss is still a valid operation: it follows the express lanes to the predecessor where the key would be inserted. The returned step exposes `found` and `index` so the miss can be handled programmatically.

```
1 #let miss = (skip-list(1, 3, 5).search)(4)
2 #assert(not miss.found)
3 #assert.eq(miss.index, none)
4 #miss.diagram
```

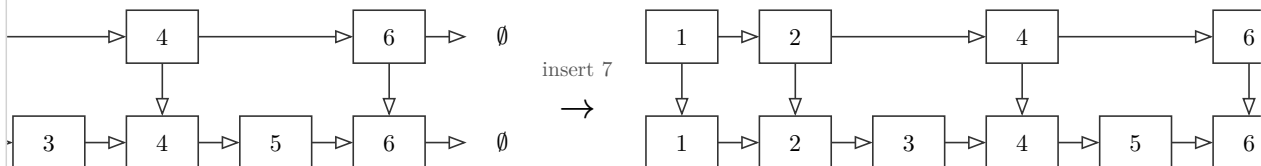
**t** Typst



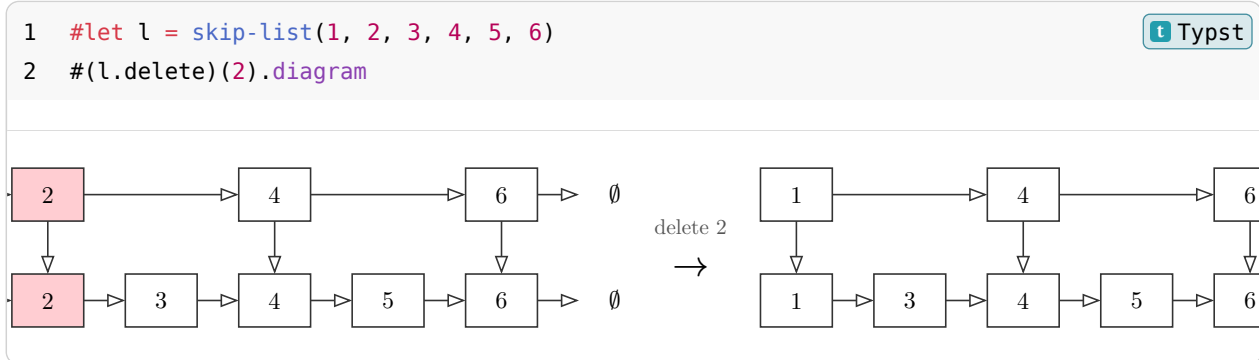
Insert only marks the node that was actually added (green); every other node's height stays exactly as it was, since `decision-fn` depends on the value alone, never on how many other values exist or where they sit.

```
1 #let l = skip-list(1, 2, 3, 4, 5, 6)
2 #(l.insert)(7).diagram
```

**t** Typst



Delete highlights the target (red) across every level it appears at, then removes it from all of them in the same step — a node is one physical tower, not several independent copies, so leaving it linked at only some levels would corrupt the structure.



### 3.8 array-view() and matrix()

Full signatures:

```
1 #array-view(..vals, style: (:), cell-customizations: (), pointers: ())
2 #matrix(rows, style: (:), cell-customizations: (), row-labels: none, column-labels: none)
```

These builders draw compact cell diagrams. They do not have operations yet; `cell-customizations` restyles individual array cells by index or matrix cells by `(row, column)` coordinate. `style.indices: (enabled: true)` draws zero-based array indices under the cells. `pointers` draws labelled arrows above array cells.

| Argument                         | Type               | Default                                 | Description                                                                                                                                                        |
|----------------------------------|--------------------|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>..vals</code>              | content            | (required for <code>array-view</code> ) | Array cells, left to right.                                                                                                                                        |
| <code>rows</code>                | array              | (required for <code>matrix</code> )     | Rows of cells, written as nested arrays such as <code>((1, 2), (3, 4))</code> .                                                                                    |
| <code>cell-customizations</code> | array / dictionary | ()                                      | Cell overrides. For arrays, use <code>((index, options), ...)</code> . For matrices, use <code>((row, col), options), ...)</code> .                                |
| <code>pointers</code>            | array              | ()                                      | <code>array-view</code> only. Labelled arrows above cells, each <code>(index: , label: , color: , text: )</code> . Markers sharing a cell spread across its width. |
| <code>row-labels</code>          | none / array       | none                                    | Matrix only. Labels drawn to the left of rows.                                                                                                                     |
| <code>column-labels</code>       | none / array       | none                                    | Matrix only. Labels drawn above columns.                                                                                                                           |
| <code>style</code>               | dictionary         | (:)                                     | Cell style override. Uses <code>box-*</code> , <code>node-text</code> , <code>label-text</code> , and diff-highlight style keys.                                   |

#### Nested keys for style

| Path                     | Type  | Default | Description |
|--------------------------|-------|---------|-------------|
| <code>style.box-w</code> | float | 0.95    | Cell width. |

|                                   |            |                                  |                                                                                           |
|-----------------------------------|------------|----------------------------------|-------------------------------------------------------------------------------------------|
| <code>style.box-h</code>          | float      | 0.7                              | Cell height.                                                                              |
| <code>style.box-shape</code>      | str        | "square"                         | "square" , "rounded" , or "capsule" .                                                     |
| <code>style.box-gap</code>        | float      | 0.55                             | Gap between cells or list nodes.                                                          |
| <code>style.box-stroke</code>     | stroke     | 0.6pt + rgb("#333333")           | Cell outline.                                                                             |
| <code>style.box-fill</code>       | color      | white                            | Default cell fill.                                                                        |
| <code>style.ptr-fill</code>       | color      | rgb("#D7ECC9")                   | Next-pointer cell fill for <code>linked-list(pointer: true)</code> .                      |
| <code>style.prev-ptr-fill</code>  | color      | rgb("#D7ECC9")                   | Previous-pointer cell fill for <code>doubly-linked-list(pointer: true)</code> .           |
| <code>style.next-ptr-fill</code>  | color      | rgb("#D7ECC9")                   | Next-pointer cell fill for <code>doubly-linked-list(pointer: true)</code> .               |
| <code>style.node-text</code>      | dictionary | (size: 9pt)                      | Cell text dictionary. See nested keys below.                                              |
| <code>style.value-text</code>     | dictionary | inherits <code>node-text</code>  | Value text overrides.                                                                     |
| <code>style.label-text</code>     | dictionary | (fill: rgb("#555555"))           | Head, address, front, and rear label text dictionary. See nested keys below.              |
| <code>style.pointer-text</code>   | dictionary | inherits <code>label-text</code> | Pointer, head, address, front, and rear typography.                                       |
| <code>style.operation-text</code> | dictionary | (size: 8pt)                      | Operation-arrow caption typography.                                                       |
| <code>style.scale</code>          | float      | 1.0                              | Uniform scale on <code>.diagram</code> , <code>.before</code> , and <code>.after</code> . |
| <code>style.diff-colors</code>    | bool       | true                             | <code>false</code> keeps operation marks but uses normal fills.                           |

#### Nested keys for `style.node-text`

| Path                                  | Type        | Default  | Description                                    |
|---------------------------------------|-------------|----------|------------------------------------------------|
| <code>style.node-text.size</code>     | length      | inherits | Text size.                                     |
| <code>style.node-text.color</code>    | color       | inherits | Text color.                                    |
| <code>style.node-text.rotation</code> | angle       | 0deg     | Text rotation applied by the diagram renderer. |
| <code>style.node-text.font</code>     | str / array | inherits | Typst text font selection.                     |
| <code>style.node-text.weight</code>   | str / int   | inherits | Typst text weight.                             |

#### Nested keys for `style.label-text`

| Path                                | Type   | Default                                  | Description       |
|-------------------------------------|--------|------------------------------------------|-------------------|
| <code>style.label-text.size</code>  | length | 85% of <code>style.node-text.size</code> | Label text size.  |
| <code>style.label-text.color</code> | color  | rgb("#555555")                           | Label text color. |

|                                        |                          |                       |                                                                                                      |
|----------------------------------------|--------------------------|-----------------------|------------------------------------------------------------------------------------------------------|
| <code>style.label-text.rotation</code> | <code>angle / str</code> | <code>0deg</code>     | Label rotation. Tree and graph edge labels also accept <code>"edge"</code> to follow the edge angle. |
| <code>style.label-text.font</code>     | <code>str / array</code> | <code>inherits</code> | Typst text font selection.                                                                           |
| <code>style.label-text.weight</code>   | <code>str / int</code>   | <code>inherits</code> | Typst text weight.                                                                                   |

### Nested keys for diff-highlight dictionaries

| Path                                   | Type                            | Default                        | Description                                                                                                                                                                         |
|----------------------------------------|---------------------------------|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>style.new-style</code>           | <code>color / dictionary</code> | <code>rgb("#C8E6C9")</code>    | Node/cell added by an operation. A color is shorthand for <code>(fill: color)</code> .                                                                                              |
| <code>style.path-style</code>          | <code>color / dictionary</code> | <code>rgb("#FFE9A8")</code>    | Traversal or sift path.                                                                                                                                                             |
| <code>style.remove-style</code>        | <code>color / dictionary</code> | <code>rgb("#FFCDD2")</code>    | Node/cell removed by an operation.                                                                                                                                                  |
| <code>style.rotate-style</code>        | <code>color / dictionary</code> | <code>rgb("#BBDEFB")</code>    | AVL rotation nodes and visible subtree roots.                                                                                                                                       |
| <code>style.*-style.fill</code>        | <code>color</code>              | <code>mark default</code>      | Fill for the marked node/cell.                                                                                                                                                      |
| <code>style.*-style.shape</code>       | <code>str</code>                | <code>style.node-shape</code>  | Marked tree/heap node shape: <code>"circle"</code> , <code>"square"</code> , <code>"rounded"</code> , <code>"capsule"</code> , <code>"diamond"</code> , or <code>"hexagon"</code> . |
| <code>style.*-style.stroke</code>      | <code>stroke</code>             | <code>normal stroke</code>     | Marked node/cell outline.                                                                                                                                                           |
| <code>style.*-style.node-radius</code> | <code>float</code>              | <code>style.node-radius</code> | Marked tree/heap node radius.                                                                                                                                                       |
| <code>style.*-style.text</code>        | <code>dictionary</code>         | <code>style.node-text</code>   | Marked node/cell text overrides. Accepts <code>size</code> , <code>color</code> , <code>font</code> , and <code>weight</code> .                                                     |

### Nested keys for `style.indices`

| Path                               | Type                      | Default                             | Description                                                                          |
|------------------------------------|---------------------------|-------------------------------------|--------------------------------------------------------------------------------------|
| <code>style.indices.enabled</code> | <code>bool</code>         | <code>false</code>                  | Draw index labels below array cells.                                                 |
| <code>style.indices.labels</code>  | <code>auto / array</code> | <code>auto</code>                   | <code>auto</code> draws zero-based numeric indices; an array supplies custom labels. |
| <code>style.indices.offset</code>  | <code>tuple</code>        | <code>(0, -0.28)</code>             | Canvas offset from the bottom-center index position.                                 |
| <code>style.indices.size</code>    | <code>length</code>       | <code>style.label-text.size</code>  | Index text size.                                                                     |
| <code>style.indices.color</code>   | <code>color</code>        | <code>style.label-text.color</code> | Index text color.                                                                    |
| <code>style.indices.font</code>    | <code>str / array</code>  | <code>inherits</code>               | Typst text font selection.                                                           |
| <code>style.indices.weight</code>  | <code>str / int</code>    | <code>inherits</code>               | Typst text weight.                                                                   |

### Nested keys for `cell-customizations`

| Path                                                | Type       | Default                      | Description                                                                    |
|-----------------------------------------------------|------------|------------------------------|--------------------------------------------------------------------------------|
| <code>cell-customizations[]</code>                  | tuple      | n/a                          | One ( <code>cell</code> , <code>options</code> ) tuple.                        |
| <code>cell-customizations[][.cell]</code>           | content    | required                     | Array index, or matrix coordinate ( <code>row</code> , <code>column</code> ) . |
| <code>cell-customizations[][.options]</code>        | dictionary | required                     | Per-cell drawing options.                                                      |
| <code>cell-customizations[][.options.fill]</code>   | color      | normal fill                  | Cell fill.                                                                     |
| <code>cell-customizations[][.options.stroke]</code> | stroke     | normal stroke                | Cell outline.                                                                  |
| <code>cell-customizations[][.options.text]</code>   | dictionary | <code>style.node-text</code> | Text style merged into the cell text.                                          |

| Field                 | Call | Effect                        |
|-----------------------|------|-------------------------------|
| <code>.diagram</code> | n/a  | The rendered array or matrix. |

```

1  #array-view(
2      4, 1, 7, 3, 9,
3      style: (
4          indices: (
5              enabled: true,
6              color: rgb("#455A64"),
7              weight: "bold",
8          ),
9      ),
10     cell-customizations: (
11         (1, (fill: rgb("#FFF3BF"), stroke: 1pt + rgb("#F59F00"))),
12         (2, (fill: rgb("#D3F9D8"), stroke: 1pt + rgb("#2B8A3E"))),
13     ),
14 ).diagram
15
16 #matrix(
17     ((0, 1, 0), (1, 0, 1), (0, 1, 0)),
18     row-labels: ([A], [B], [C]),
19     column-labels: ([A], [B], [C]),
20     cell-customizations: (
21         ((0, 1), (fill: rgb("#E7F5FF"), stroke: 1pt + rgb("#1971C2"))),
22         ((1, 2), (fill: rgb("#D3F9D8"), stroke: 1pt + rgb("#2B8A3E"))),
23     ),
24 ).diagram

```

|   |   |   |   |   |   |
|---|---|---|---|---|---|
|   | 4 | 1 | 7 | 3 | 9 |
|   | 0 | 1 | 2 | 3 | 4 |
|   | A |   | B | C |   |
| A | 0 | 1 | 0 |   |   |
| B | 1 | 0 | 1 |   |   |
| C | 0 | 1 | 0 |   |   |

### 3.9 hash-table()

`hash-table` supports separate chaining and linear probing while using the same cell/list styling and chainable operation steps as the other structures. Each chaining bucket points to its head entry or `none` with an arrow. Chain ends use the same compact `ø` terminator as linked lists. Chained entries default to a wider `style.box-w` of 1.5 so key/value pairs fit; an explicit `style.box-w` overrides it.

```

1  #hash-table(..entries, size: 7, collision: "chaining", hash: auto, style:
    (:), language: "en", messages: (:))

```

| Argument               | Type             | Default | Description                  |
|------------------------|------------------|---------|------------------------------|
| <code>..entries</code> | content<br>tuple | ( )     | Keys, or (key, value) pairs. |

|           |                 |            |                                                                                                              |
|-----------|-----------------|------------|--------------------------------------------------------------------------------------------------------------|
| size      | int             | 7          | Positive bucket/slot count.                                                                                  |
| collision | str             | "chaining" | "chaining" / "chain" or "linear" .                                                                           |
| hash      | auto / function | auto       | Custom <code>key =&gt; int</code> function. The default hashes integers and the UTF-8 bytes of other values. |
| style     | dictionary      | (:)        | Uses the existing box, value/index text, pointer text, scale, and mark-style keys.                           |

| Field    | Call                           | Effect                                                                        |
|----------|--------------------------------|-------------------------------------------------------------------------------|
| .diagram | n/a                            | Rendered buckets or probe table.                                              |
| .insert  | (obj.insert)(key, value: none) | Insert or update a key. Returns a step.                                       |
| .delete  | (obj.delete)(key)              | Delete a key; linear probing leaves a tombstone.                              |
| .search  | (obj.search)(key)              | Highlight traversed chain nodes or probe slots; exposes <code>.found</code> . |

```
1 #std.stack(dir: ltr, spacing: 1.5em,
2   hash-table(("Ada", 1), ("Grace", 2), size: 4).diagram,
3   hash-table(1, 6, 11, size: 5, collision: "linear").diagram,
4 )
```

```
graph LR
    subgraph Indices
        direction TB
        I0[0] --> N0[Grace: 2]
        I1[1] --> E1[∅]
        I2[2] --> N2[Ada: 1]
        I3[3] --> E3[∅]
    end
    N0 --> E0[∅]
    N2 --> E2[∅]
    subgraph ProbeTable
        direction LR
        P0[ ] --- P1[1] --- P2[6] --- P3[11] --- P4[ ]
        P0 --- I0_0[0]
        P1 --- I0_1[1]
        P2 --- I0_2[2]
        P3 --- I0_3[3]
        P4 --- I0_4[4]
    end
```

### 3.10 Sorting algorithms

Full signatures:

```

1 #merge-sort(array, order: "asc", labels: true, language: "en", messages: ()): t Typst
2 #merge-operation(left, right, order: "asc", pointers: true, labels: true, language:
  "en", messages: ()):
3 #partition-step(array, order: "asc", pivot: "middle", pointers: false, labels: true,
  language: "en", messages: ()):
4 #quick-sort(array, order: "asc", pivot: "last", labels: true, language: "en", messages:
  ()):
5 #bubble-sort(array, order: "asc", pointers: true, labels: true, compare: none, swap:
  none, language: "en", messages: ()):
6 #insertion-sort(array, order: "asc", pointers: true, labels: true, compare: none, swap:
  none, language: "en", messages: ()):
7 #selection-sort(array, order: "asc", pointers: true, labels: true,
  compare: none, current: none, minimum: none, swap: none, language: "en", messages:
  ()):
9 #sort-sequence(steps, columns: 3, gap: 1em, row-gap: 1em)
```

Sorting builders generate full traces from a single input array. Each returns `.diagram`, `.steps`, and `.result`. Use `.diagram` for the default wrapped trace, or pass `.steps` to `sort-sequence(...)` when you want different columns or spacing. Every trace records all comparison and swap steps. Each step keeps its label and array together when a page break is needed. Merge sort renders a connected tree: arrows lead from each array to its two divided arrays, then from each pair into its merged result. Curly braces identify the Divide, Merge, and Partition phases. Quicksort groups all active recursive partitions on the same row before expanding them together on the next row. Use `partition-step(..., pivot: "last")` for the detailed partition used by a last-pivot quicksort; `pointers: true` adds labelled cursor arrows, and each successful comparison includes the subsequent `i` advance as its own frame.

To style a trace, pass a styled `array-view(...)` in place of the bare array: its style (fills, text, and index configuration) is carried through every step of the trace. Bubble sort adds a pointer-free settled frame after each pass to highlight its green suffix, and selection sort adds one after each minimum swap to highlight its sorted prefix. Settled frames name the newly settled value, and selection scan labels list position, minimum, then item.

Merge operation, bubble, insertion, and selection sort show labelled arrows above their active positions by default. Pass `pointers: false` to hide them. Merge operation labels its left, right, and output cursors `i`, `j`, and `i+j` respectively. Each arrow sits over a marked cell, and several markers on one cell spread across its width. The arrows are additive: the role still fills its cells, so pointers layer on top of the fill. Each role ( `compare`, `swap`, and, for selection sort, `current` and `minimum` ) takes a `node-mark-style(...)` that restyles that role's fill, stroke, and text in either mode; the arrow colour comes from the mark's stroke, or its fill when no stroke is set. To keep a cell unfilled under a pointer, set the role's `fill` to `none`. Quick sort and merge sort render composite phase diagrams and do not take marker roles.

Pass `labels: false` to hide generated text captions while preserving the arrays, highlights, indices, and optional pointer arrows. This also hides merge's left/right/result headings, partition metadata, and composite phase labels, which lets you supply your own translated explanation alongside a trace.

Sorting traces work directly with Touying reveals: `.steps` is an ordered array of rendered frames, so it can be revealed one at a time.

```
1 #let trace = bubble-sort((5, 1, 4, 2))
2
3 #for (index, step) in trace.steps.enumerate() [
4   #only(index + 1)[#step.diagram]
5 ]
```

 Typst

`only(index + 1)` shows exactly one step per reveal in the same spot. Using `#pause` instead accumulates the steps, stacking every frame on screen at once.

| Argument                                                                               | Type                                                                                  | Default                                     | Description                                                                                                                                                                                                                                                                                                                                             |
|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>array</code>                                                                     | <code>array</code> / <code>array-view</code>                                          | (required)                                  | Values to sort from left to right, as a plain array or a styled <code>array-view(...)</code> whose style is reused for every step. Values must be comparable with <code>&lt;</code> / <code>&gt;</code> . <code>merge-operation</code> takes two such arguments ( <code>left</code> , <code>right</code> ); every other builder takes one.              |
| <code>order</code>                                                                     | <code>"asc"</code> / <code>"desc"</code>                                              | <code>"asc"</code>                          | Sort direction.                                                                                                                                                                                                                                                                                                                                         |
| <code>pivot</code>                                                                     | <code>"first"</code> / <code>"last"</code> / <code>int</code> / <code>"middle"</code> | <code>"last"</code> / <code>"middle"</code> | Quicksort selects an end of each subarray or a position index. <code>partition-step</code> accepts <code>middle</code> or <code>last</code> use <code>last</code> for the detailed quicksort partition trace.                                                                                                                                           |
| <code>pointers</code>                                                                  | <code>bool</code>                                                                     | <code>true</code> / <code>false</code>      | Merge operation, bubble, insertion, selection sort, and last-pivot <code>partition-step</code> only. Draw labelled index arrows above the marked cells, layered on top of their fills. Merge operation uses <code>i</code> , <code>j</code> , and <code>i+j</code> last-pivot partition uses <code>i</code> , <code>j</code> , and <code>pivot</code> . |
| <code>labels</code>                                                                    | <code>bool</code>                                                                     | <code>true</code>                           | All sorting builders. Set to <code>false</code> to hide generated step captions and structural text while preserving diagrams, highlights, indices, and pointer arrows.                                                                                                                                                                                 |
| <code>compare</code> , <code>swap</code> , <code>current</code> , <code>minimum</code> | <code>dictionary</code>                                                               | <code>none</code>                           | Bubble, insertion, and selection sort only. A <code>node-mark-style(...)</code> override for that role's marker. <code>current</code> and <code>minimum</code> apply to selection sort only.                                                                                                                                                            |

| Field                 | Call               | Effect                                                                                                           |
|-----------------------|--------------------|------------------------------------------------------------------------------------------------------------------|
| <code>.diagram</code> | n/a                | Rendered full sorting trace.                                                                                     |
| <code>.steps</code>   | <code>array</code> | Generated step dictionaries. Each step has <code>label</code> , <code>diagram</code> , and <code>values</code> . |
| <code>.result</code>  | <code>array</code> | Sorted values.                                                                                                   |

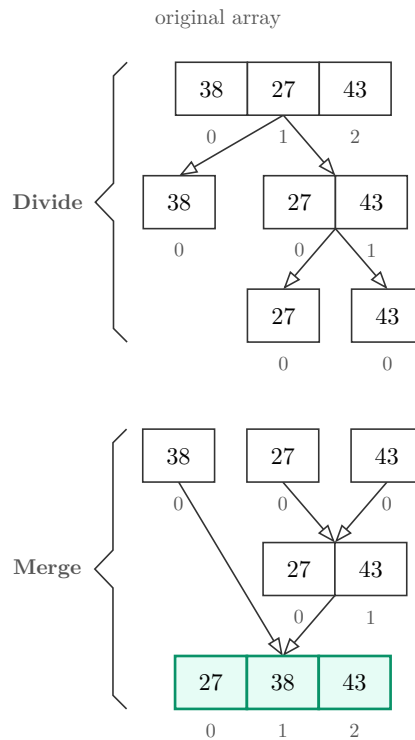
## Full traces

Long traces below flow one labelled step at a time, so page breaks cannot separate a label from its array or make neighbouring frames overlap.

### Merge sort

```
1 #let trace = merge-sort((38, 27, 43))
2 #trace.diagram
```

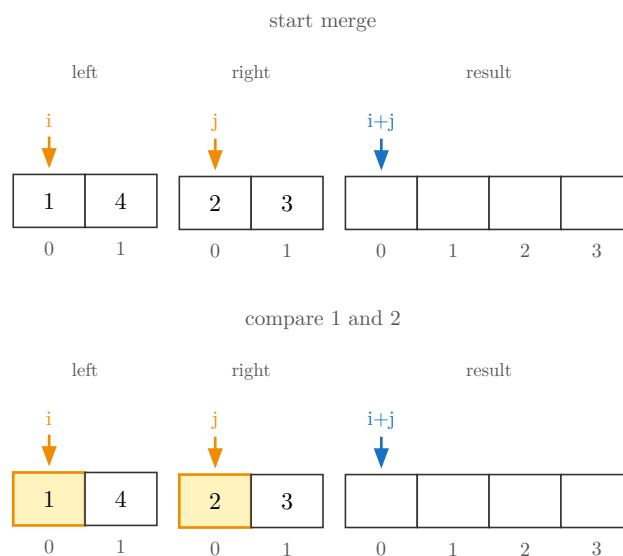
**t** Typst



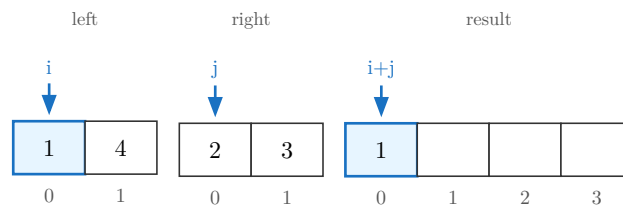
### Merge operation

```
1 #let trace = merge-operation((1, 4), (2, 3))
2 #for step in trace.steps [#step.diagram]
```

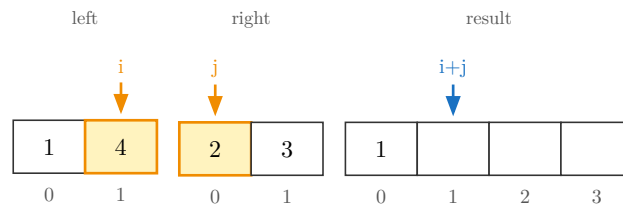
**t** Typst



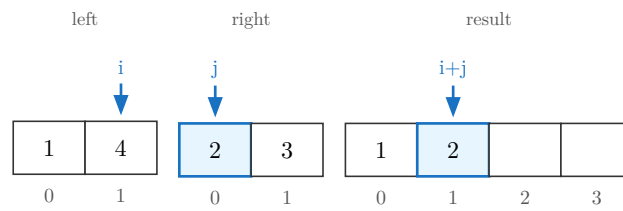
take 1



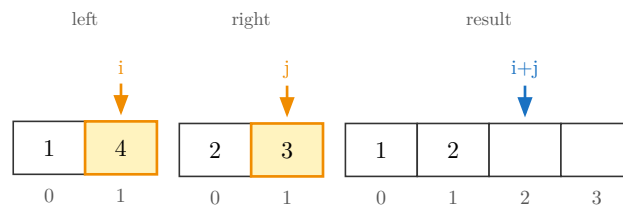
compare 4 and 2



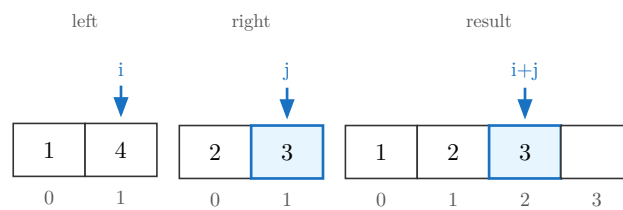
take 2



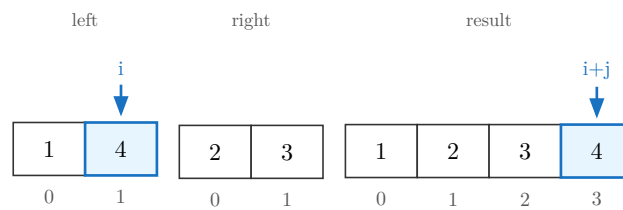
compare 4 and 3

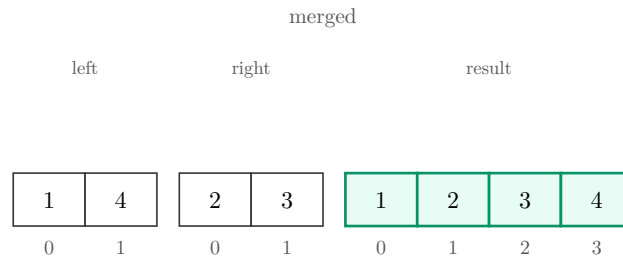


take 3



take remaining 4

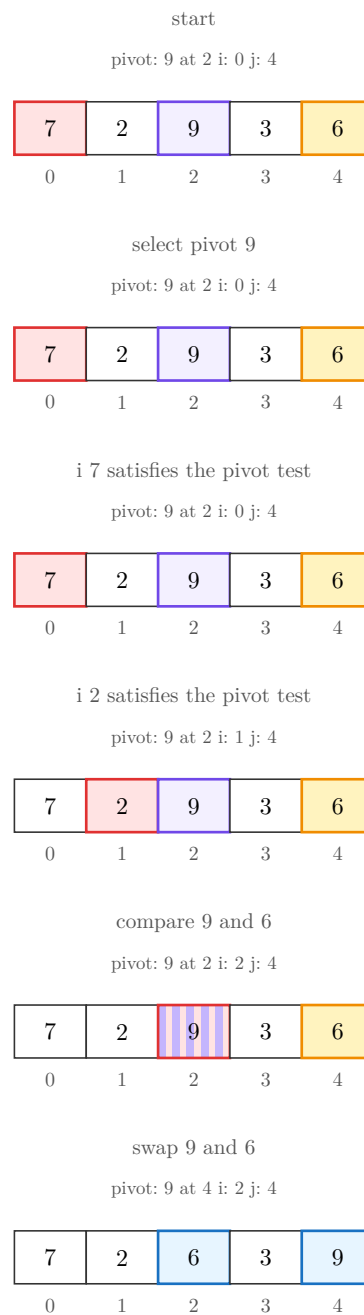




## Partition around a middle pivot

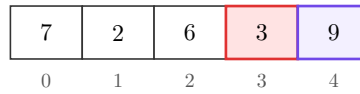
```
1 #let trace = partition-step((7, 2, 9, 3, 6))
2 #for step in trace.steps [#step.diagram]
```

 Typst



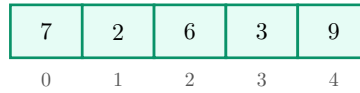
i 3 satisfies the pivot test

pivot: 9 at 4 i: 3 j: 3



partitioned

pivot: 9 at 4 i: 4 j: 3

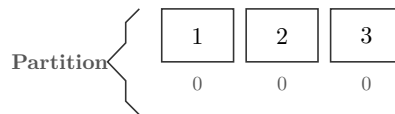
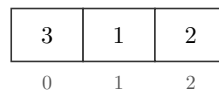


## Quick sort

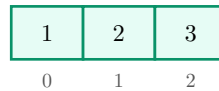
```
1 #let trace = quick-sort((3, 1, 2))
2 #trace.diagram
```

Typst

original array



sorted array

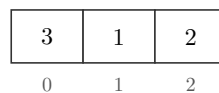


## Bubble sort

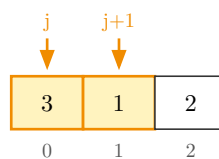
```
1 #let trace = bubble-sort((3, 1, 2))
2 #for step in trace.steps [#step.diagram]
```

Typst

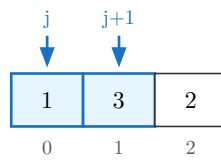
start



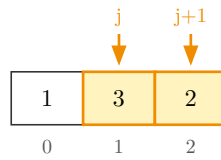
compare 3 and 1



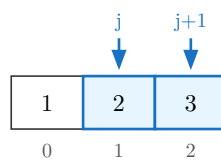
swap 3 and 1



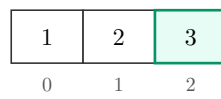
compare 3 and 2



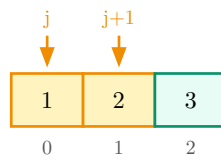
swap 3 and 2



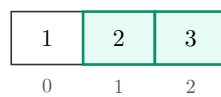
3 settled



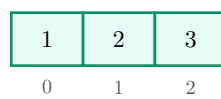
compare 1 and 2



2 settled



sorted



## Insertion sort

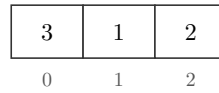
```

1 #let trace = insertion-sort((3, 1, 2))
2 #for step in trace.steps [#step.diagram]

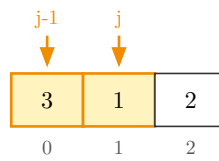
```

**t** Typst

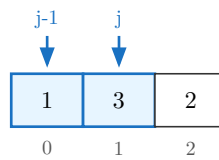
start



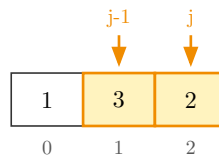
compare 3 and 1



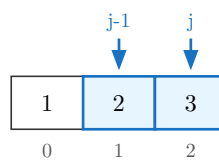
swap 3 and 1



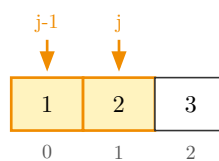
compare 3 and 2



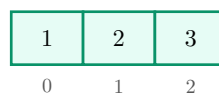
swap 3 and 2



compare 1 and 2



sorted

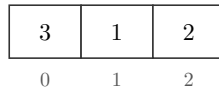


## Selection sort

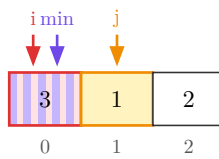
```
1 #let trace = selection-sort((3, 1, 2))
2 #for step in trace.steps [#step.diagram]
```

**t** Typst

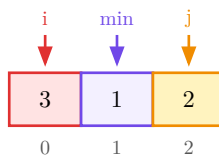
start



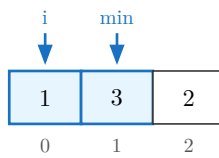
position 0, minimum 0, item 1



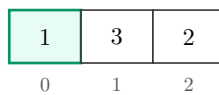
position 0, minimum 1, item 2



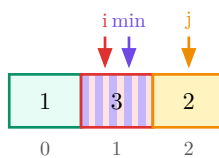
swap 3 and 1

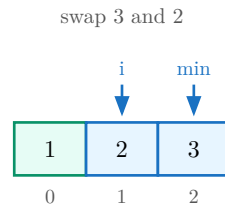


1 settled

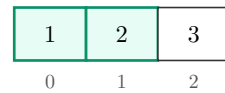


position 1, minimum 1, item 2

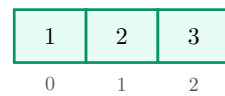




2 settled



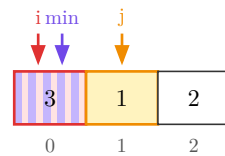
sorted



### Hide generated captions

```
1 #let trace = selection-sort((3, 1, 2), labels: false)
2 #trace.steps.at(1).diagram
```

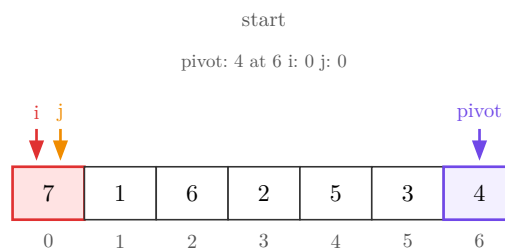
Typst



With `pivot: "last"` , placing the median at the end produces an even quicksort partition: three values on each side of the pivot. This trace renders every comparison, swap, and cursor position.

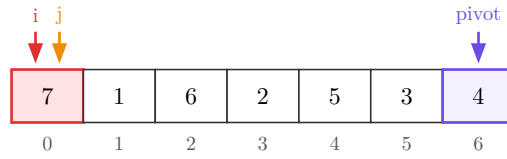
```
1 #let trace = partition-step(
2   (7, 1, 6, 2, 5, 3, 4),
3   pivot: "last",
4   pointers: true,
5 )
6 #for step in trace.steps [#step.diagram]
```

Typst



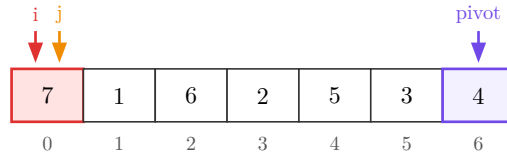
select last pivot 4

pivot: 4 at 6 i: 0 j: 0



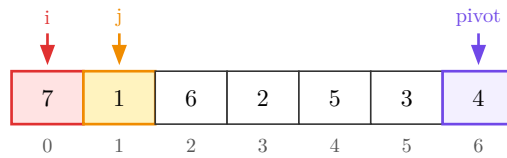
compare 7 and pivot 4

pivot: 4 at 6 i: 0 j: 0



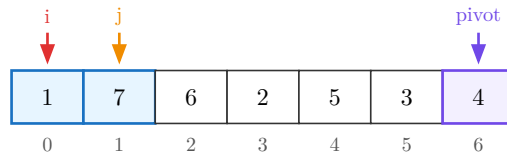
compare 1 and pivot 4

pivot: 4 at 6 i: 0 j: 1



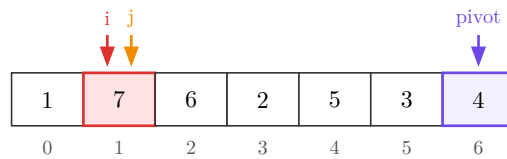
swap 7 and 1

pivot: 4 at 6 i: 0 j: 1



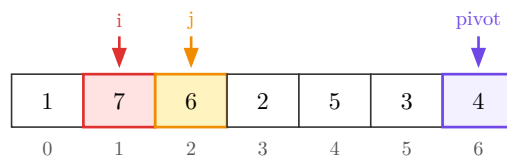
advance i to 1

pivot: 4 at 6 i: 1 j: 1



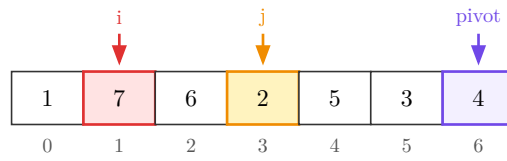
compare 6 and pivot 4

pivot: 4 at 6 i: 1 j: 2



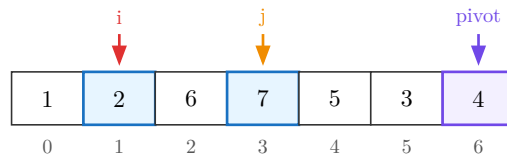
compare 2 and pivot 4

pivot: 4 at 6 i: 1 j: 3



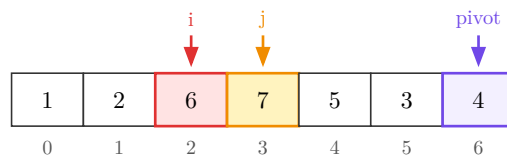
swap 7 and 2

pivot: 4 at 6 i: 1 j: 3



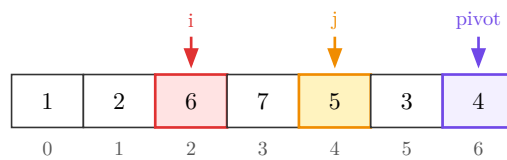
advance i to 2

pivot: 4 at 6 i: 2 j: 3



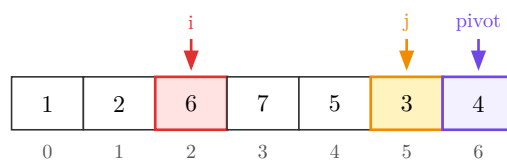
compare 5 and pivot 4

pivot: 4 at 6 i: 2 j: 4



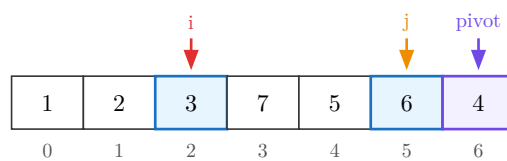
compare 3 and pivot 4

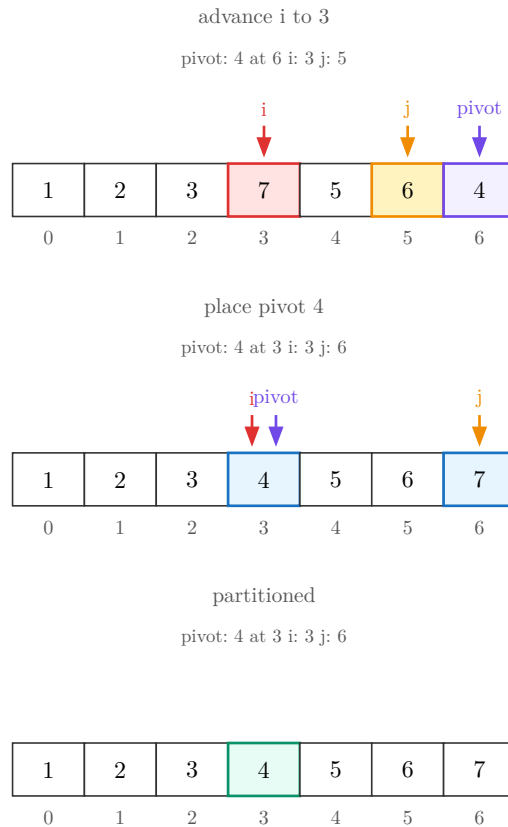
pivot: 4 at 6 i: 2 j: 5



swap 6 and 3

pivot: 4 at 6 i: 2 j: 5





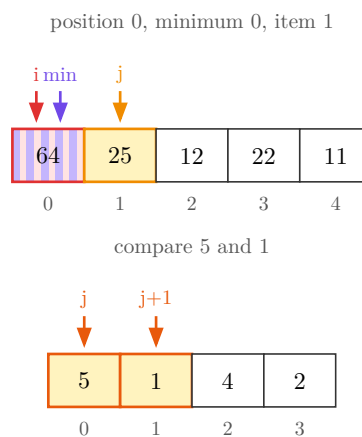
Pointer markers keep the cell fill clean and let each role carry its own colour. These focused frames show selection's `i`, `min`, and `j` markers together, plus customized bubble-sort marker strokes.

```

1  #let selection = selection-sort((64, 25, 12, 22, 11), pointers: true)
2  #selection.steps.at(1).diagram
3
4  #let bubble = bubble-sort(
5    (5, 1, 4, 2),
6    pointers: true,
7    compare: node-mark-style(stroke: 1pt + rgb("#E8590C")),
8    swap: node-mark-style(stroke: 1pt + rgb("#2F9E44")),
9  )
10 #bubble.steps.at(1).diagram

```

 Typst



### 3.11 graph()

Full signature:

```

1  #graph(
2      adjacency,
3      directed: true,
4      labels: (:),
5      positions: (:),
6      layout: "auto",
7      radius: auto,
8      gap: auto,
9      edge-customizations: (),
10     node-customizations: (),
11     node-labels: (:),
12     style: (:),
13 )

```

 Typst

`adjacency` maps each node label to an array of that node's neighbor labels. To label an edge, use `(neighbor, label)` instead of just `neighbor`. A node with no outgoing edges still needs its key present, with an empty array; a label that only ever shows up as someone else's neighbor gets added on its own.

| Argument                         | Type         | Default    | Description                                                                                                                                                             |
|----------------------------------|--------------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>adjacency</code>           | dictionary   | (required) | Node label ( <code>str</code> ) to array of neighbor labels or <code>(neighbor, edge-label)</code> pairs.                                                               |
| <code>directed</code>            | bool         | true       | Draw an arrowhead on every declared pair. <code>false</code> drops arrowheads and collapses a reciprocal pair into the one edge it represents.                          |
| <code>labels</code>              | dictionary   | (:)        | Node label to drawn content: math, styled text, an image, anything. A label left out keeps the plain key as its own content.                                            |
| <code>positions</code>           | dictionary   | (:)        | Node label to absolute <code>(x, y)</code> or <code>(rel:, offset:)</code> placement.                                                                                   |
| <code>layout</code>              | str          | "auto"     | "auto" starts from the circular layout and applies <code>positions</code> . "linear" starts from a row layout. "manual" requires every node in <code>positions</code> . |
| <code>radius</code>              | auto / float | auto       | Circle radius for <code>layout: "auto"</code> . Passing a radius with <code>layout: "manual"</code> is an error.                                                        |
| <code>gap</code>                 | auto / float | auto       | Spacing between nodes for <code>layout: "linear"</code> , in canvas units. <code>auto</code> is 1.5. Has no effect on "auto" or "manual" layout.                        |
| <code>edge-customizations</code> | array        | ()         | <code>(from, to, options)</code> tuples restyling one edge. See below.                                                                                                  |
| <code>node-customizations</code> | array        | ()         | <code>(node, options)</code> tuples restyling one node by graph identity.                                                                                               |

|                          |                    |     |                                                                                   |
|--------------------------|--------------------|-----|-----------------------------------------------------------------------------------|
| <code>node-labels</code> | array / dictionary | (:) | Labels drawn outside individual graph nodes.                                      |
| <code>style</code>       | dictionary         | (:) | Per-call style override merged over the defaults. See <a href="#">Section 8</a> . |

### Nested keys for `adjacency`

| Path                                             | Type            | Default  | Description                                                                                                   |
|--------------------------------------------------|-----------------|----------|---------------------------------------------------------------------------------------------------------------|
| <code>adjacency.&lt;node&gt;</code>              | array           | required | Outgoing neighbors for one node. Use an empty array to draw an isolated node.                                 |
| <code>adjacency.&lt;node&gt;[]</code>            | content tuple / | required | Either a neighbor label, or ( <code>neighbor</code> , <code>edge-label</code> ) to draw a label on that edge. |
| <code>adjacency.&lt;node&gt;[].neighbor</code>   | content         | required | Target node identity.                                                                                         |
| <code>adjacency.&lt;node&gt;[].edge-label</code> | content         | none     | Optional edge label, such as a weight.                                                                        |

### Nested keys for `labels`

| Path                             | Type    | Default  | Description                                                                               |
|----------------------------------|---------|----------|-------------------------------------------------------------------------------------------|
| <code>labels.&lt;node&gt;</code> | content | node key | Drawn content for one node. Identity still comes from the plain adjacency/position label. |

### Nested keys for `positions`

| Path                                       | Type               | Default                         | Description                                                                             |
|--------------------------------------------|--------------------|---------------------------------|-----------------------------------------------------------------------------------------|
| <code>positions.&lt;node&gt;</code>        | point / dictionary | auto layout position            | Absolute ( <code>x</code> , <code>y</code> ) position or relative placement dictionary. |
| <code>positions.&lt;node&gt;.rel</code>    | content            | required for relative placement | Another node label to place from.                                                       |
| <code>positions.&lt;node&gt;.offset</code> | point              | required for relative placement | ( <code>dx</code> , <code>dy</code> ) offset from <code>rel</code> .                    |

### Nested keys for `edge-customizations`

| Path                                              | Type       | Default  | Description                                                                                                                                                                    |
|---------------------------------------------------|------------|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>edge-customizations[]</code>                | tuple      | n/a      | One ( <code>from</code> , <code>to</code> , <code>options</code> ) tuple. For trees, <code>from</code> / <code>to</code> are parent and child values; for graphs, node labels. |
| <code>edge-customizations[].from</code>           | content    | required | Source node identity.                                                                                                                                                          |
| <code>edge-customizations[].to</code>             | content    | required | Target node identity.                                                                                                                                                          |
| <code>edge-customizations[].options</code>        | dictionary | required | Per-edge options.                                                                                                                                                              |
| <code>edge-customizations[].options.stroke</code> | stroke     | none     | Full stroke override. Wins over <code>color</code> if both are set.                                                                                                            |

|                                                    |                      |          |                                                                                                                       |
|----------------------------------------------------|----------------------|----------|-----------------------------------------------------------------------------------------------------------------------|
| <code>edge-customizations[].options.color</code>   | color                | none     | Paint only, at the default thickness.                                                                                 |
| <code>edge-customizations[].options.pattern</code> | str                  | "normal" | "normal" , "dashed" , "dotted" , or "wavy" .                                                                          |
| <code>edge-customizations[].options.arrow</code>   | none / bool / str    | none     | Per-edge arrowheads: <code>none</code> , "end" / <code>true</code> , "start" , or "both" .                            |
| <code>edge-customizations[].options.bend</code>    | str / bool           | false    | "left" or "right" curves the edge relative to its travel direction. <code>false</code> keeps it straight.             |
| <code>edge-customizations[].options.angle</code>   | angle                | 25deg    | How sharply a bent edge curves. Ignored when <code>bend</code> is <code>false</code> .                                |
| <code>edge-customizations[].options.label</code>   | content / dictionary | none     | Tree edge label content, or graph edge-label text overrides. A dictionary can include <code>content</code> for trees. |

#### Nested keys for `edge-customizations[].options.label`

| Path                                                      | Type        | Default                                | Description                                                                                 |
|-----------------------------------------------------------|-------------|----------------------------------------|---------------------------------------------------------------------------------------------|
| <code>edge-customizations[].options.label.size</code>     | length      | <code>style.label-text.size</code>     | Edge-label size.                                                                            |
| <code>edge-customizations[].options.label.content</code>  | content     | tree only                              | Tree edge label content. Graph edge labels come from adjacency entries such as ("B", [4]) . |
| <code>edge-customizations[].options.label.color</code>    | color       | <code>style.label-text.color</code>    | Edge-label text color.                                                                      |
| <code>edge-customizations[].options.label.rotation</code> | angle / str | <code>style.label-text.rotation</code> | An angle, or "edge" to follow the tree/graph edge angle.                                    |
| <code>edge-customizations[].options.label.font</code>     | str / array | <code>style.label-text.font</code>     | Typst text font selection.                                                                  |
| <code>edge-customizations[].options.label.weight</code>   | str / int   | <code>style.label-text.weight</code>   | Typst text weight.                                                                          |

#### Nested keys for `style`

| Path                           | Type              | Default                | Description                                                                                                                                                                             |
|--------------------------------|-------------------|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>style.node-radius</code> | float             | 0.34                   | Circle radius or base size for other node shapes.                                                                                                                                       |
| <code>style.node-shape</code>  | str               | "circle"               | "circle" , "square" , "rounded" , "capsule" , "diamond" , or "hexagon" .                                                                                                                |
| <code>style.node-stroke</code> | stroke            | 0.6pt + rgb("#333333") | Node outline.                                                                                                                                                                           |
| <code>style.node-fill</code>   | color             | white                  | Default node fill.                                                                                                                                                                      |
| <code>style.edge-stroke</code> | stroke            | 0.6pt + rgb("#333333") | Default edge stroke.                                                                                                                                                                    |
| <code>style.edge-arrow</code>  | none / bool / str | none                   | Global arrowhead override: <code>none</code> / <code>false</code> , "end" or <code>true</code> , "start" , or "both" . <code>directed: true</code> uses "end" unless this overrides it. |

|                                         |                           |                                                   |                                                                                                  |
|-----------------------------------------|---------------------------|---------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <code>style.edge-arrow-fill</code>      | <code>none / color</code> | <code>none</code>                                 | <code>none</code> gives open arrowheads; a color fills arrowheads solid.                         |
| <code>style.edge-pattern</code>         | <code>str</code>          | <code>"normal"</code>                             | <code>"normal"</code> , <code>"dashed"</code> , <code>"dotted"</code> , or <code>"wavy"</code> . |
| <code>style.edge-wave-amplitude</code>  | <code>float</code>        | <code>0.07</code>                                 | Wave height in canvas units.                                                                     |
| <code>style.edge-wave-step</code>       | <code>float</code>        | <code>0.14</code>                                 | Approximate wavelength in canvas units.                                                          |
| <code>style.node-text</code>            | <code>dictionary</code>   | <code>(size: 9pt)</code>                          | Node text dictionary. See nested keys below.                                                     |
| <code>style.value-text</code>           | <code>dictionary</code>   | inherits <code>node-text</code>                   | Node-value typography.                                                                           |
| <code>style.label-text</code>           | <code>dictionary</code>   | <code>(fill: "#555555")</code> <code>rgb()</code> | Edge label text dictionary. See nested keys below.                                               |
| <code>style.edge-label-text</code>      | <code>dictionary</code>   | inherits <code>label-text</code>                  | Edge-label typography.                                                                           |
| <code>style.algorithm-label-text</code> | <code>dictionary</code>   | <code>(size: 8pt)</code>                          | BFS/DFS/Dijkstra trace captions.                                                                 |
| <code>style.visited-style</code>        | <code>dictionary</code>   | <code>green</code>                                | Visited-node fill/stroke/text/shape overrides.                                                   |
| <code>style.current-style</code>        | <code>dictionary</code>   | <code>blue</code>                                 | Current-node overrides.                                                                          |
| <code>style.queued-style</code>         | <code>dictionary</code>   | <code>yellow/orange</code>                        | Queued or stacked node overrides.                                                                |
| <code>style.active-edge-style</code>    | <code>dictionary</code>   | <code>blue 2pt</code> stroke                      | Currently inspected edge overrides.                                                              |
| <code>style.scale</code>                | <code>float</code>        | <code>1.0</code>                                  | Uniform scale on the rendered graph.                                                             |

#### Nested keys for `style.node-text`

| Path                                  | Type                     | Default           | Description                                    |
|---------------------------------------|--------------------------|-------------------|------------------------------------------------|
| <code>style.node-text.size</code>     | <code>length</code>      | inherits          | Text size.                                     |
| <code>style.node-text.color</code>    | <code>color</code>       | inherits          | Text color.                                    |
| <code>style.node-text.rotation</code> | <code>angle</code>       | <code>0deg</code> | Text rotation applied by the diagram renderer. |
| <code>style.node-text.font</code>     | <code>str / array</code> | inherits          | Typst text font selection.                     |
| <code>style.node-text.weight</code>   | <code>str / int</code>   | inherits          | Typst text weight.                             |

#### Nested keys for `style.label-text`

| Path                                   | Type                     | Default                                  | Description                                                                                          |
|----------------------------------------|--------------------------|------------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>style.label-text.size</code>     | <code>length</code>      | 85% of <code>style.node-text.size</code> | Label text size.                                                                                     |
| <code>style.label-text.color</code>    | <code>color</code>       | <code>rgb("#555555")</code>              | Label text color.                                                                                    |
| <code>style.label-text.rotation</code> | <code>angle / str</code> | <code>0deg</code>                        | Label rotation. Tree and graph edge labels also accept <code>"edge"</code> to follow the edge angle. |

|                                      |                          |          |                            |
|--------------------------------------|--------------------------|----------|----------------------------|
| <code>style.label-text.font</code>   | <code>str / array</code> | inherits | Typst text font selection. |
| <code>style.label-text.weight</code> | <code>str / int</code>   | inherits | Typst text weight.         |

### Nested keys for `node-customizations`

| Path                                                     | Type       | Default                        | Description                                                                                                                                                               |
|----------------------------------------------------------|------------|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>node-customizations[]</code>                       | tuple      | n/a                            | One <code>(node, options)</code> tuple.                                                                                                                                   |
| <code>node-customizations[][.node]</code>                | content    | required                       | Graph identity key, not the display label from <code>labels</code> .                                                                                                      |
| <code>node-customizations[][.options]</code>             | dictionary | required                       | Per-node drawing options.                                                                                                                                                 |
| <code>node-customizations[][.options.fill]</code>        | color      | normal fill                    | Node fill.                                                                                                                                                                |
| <code>node-customizations[][.options.stroke]</code>      | stroke     | normal stroke                  | Node outline.                                                                                                                                                             |
| <code>node-customizations[][.options.shape]</code>       | str        | <code>style.node-shape</code>  | Trees/graphs only: <code>"circle"</code> , <code>"square"</code> , <code>"rounded"</code> , <code>"capsule"</code> , <code>"diamond"</code> , or <code>"hexagon"</code> . |
| <code>node-customizations[][.options.node-radius]</code> | float      | <code>style.node-radius</code> | Trees/graphs only. Shape size.                                                                                                                                            |
| <code>node-customizations[][.options.text]</code>        | dictionary | <code>style.node-text</code>   | Text style merged into the node text.                                                                                                                                     |

### Nested keys for `node-labels`

| Path                                        | Type                 | Default                                | Description                                                                                             |
|---------------------------------------------|----------------------|----------------------------------------|---------------------------------------------------------------------------------------------------------|
| <code>node-labels[]</code>                  | tuple                | n/a                                    | One <code>(node, label)</code> tuple.                                                                   |
| <code>node-labels[][.node]</code>           | content              | required                               | Graph identity key, not the display label from <code>labels</code> .                                    |
| <code>node-labels[][.label]</code>          | content / dictionary | required                               | Bare content for quick labels, or a dictionary with <code>content</code> plus style and placement keys. |
| <code>node-labels[][.label.content]</code>  | content              | required for dict labels               | Label body. <code>body</code> is also accepted.                                                         |
| <code>node-labels[][.label.position]</code> | str / angle          | <code>"right"</code>                   | <code>"right"</code> , <code>"left"</code> , <code>"top"</code> , <code>"bottom"</code> , or an angle.  |
| <code>node-labels[][.label.offset]</code>   | point                | <code>(0, 0)</code>                    | Extra <code>(dx, dy)</code> shift after the position is applied.                                        |
| <code>node-labels[][.label.size]</code>     | length               | <code>style.label-text.size</code>     | Node-label size.                                                                                        |
| <code>node-labels[][.label.color]</code>    | color                | <code>style.label-text.color</code>    | Node-label text color.                                                                                  |
| <code>node-labels[][.label.rotation]</code> | angle                | <code>style.label-text.rotation</code> | Rotation of the label text itself.                                                                      |
| <code>node-labels[][.label.font]</code>     | str / array          | <code>style.label-text.font</code>     | Typst text font selection.                                                                              |
| <code>node-labels[][.label.weight]</code>   | str / int            | <code>style.label-text.weight</code>   | Typst text weight.                                                                                      |

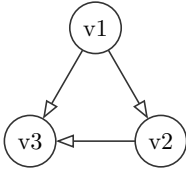
### Nested keys for `style.node-labels`

| Path                                    | Type                     | Default                                | Description                                                    |
|-----------------------------------------|--------------------------|----------------------------------------|----------------------------------------------------------------|
| <code>style.node-labels.position</code> | <code>str / angle</code> | <code>"right"</code>                   | Default position for all node labels in this diagram.          |
| <code>style.node-labels.offset</code>   | <code>point</code>       | <code>(0, 0)</code>                    | Default extra <code>(dx, dy)</code> shift for all node labels. |
| <code>style.node-labels.gap</code>      | <code>float</code>       | <code>0.22</code>                      | Default distance from the node boundary in canvas units.       |
| <code>style.node-labels.size</code>     | <code>length</code>      | <code>style.label-text.size</code>     | Default node-label text size.                                  |
| <code>style.node-labels.color</code>    | <code>color</code>       | <code>style.label-text.color</code>    | Default node-label text color.                                 |
| <code>style.node-labels.font</code>     | <code>str / array</code> | <code>inherits</code>                  | Typst text font selection.                                     |
| <code>style.node-labels.weight</code>   | <code>str / int</code>   | <code>inherits</code>                  | Typst text weight.                                             |
| <code>style.node-labels.rotation</code> | <code>angle</code>       | <code>style.label-text.rotation</code> | Default rotation of the label text itself.                     |

| Field                 | Call             | Effect              |
|-----------------------|------------------|---------------------|
| <code>.diagram</code> | <code>n/a</code> | The rendered graph. |

```
1 #graph(("v1": ("v2", "v3"), "v2": ("v3",), "v3": ())).diagram
```

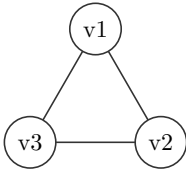
Typst



Declaring both directions of an edge still draws it once.

```
1 #graph(  
2   ("v1": ("v2", "v3"), "v2": ("v1", "v3"),  
3   "v3": ()),  
4   directed: false,  
5 ).diagram
```

Typst



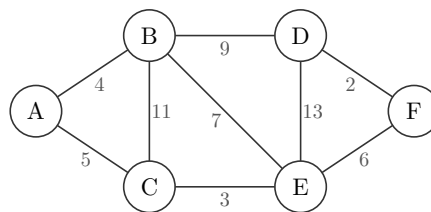
Use (`neighbor`, `label`) entries to draw edge labels, such as weights.

```

1  #graph(
2    (
3      "A": (("B", [4]), ("C", [5])),
4      "B": (("C", [11]), ("D", [9]), ("E", [7])),
5      "C": (("E", [3]),),
6      "D": (("E", [13]), ("F", [2])),
7      "E": (("F", [6]),),
8      "F": (),
9    ),
10   directed: false,
11   layout: "manual",
12   positions: (
13     "A": (0, 0),
14     "B": (rel: "A", offset: (1.5, 1.0)),
15     "C": (rel: "A", offset: (1.5, -1.0)),
16     "D": (rel: "B", offset: (2.0, 0.0)),
17     "E": (rel: "C", offset: (2.0, 0.0)),
18     "F": (rel: "D", offset: (1.5, -1.0)),
19   ),
20 ).diagram

```

**t** Typst

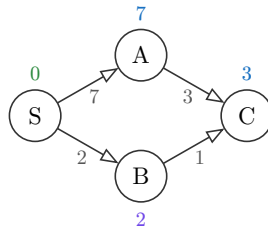


Use `node-labels` for annotations such as tentative distances, predecessor tags, ranks, or algorithm state. The node's identity and its drawn inside label stay unchanged.

```

1  #graph(
2    ("S": (("A", [7]), ("B", [2])), "A": (("C", [3]),), "B": (("C", [1]),), "C": ()),
3    layout: "manual",
4    positions: ("S": (0, 0), "A": (1.4, 0.8), "B": (1.4, -0.8), "C": (2.8, 0)),
5    node-labels: (
6      ("S", (content: [$0$], position: "top", color: rgb("#2B8A3E"))),
7      ("A", [$7$]),
8      ("B", (content: [$2$], position: "bottom", color: rgb("#7048E8"))),
9      ("C", [$3$]),
10   ),
11   style: (
12     edge-arrow: "end",
13     node-labels: (position: "top", color: rgb("#1971C2")),
14   ),
15 ).diagram

```

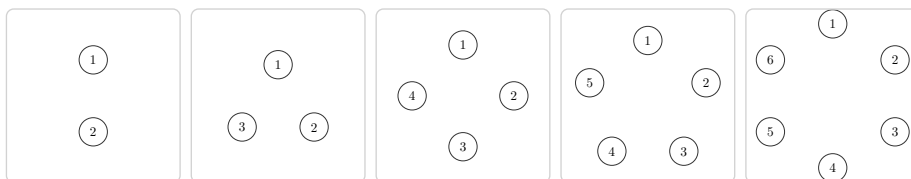


`layout: "auto"` is the default. Nodes are placed around a circle; the radius grows as the node count increases. Pass `radius:` to set the circle radius yourself.

```

1  #let panel(body) = block(width: 6.2em, height: 6.2em, inset: 0.2em,
2    stroke: 0.5pt + luma(210), radius: 3pt, clip: true,
3    align(center + horizon, scale(55%, reflow: true, body)))
4  #std.stack(dir: ltr, spacing: 0.4em,
5    panel(graph(("1": (), "2": ())).diagram),
6    panel(graph(("1": (), "2": (), "3": ())).diagram),
7    panel(graph(("1": (), "2": (), "3": (), "4": ())).diagram),
8    panel(graph(("1": (), "2": (), "3": (), "4": (), "5": ())).diagram),
9    panel(graph(("1": (), "2": (), "3": (), "4": (), "5": (), "6": ())).diagram),
10 )

```



`radius` only applies to `layout: "auto"` . In manual layout, every node already has an explicit position, so combining `radius` with `layout: "manual"` raises an error.

`layout: "linear"` places nodes left to right in a single row, useful for topological orderings. `gap`: sets the spacing between nodes; it only applies to this layout.

```
1 #graph(  
2   ("A": ("B",), "B": ("C",), "C": ("D",), "D": ()),  
3   layout: "linear",  
4   gap: 2.5,  
5 ).diagram
```

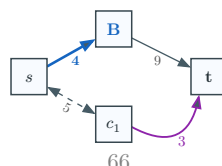
[t Typst](#)

Argument coverage: this graph combines adjacency edge labels, node display labels, manual and relative positions, graph-wide style, per-edge options, and nested edge-label overrides.

```

1  #graph(
2    (
3      "A": (("B", [4]), ("C", [5])),
4      "B": (("D", [9])),
5      "C": (("D", [3])),
6      "D": (),
7    ),
8    directed: true,
9    labels: (
10     "A": $s$,
11     "B": text(fill: rgb("#1565C0"))[*B*],
12     "C": $c_1$,
13     "D": text(weight: "bold")[t],
14   ),
15   layout: "manual",
16   positions: (
17     "A": (0, 0),
18     "B": (rel: "A", offset: (1.6, 0.9)),
19     "C": (rel: "A", offset: (1.6, -0.9)),
20     "D": (rel: "B", offset: (1.8, -0.9)),
21   ),
22   edge-customizations: (
23     ("A", "B", (stroke: 1.4pt + rgb("#1565C0"), label: (color: rgb("#1565C0"), weight:
24       "bold"))),
25     ("A", "C", (pattern: "dashed", arrow: "both", label: (rotation: "edge"))),
26     ("C", "D", (bend: "right", angle: 35deg, color: rgb("#8E24AA"), label: (size: 8pt,
27       color: rgb("#8E24AA")))),
28   ),
29   style: (
30     node-shape: "square",
31     node-radius: 0.34,
32     node-fill: rgb("#F8FBFF"),
33     node-stroke: 0.8pt + rgb("#37474F"),
34     node-text: (size: 9pt, color: rgb("#263238")),
35     edge-stroke: 0.7pt + rgb("#455A64"),
36     edge-arrow: "end",
37     edge-arrow-fill: rgb("#455A64"),
38     edge-pattern: "normal",
39     label-text: (size: 7.5pt, color: rgb("#555555")),
40     scale: 0.7,
41   ),
42 ).diagram

```

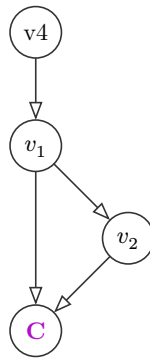


### 3.12 labels and positions

`labels` swaps what's drawn inside a node for any content, keyed by the plain-string label that `adjacency` / `positions` / `edge-customizations` still use for identity. With the default `layout: "auto"`, `positions` places a node at an absolute `(x, y)` point or at `(rel: other-label, offset: (dx, dy))` on top of the automatic circular layout.

```
1 #graph(
2   ("v1": ("v2", "v3"), "v2": ("v3",), "v3": (), "v4": ("v1",)),
3   labels: ("v1": $v_1$, "v2": $v_2$, "v3": text(fill: purple)[*C*]),
4   positions: ("v4": (rel: "v1", offset: (0, 1.5))),
5 ).diagram
```

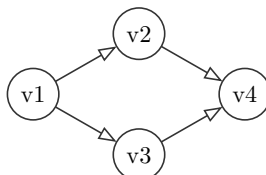
 Typst



Set `layout: "manual"` to skip the circular layout. Every node, including a neighbor-only node, must have a `positions` entry. At least one absolute `(x, y)` entry should act as an anchor for relative placements.

```
1 #graph(
2   ("v1": ("v2", "v3"), "v2": ("v4",), "v3": ("v4",), "v4": ()),
3   layout: "manual",
4   positions: (
5     "v1": (0, 0),
6     "v2": (rel: "v1", offset: (1.4, 0.8)),
7     "v3": (rel: "v1", offset: (1.4, -0.8)),
8     "v4": (rel: "v2", offset: (1.4, -0.8)),
9   ),
10 ).diagram
```

 Typst



### 3.13 edge-customizations

An array of `(from, to, options)` tuples, each restyling one edge. For `graph`, `from` and `to` are node labels. For generated trees, they are parent and child values. `options` is a dictionary.

#### Nested keys for `edge-customizations`

| Path                                               | Type                 | Default  | Description                                                                                                                                      |
|----------------------------------------------------|----------------------|----------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>edge-customizations[]</code>                 | tuple                | n/a      | One <code>(from, to, options)</code> tuple. For trees, <code>from</code> / <code>to</code> are parent and child values; for graphs, node labels. |
| <code>edge-customizations[].from</code>            | content              | required | Source node identity.                                                                                                                            |
| <code>edge-customizations[].to</code>              | content              | required | Target node identity.                                                                                                                            |
| <code>edge-customizations[].options</code>         | dictionary           | required | Per-edge options.                                                                                                                                |
| <code>edge-customizations[].options.stroke</code>  | stroke               | none     | Full stroke override. Wins over <code>color</code> if both are set.                                                                              |
| <code>edge-customizations[].options.color</code>   | color                | none     | Paint only, at the default thickness.                                                                                                            |
| <code>edge-customizations[].options.pattern</code> | str                  | "normal" | "normal" , "dashed" , "dotted" , or "wavy" .                                                                                                     |
| <code>edge-customizations[].options.arrow</code>   | none / bool / str    | none     | Per-edge arrowheads: <code>none</code> , "end" / <code>true</code> , "start" , or "both" .                                                       |
| <code>edge-customizations[].options.bend</code>    | str / bool           | false    | "left" or "right" curves the edge relative to its travel direction. <code>false</code> keeps it straight.                                        |
| <code>edge-customizations[].options.angle</code>   | angle                | 25deg    | How sharply a bent edge curves. Ignored when <code>bend</code> is <code>false</code> .                                                           |
| <code>edge-customizations[].options.label</code>   | content / dictionary | none     | Tree edge label content, or graph edge-label text overrides. A dictionary can include <code>content</code> for trees.                            |

#### Nested keys for `edge-customizations[].options.label`

| Path                                                      | Type        | Default                                | Description                                                                                              |
|-----------------------------------------------------------|-------------|----------------------------------------|----------------------------------------------------------------------------------------------------------|
| <code>edge-customizations[].options.label.size</code>     | length      | <code>style.label-text.size</code>     | Edge-label size.                                                                                         |
| <code>edge-customizations[].options.label.content</code>  | content     | tree only                              | Tree edge label content. Graph edge labels come from adjacency entries such as <code>("B", [4])</code> . |
| <code>edge-customizations[].options.label.color</code>    | color       | <code>style.label-text.color</code>    | Edge-label text color.                                                                                   |
| <code>edge-customizations[].options.label.rotation</code> | angle / str | <code>style.label-text.rotation</code> | An angle, or "edge" to follow the tree/graph edge angle.                                                 |
| <code>edge-customizations[].options.label.font</code>     | str / array | <code>style.label-text.font</code>     | Typst text font selection.                                                                               |
| <code>edge-customizations[].options.label.weight</code>   | str / int   | <code>style.label-text.weight</code>   | Typst text weight.                                                                                       |

```

1 #graph(
2   ("v1": ("v2", "v3"), "v2": ("v3",), "v3": ()),
3   edge-customizations: (
4     ("v1", "v2", (stroke: 2pt + red)),    69
5     ("v2", "v3", (color: rgb("#2E7D32"))),
6     ("v1", "v3", (pattern: "wavy", arrow: "both")),
7   ),

```

 Typst

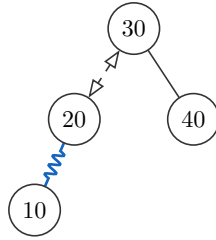
Tree edge customizations use the values in the parent and child nodes:

```

1 #bst(
2   30, 20, 40, 10,
3   edge-customizations: (
4     (30, 20, (pattern: "dashed", arrow: "both")),
5     (20, 10, (pattern: "wavy", color: rgb("#1565C0"))),
6   ),
7 ).diagram

```

 Typst



**Note.** A mutual pair bent the same way on both directions ( ("v1", "v2", (bend: "left")) and ("v2", "v1", (bend: "left")) ) curves to opposite sides, since "left" is relative to each edge's own direction of travel. The two arrows read as two edges instead of one double-headed line.

**Note.** Nodes default to a circle, in the order their labels first show up, radius growing with the node count. Connected nodes don't cluster closer together yet, and `graph` has no operations: no `add-node`, `add-edge`, or a transition to match trees and heaps. See [Section 10](#).

### 3.14 Graph algorithm traces

`bfs` , `dfs` , and `dijkstra` render algorithm steps directly on the graph. Green nodes are visited, blue is current, yellow/orange nodes are queued or stacked, and the blue edge is the neighbor currently being inspected. When `dijkstra` reaches a target, its final step also highlights every edge in the shortest path. `source` is required; `target` is optional so a trace can either stop at a goal or traverse every reachable node.

```

1 #bfs(adjacency, source, target: none, directed: true, goal-test: "discovery",
   columns: 1, captions: true, language: "en", messages: (:), ..graph-options)
2 #dfs(adjacency, source, target: none, directed: true, columns: 1, captions: true,
   language: "en", messages: (:), ..graph-options)
3 #dijkstra(adjacency, source, target: none, directed: true, columns: 1, captions: true,
   language: "en", messages: (:), ..graph-options)

```

 Typst

| Argument               | Type                      | Default     | Description                                                                     |
|------------------------|---------------------------|-------------|---------------------------------------------------------------------------------|
| <code>adjacency</code> | dictionary                | (required)  | Same graph adjacency dictionary accepted by <code>graph</code> .                |
| <code>source</code>    | str                       | (required)  | Starting node identity.                                                         |
| <code>target</code>    | none / str                | none        | Optional early-stop node.                                                       |
| <code>goal-test</code> | "discovery" / "expansion" | "discovery" | BFS only. Stop when the target is first enqueued, or wait until it is dequeued. |

|                                                                                                                |                               |                    |                                                                                                                                                                                 |
|----------------------------------------------------------------------------------------------------------------|-------------------------------|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>directed</code>                                                                                          | <code>bool</code>             | <code>true</code>  | Whether adjacency pairs are directed.                                                                                                                                           |
| <code>columns</code>                                                                                           | <code>int</code>              | <code>1</code>     | Trace grid columns.                                                                                                                                                             |
| <code>row-gap</code>                                                                                           | <code>length</code>           | <code>0.8em</code> | Gap between trace cells.                                                                                                                                                        |
| <code>captions</code>                                                                                          | <code>bool</code>             | <code>true</code>  | Show generated step captions.                                                                                                                                                   |
| <code>labels</code> / <code>positions</code> /<br><code>layout</code> / <code>radius</code> / <code>gap</code> | same as<br><code>graph</code> | same               | Node display and placement options.                                                                                                                                             |
| <code>edge-customizations</code> /<br><code>node-customizations</code> /<br><code>node-labels</code>           | same as<br><code>graph</code> | same               | Base customizations merged with algorithm states.                                                                                                                               |
| <code>style</code>                                                                                             | <code>dictionary</code>       | <code>(:)</code>   | Graph style plus <code>visited-style</code> , <code>current-style</code> , <code>queued-style</code> , <code>active-edge-style</code> , and <code>algorithm-label-text</code> . |

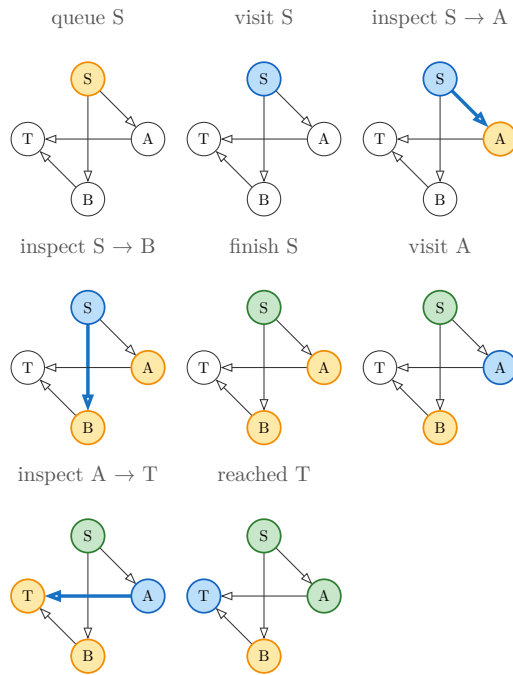
**Note.** For a targeted unweighted BFS, the default `goal-test: "discovery"` stops as soon as an unvisited target is found while expanding a node: that first discovery already gives a shortest path. Choose `goal-test: "expansion"` to keep the traditional dequeue-and-test trace, which may show work on other queued nodes before the target is removed.

**Note.** `dijkstra` treats an unlabeled edge as weight `1` . Weighted entries must use non-negative numeric values such as `("B", 4)` . It draws a distinct purple `d = ...` label beside every node on every step; the default label gap is `0.5` canvas units. `style.node-labels.gap` or per-node `node-labels` options can override it.

```

1 #bfs(
2   ("S": ("A", "B"), "A": ("T",), "B": ("T",), "T": ()),
3   "S", target: "T", columns: 3,
4   style: graph-style(scale: 0.65),
5 ).diagram

```

 Typst


## 4 Hand-composed trees

For abstract diagrams, build a tree by hand with `node(...)` and `subtree(...)` instead of inserting real keys. Draw a node over two elided subtrees, the way you'd draw an AVL height invariant.

```
1 #tree(root, style: (:), edge-customizations: (), node-customizations: (),
  node-labels: (:))
2 #node(label, left: none, right: none, children: none, fill: none)
3 #subtree(label, fill: none, height: none, scale: 1)
```

 Typst

| Argument                         | Type                          | Default          | Description                                                                                                                    |
|----------------------------------|-------------------------------|------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <code>root</code>                | result of <code>node()</code> | (required)       | The tree to render.                                                                                                            |
| <code>style</code>               | dictionary                    | <code>(:)</code> | Per-call style override merged over the defaults. See <a href="#">Section 8</a> .                                              |
| <code>edge-customizations</code> | array                         | <code>()</code>  | <code>(parent, child, options)</code> tuples restyling individual tree edges by node label. See <a href="#">Section 3.13</a> . |
| <code>node-customizations</code> | array                         | <code>()</code>  | <code>(node, options)</code> tuples restyling individual nodes by label.                                                       |
| <code>node-labels</code>         | array / dictionary            | <code>(:)</code> | Labels drawn outside individual nodes.                                                                                         |

### Nested keys for style

| Path                                   | Type              | Default                | Description                                                                                                  |
|----------------------------------------|-------------------|------------------------|--------------------------------------------------------------------------------------------------------------|
| <code>style.node-radius</code>         | float             | 0.34                   | Circle radius, or base size for other node shapes, in canvas units.                                          |
| <code>style.node-shape</code>          | str               | "circle"               | "circle" , "square" , "rounded" , "capsule" , "diamond" , or "hexagon" .                                     |
| <code>style.x-gap</code>               | float             | 1.05                   | Horizontal spacing between in-order columns.                                                                 |
| <code>style.y-gap</code>               | float             | 1.2                    | Vertical spacing between depths.                                                                             |
| <code>style.node-stroke</code>         | stroke            | 0.6pt + rgb("#333333") | Node outline.                                                                                                |
| <code>style.node-fill</code>           | color             | white                  | Default node fill.                                                                                           |
| <code>style.edge-stroke</code>         | stroke            | 0.6pt + rgb("#333333") | Parent-child edge stroke.                                                                                    |
| <code>style.edge-arrow</code>          | none / bool / str | none                   | Edge arrowheads: <code>none</code> / <code>false</code> , "end" or <code>true</code> , "start" , or "both" . |
| <code>style.edge-pattern</code>        | str               | "normal"               | "normal" , "dashed" , "dotted" , or "wavy" .                                                                 |
| <code>style.edge-wave-amplitude</code> | float             | 0.07                   | Wave height in canvas units.                                                                                 |
| <code>style.edge-wave-step</code>      | float             | 0.14                   | Approximate wavelength in canvas units.                                                                      |
| <code>style.node-text</code>           | dictionary        | (size: 9pt)            | Node text dictionary. See nested keys below.                                                                 |

|                                    |            |                                                    |                                                                                           |
|------------------------------------|------------|----------------------------------------------------|-------------------------------------------------------------------------------------------|
| <code>style.value-text</code>      | dictionary | inherits <code>node-text</code>                    | Value text overrides.                                                                     |
| <code>style.label-text</code>      | dictionary | ( <code>fill: "#555555"</code> ) <code>rgb(</code> | Annotation text dictionary. See nested keys below.                                        |
| <code>style.edge-label-text</code> | dictionary | inherits <code>label-text</code>                   | Edge-label typography.                                                                    |
| <code>style.operation-text</code>  | dictionary | ( <code>size: 8pt</code> )                         | Operation-arrow caption typography.                                                       |
| <code>style.scale</code>           | float      | 1.0                                                | Uniform scale on <code>.diagram</code> , <code>.before</code> , and <code>.after</code> . |
| <code>style.diff-colors</code>     | bool       | true                                               | <code>false</code> keeps operation marks but uses normal fills.                           |

#### Nested keys for `style.node-text`

| Path                                  | Type        | Default  | Description                                    |
|---------------------------------------|-------------|----------|------------------------------------------------|
| <code>style.node-text.size</code>     | length      | inherits | Text size.                                     |
| <code>style.node-text.color</code>    | color       | inherits | Text color.                                    |
| <code>style.node-text.rotation</code> | angle       | 0deg     | Text rotation applied by the diagram renderer. |
| <code>style.node-text.font</code>     | str / array | inherits | Typst text font selection.                     |
| <code>style.node-text.weight</code>   | str / int   | inherits | Typst text weight.                             |

#### Nested keys for `style.label-text`

| Path                                   | Type        | Default                                  | Description                                                                                          |
|----------------------------------------|-------------|------------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>style.label-text.size</code>     | length      | 85% of <code>style.node-text.size</code> | Label text size.                                                                                     |
| <code>style.label-text.color</code>    | color       | <code>rgb("#555555")</code>              | Label text color.                                                                                    |
| <code>style.label-text.rotation</code> | angle / str | 0deg                                     | Label rotation. Tree and graph edge labels also accept <code>"edge"</code> to follow the edge angle. |
| <code>style.label-text.font</code>     | str / array | inherits                                 | Typst text font selection.                                                                           |
| <code>style.label-text.weight</code>   | str / int   | inherits                                 | Typst text weight.                                                                                   |

#### Nested subtree style keys

| Path                     | Type  | Default | Description                                               |
|--------------------------|-------|---------|-----------------------------------------------------------|
| <code>style.tri-w</code> | float | 1.2     | Triangle base width before <code>subtree(scale:)</code> . |
| <code>style.tri-h</code> | float | 1.4     | Triangle height before <code>subtree(scale:)</code> .     |

#### Nested keys for diff-highlight dictionaries

| Path | Type | Default | Description |
|------|------|---------|-------------|
|------|------|---------|-------------|

|                                        |                       |                                |                                                                                                                                                                                     |
|----------------------------------------|-----------------------|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>style.new-style</code>           | color<br>dictionary / | <code>rgb("#C8E6C9")</code>    | Node/cell added by an operation. A color is shorthand for <code>(fill: color)</code> .                                                                                              |
| <code>style.path-style</code>          | color<br>dictionary / | <code>rgb("#FFE9A8")</code>    | Traversal or sift path.                                                                                                                                                             |
| <code>style.remove-style</code>        | color<br>dictionary / | <code>rgb("#FFCDD2")</code>    | Node/cell removed by an operation.                                                                                                                                                  |
| <code>style.rotate-style</code>        | color<br>dictionary / | <code>rgb("#BBDEFB")</code>    | AVL rotation nodes and visible subtree roots.                                                                                                                                       |
| <code>style.*-style.fill</code>        | color                 | mark default                   | Fill for the marked node/cell.                                                                                                                                                      |
| <code>style.*-style.shape</code>       | str                   | <code>style.node-shape</code>  | Marked tree/heap node shape: <code>"circle"</code> , <code>"square"</code> , <code>"rounded"</code> , <code>"capsule"</code> , <code>"diamond"</code> , or <code>"hexagon"</code> . |
| <code>style.*-style.stroke</code>      | stroke                | normal stroke                  | Marked node/cell outline.                                                                                                                                                           |
| <code>style.*-style.node-radius</code> | float                 | <code>style.node-radius</code> | Marked tree/heap node radius.                                                                                                                                                       |
| <code>style.*-style.text</code>        | dictionary            | <code>style.node-text</code>   | Marked node/cell text overrides. Accepts <code>size</code> , <code>color</code> , <code>font</code> , and <code>weight</code> .                                                     |

### Nested keys for `edge-customizations`

| Path                                               | Type                    | Default  | Description                                                                                                                                      |
|----------------------------------------------------|-------------------------|----------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>edge-customizations[]</code>                 | tuple                   | n/a      | One <code>(from, to, options)</code> tuple. For trees, <code>from</code> / <code>to</code> are parent and child values; for graphs, node labels. |
| <code>edge-customizations[].from</code>            | content                 | required | Source node identity.                                                                                                                            |
| <code>edge-customizations[].to</code>              | content                 | required | Target node identity.                                                                                                                            |
| <code>edge-customizations[].options</code>         | dictionary              | required | Per-edge options.                                                                                                                                |
| <code>edge-customizations[].options.stroke</code>  | stroke                  | none     | Full stroke override. Wins over <code>color</code> if both are set.                                                                              |
| <code>edge-customizations[].options.color</code>   | color                   | none     | Paint only, at the default thickness.                                                                                                            |
| <code>edge-customizations[].options.pattern</code> | str                     | "normal" | <code>"normal"</code> , <code>"dashed"</code> , <code>"dotted"</code> , or <code>"wavy"</code> .                                                 |
| <code>edge-customizations[].options.arrow</code>   | none / bool / str       | none     | Per-edge arrowheads: <code>none</code> , <code>"end"</code> / <code>true</code> , <code>"start"</code> , or <code>"both"</code> .                |
| <code>edge-customizations[].options.bend</code>    | str / bool              | false    | <code>"left"</code> or <code>"right"</code> curves the edge relative to its travel direction. <code>false</code> keeps it straight.              |
| <code>edge-customizations[].options.angle</code>   | angle                   | 25deg    | How sharply a bent edge curves. Ignored when <code>bend</code> is <code>false</code> .                                                           |
| <code>edge-customizations[].options.label</code>   | content<br>dictionary / | none     | Tree edge label content, or graph edge-label text overrides. A dictionary can include <code>content</code> for trees.                            |

Nested keys for `edge-customizations[].options.label`

| Path                                                      | Type        | Default                                | Description                                                                                 |
|-----------------------------------------------------------|-------------|----------------------------------------|---------------------------------------------------------------------------------------------|
| <code>edge-customizations[].options.label.size</code>     | length      | <code>style.label-text.size</code>     | Edge-label size.                                                                            |
| <code>edge-customizations[].options.label.content</code>  | content     | tree only                              | Tree edge label content. Graph edge labels come from adjacency entries such as ("B", [4]) . |
| <code>edge-customizations[].options.label.color</code>    | color       | <code>style.label-text.color</code>    | Edge-label text color.                                                                      |
| <code>edge-customizations[].options.label.rotation</code> | angle / str | <code>style.label-text.rotation</code> | An angle, or "edge" to follow the tree/graph edge angle.                                    |
| <code>edge-customizations[].options.label.font</code>     | str / array | <code>style.label-text.font</code>     | Typst text font selection.                                                                  |
| <code>edge-customizations[].options.label.weight</code>   | str / int   | <code>style.label-text.weight</code>   | Typst text weight.                                                                          |

Nested keys for `node-customizations`

| Path                                                   | Type       | Default                        | Description                                                                                 |
|--------------------------------------------------------|------------|--------------------------------|---------------------------------------------------------------------------------------------|
| <code>node-customizations[]</code>                     | tuple      | n/a                            | One (node, options) tuple.                                                                  |
| <code>node-customizations[].node</code>                | content    | required                       | Hand-composed node label.                                                                   |
| <code>node-customizations[].options</code>             | dictionary | required                       | Per-node drawing options.                                                                   |
| <code>node-customizations[].options.fill</code>        | color      | normal fill                    | Node fill.                                                                                  |
| <code>node-customizations[].options.stroke</code>      | stroke     | normal stroke                  | Node outline.                                                                               |
| <code>node-customizations[].options.shape</code>       | str        | <code>style.node-shape</code>  | Trees/graphs only: "circle" , "square" , "rounded" , "capsule" , "diamond" , or "hexagon" . |
| <code>node-customizations[].options.node-radius</code> | float      | <code>style.node-radius</code> | Trees/graphs only. Shape size.                                                              |
| <code>node-customizations[].options.text</code>        | dictionary | <code>style.node-text</code>   | Text style merged into the node text.                                                       |

Nested keys for `node-labels`

| Path                                      | Type                 | Default                  | Description                                                                                             |
|-------------------------------------------|----------------------|--------------------------|---------------------------------------------------------------------------------------------------------|
| <code>node-labels[]</code>                | tuple                | n/a                      | One (node, label) tuple.                                                                                |
| <code>node-labels[].node</code>           | content              | required                 | Hand-composed node label.                                                                               |
| <code>node-labels[].label</code>          | content / dictionary | required                 | Bare content for quick labels, or a dictionary with <code>content</code> plus style and placement keys. |
| <code>node-labels[].label.content</code>  | content              | required for dict labels | Label body. <code>body</code> is also accepted.                                                         |
| <code>node-labels[].label.position</code> | str / angle          | "right"                  | "right" , "left" , "top" , "bottom" , or an angle.                                                      |

|                                           |             |                                        |                                                     |
|-------------------------------------------|-------------|----------------------------------------|-----------------------------------------------------|
| <code>node-labels[].label.offset</code>   | point       | (0, 0)                                 | Extra (dx, dy) shift after the position is applied. |
| <code>node-labels[].label.size</code>     | length      | <code>style.label-text.size</code>     | Node-label size.                                    |
| <code>node-labels[].label.color</code>    | color       | <code>style.label-text.color</code>    | Node-label text color.                              |
| <code>node-labels[].label.rotation</code> | angle       | <code>style.label-text.rotation</code> | Rotation of the label text itself.                  |
| <code>node-labels[].label.font</code>     | str / array | <code>style.label-text.font</code>     | Typst text font selection.                          |
| <code>node-labels[].label.weight</code>   | str / int   | <code>style.label-text.weight</code>   | Typst text weight.                                  |

### Nested keys for `style.node-labels`

| Path                                    | Type        | Default                                | Description                                              |
|-----------------------------------------|-------------|----------------------------------------|----------------------------------------------------------|
| <code>style.node-labels.position</code> | str / angle | "right"                                | Default position for all node labels in this diagram.    |
| <code>style.node-labels.offset</code>   | point       | (0, 0)                                 | Default extra (dx, dy) shift for all node labels.        |
| <code>style.node-labels.gap</code>      | float       | 0.22                                   | Default distance from the node boundary in canvas units. |
| <code>style.node-labels.size</code>     | length      | <code>style.label-text.size</code>     | Default node-label text size.                            |
| <code>style.node-labels.color</code>    | color       | <code>style.label-text.color</code>    | Default node-label text color.                           |
| <code>style.node-labels.font</code>     | str / array | inherits                               | Typst text font selection.                               |
| <code>style.node-labels.weight</code>   | str / int   | inherits                               | Typst text weight.                                       |
| <code>style.node-labels.rotation</code> | angle       | <code>style.label-text.rotation</code> | Default rotation of the label text itself.               |

`node(label, left:, right:, children:, fill:)` : a node with any content label. Use `left` / `right` for binary trees, or `children` for a multiway tree such as a B-tree schematic.

| Argument              | Type                                                             | Default           | Description                                                                                                 |
|-----------------------|------------------------------------------------------------------|-------------------|-------------------------------------------------------------------------------------------------------------|
| <code>label</code>    | content                                                          | (required)        | The node's label; any content, not just a key.                                                              |
| <code>left</code>     | <code>none</code> / <code>node()</code> / <code>subtree()</code> | <code>none</code> | Left child.                                                                                                 |
| <code>right</code>    | <code>none</code> / <code>node()</code> / <code>subtree()</code> | <code>none</code> | Right child.                                                                                                |
| <code>children</code> | <code>none</code> / array                                        | <code>none</code> | Multiway children. When set, this replaces <code>left</code> / <code>right</code> for layout and rendering. |
| <code>fill</code>     | <code>none</code> / color                                        | <code>none</code> | Fill tint for this node.                                                                                    |

#### Nested keys for `node()` children

| Path                     | Type                                                             | Default                                                                           | Description                                           |
|--------------------------|------------------------------------------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------|
| <code>left.label</code>  | content                                                          | required when <code>left</code> is <code>node()</code> or <code>subtree()</code>  | Label drawn inside the left child.                    |
| <code>left.left</code>   | <code>none</code> / <code>node()</code> / <code>subtree()</code> | <code>none</code>                                                                 | Nested left child.                                    |
| <code>left.right</code>  | <code>none</code> / <code>node()</code> / <code>subtree()</code> | <code>none</code>                                                                 | Nested right child.                                   |
| <code>right.label</code> | content                                                          | required when <code>right</code> is <code>node()</code> or <code>subtree()</code> | Label drawn inside the right child.                   |
| <code>right.left</code>  | <code>none</code> / <code>node()</code> / <code>subtree()</code> | <code>none</code>                                                                 | Nested left child.                                    |
| <code>right.right</code> | <code>none</code> / <code>node()</code> / <code>subtree()</code> | <code>none</code>                                                                 | Nested right child.                                   |
| <code>children[]</code>  | <code>node()</code> / <code>subtree()</code>                     | n/a                                                                               | One child in a multiway tree, laid out left to right. |

`subtree(label, fill:, height:, scale:)` : a triangle leaf standing in for an elided subtree.

| Argument            | Type            | Default    | Description                                                                            |
|---------------------|-----------------|------------|----------------------------------------------------------------------------------------|
| <code>label</code>  | content         | (required) | Label drawn inside the triangle.                                                       |
| <code>fill</code>   | none<br>color   | /<br>none  | Outline and label tint.                                                                |
| <code>height</code> | none<br>content | /<br>none  | Optional side bracket labeling the subtree's height.                                   |
| <code>scale</code>  | float           | 1          | Resizes the triangle relative to the theme's <code>tri-w</code> / <code>tri-h</code> . |

#### Style keys used by `subtree()`

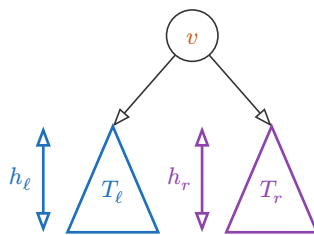
| Path                         | Type       | Default     | Description                                               |
|------------------------------|------------|-------------|-----------------------------------------------------------|
| <code>style.tri-w</code>     | float      | 1.2         | Triangle base width before <code>subtree(scale:)</code> . |
| <code>style.tri-h</code>     | float      | 1.4         | Triangle height before <code>subtree(scale:)</code> .     |
| <code>style.node-text</code> | dictionary | (size: 9pt) | Triangle label and height-bracket text styling.           |

```

1 #tree(
2   node(
3     text(fill: rgb("#C75B12"))[$v$],
4     left: subtree($T_ell$, fill: rgb("#1F6FBF"), height: $h_ell$),
5     right: subtree($T_r$, fill: rgb("#8E44AD"), height: $h_r$),
6   ),
7   style: (edge-arrow: true),
8 )

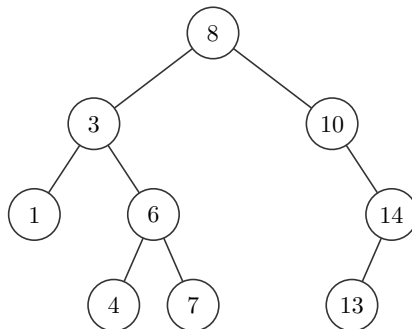
```

**t** Typst



Nest `node(...)` calls to draw a full manual tree. The indentation below mirrors the tree shape: each child is written inside its parent's `left:` or `right:` argument.

```
1  #tree(  
2    node(  
3      [8],  
4      left: node(  
5        [3],  
6        left: node([1]),  
7        right: node(  
8          [6],  
9          left: node([4]),  
10         right: node([7]),  
11      ),  
12    ),  
13    right: node(  
14      [10],  
15      right: node([14], left: node([13])),  
16    ),  
17  ),  
18 )
```

 Typst

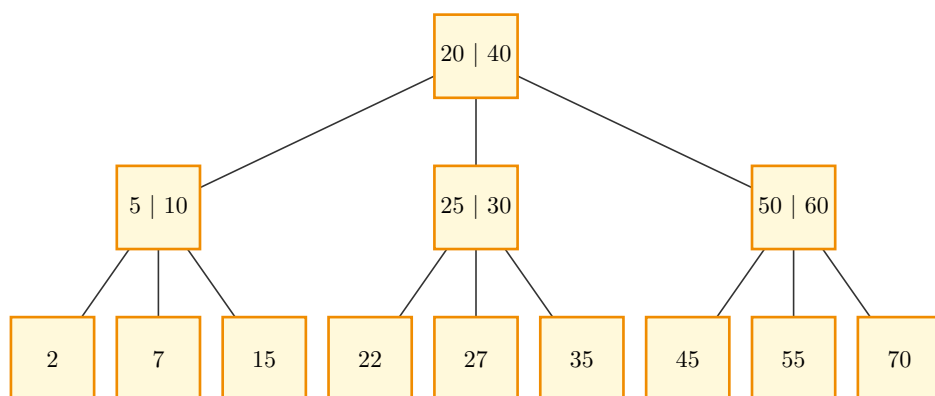
Use `children: (...)` for B-tree-style or n-ary schematics.

```

1  #tree(
2    node([20 | 40], children: (
3      node([5 | 10], children: (
4        node([2]),
5        node([7]),
6        node([15]),
7      )),
8    node([25 | 30], children: (
9      node([22]),
10     node([27]),
11     node([35]),
12   )),
13   node([50 | 60], children: (
14     node([45]),
15     node([55]),
16     node([70]),
17   )),
18  )),
19  style: (
20    node-shape: "square",
21    node-radius: 0.55,
22    node-fill: rgb("#FFF9DB"),
23    node-stroke: 1pt + rgb("#F08C00"),
24    x-gap: 1.4,
25    y-gap: 2.0,
26  ),
27 )

```

**t** Typst

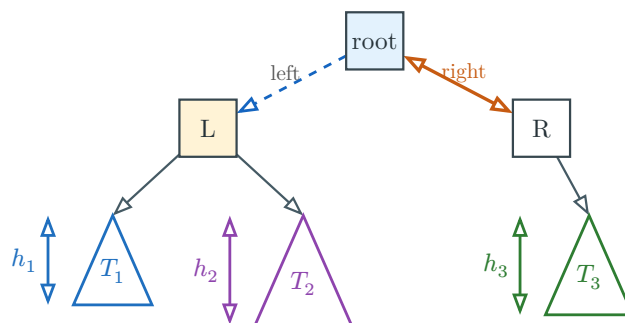


Argument coverage: hand-composed trees combine nested `node()` calls, `subtree()` leaves, per-node fills, subtree height labels, tree style, and edge customizations keyed by the visible node labels.

```

1  #tree(
2    node(
3      [root],
4      fill: rgb("#E3F2FD"),
5      left: node(
6        [L],
7        fill: rgb("#FFF3D6"),
8        left: subtree($T_1$, fill: rgb("#1F6FBF"), height: $h_1$, scale: 0.9),
9        right: subtree($T_2$, fill: rgb("#8E44AD"), height: $h_2$, scale: 1.1),
10     ),
11     right: node(
12       [R],
13       right: subtree($T_3$, fill: rgb("#2E7D32"), height: $h_3$),
14     ),
15   ),
16   style: (
17     node-shape: "square",
18     node-radius: 0.36,
19     x-gap: 1.2,
20     y-gap: 1.1,
21     tri-w: 1.1,
22     tri-h: 1.25,
23     node-stroke: 0.8pt + rgb("#37474F"),
24     node-text: (size: 9pt, color: rgb("#263238")),
25     edge-stroke: 0.8pt + rgb("#455A64"),
26     edge-arrow: "end",
27     scale: 1.05,
28   ),
29   edge-customizations: (
30     ([root], [L], (pattern: "dashed", color: rgb("#1565C0"), label: [left])),
31     ([root], [R], (stroke: 1.2pt + rgb("#C75B12"), arrow: "both", label: (content:
32       [right], color: rgb("#C75B12")))),
33   )

```



## 5 Operation transitions

`transition` is one-shot: build the structure from keys, apply the operation, and render the before state, an arrow, and the derived after state with the diff highlighted, in one call, no variable required. To keep operating on a structure across several calls, use the object model in [Section 6](#) instead.

```
1 #transition(variant, keys, op, style: (:), edge-customizations: (), node-
  customizations: (), node-labels: ())
```

 Typst

| Argument                         | Type                                         | Default          | Description                                                                                                                                                                       |
|----------------------------------|----------------------------------------------|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>variant</code>             | <code>str</code>                             | (required)       | "bst" , "avl" , "min-heap" , or "max-heap" .                                                                                                                                      |
| <code>keys</code>                | <code>array</code>                           | (required)       | Keys the structure is built from, inserted in order.                                                                                                                              |
| <code>op</code>                  | <code>operation</code>                       | (required)       | <code>tree-insert(key)</code> , <code>tree-delete(key)</code> , or <code>tree-search(key)</code> for trees. <code>heap-insert(key)</code> or <code>heap-extract</code> for heaps. |
| <code>style</code>               | <code>dictionary</code>                      | <code>(:)</code> | Per-call style override merged over the defaults. See <a href="#">Section 8</a> .                                                                                                 |
| <code>edge-customizations</code> | <code>array</code>                           | <code>()</code>  | Tree transition only. ( <code>parent</code> , <code>child</code> , <code>options</code> ) tuples restyling individual tree edges by value.                                        |
| <code>node-customizations</code> | <code>array</code>                           | <code>()</code>  | Tree transition only. ( <code>node</code> , <code>options</code> ) tuples restyling individual tree nodes by value.                                                               |
| <code>node-labels</code>         | <code>array</code> / <code>dictionary</code> | <code>(:)</code> | Tree transition only. Labels drawn outside individual tree nodes.                                                                                                                 |

### Nested keys for style

| Path                                   | Type                                                     | Default                | Description                                                                                                  |
|----------------------------------------|----------------------------------------------------------|------------------------|--------------------------------------------------------------------------------------------------------------|
| <code>style.node-radius</code>         | <code>float</code>                                       | 0.34                   | Circle radius, or base size for other node shapes, in canvas units.                                          |
| <code>style.node-shape</code>          | <code>str</code>                                         | "circle"               | "circle" , "square" , "rounded" , "capsule" , "diamond" , or "hexagon" .                                     |
| <code>style.x-gap</code>               | <code>float</code>                                       | 1.05                   | Horizontal spacing between in-order columns.                                                                 |
| <code>style.y-gap</code>               | <code>float</code>                                       | 1.2                    | Vertical spacing between depths.                                                                             |
| <code>style.node-stroke</code>         | <code>stroke</code>                                      | 0.6pt + rgb("#333333") | Node outline.                                                                                                |
| <code>style.node-fill</code>           | <code>color</code>                                       | white                  | Default node fill.                                                                                           |
| <code>style.edge-stroke</code>         | <code>stroke</code>                                      | 0.6pt + rgb("#333333") | Parent-child edge stroke.                                                                                    |
| <code>style.edge-arrow</code>          | <code>none</code> / <code>bool</code> / <code>str</code> | none                   | Edge arrowheads: <code>none</code> / <code>false</code> , "end" or <code>true</code> , "start" , or "both" . |
| <code>style.edge-pattern</code>        | <code>str</code>                                         | "normal"               | "normal" , "dashed" , "dotted" , or "wavy" .                                                                 |
| <code>style.edge-wave-amplitude</code> | <code>float</code>                                       | 0.07                   | Wave height in canvas units.                                                                                 |

|                                    |            |                                      |                                                                                           |
|------------------------------------|------------|--------------------------------------|-------------------------------------------------------------------------------------------|
| <code>style.edge-wave-step</code>  | float      | 0.14                                 | Approximate wavelength in canvas units.                                                   |
| <code>style.node-text</code>       | dictionary | (size: 9pt)                          | Node text dictionary. See nested keys below.                                              |
| <code>style.value-text</code>      | dictionary | inherits <code>node-text</code>      | Value text overrides.                                                                     |
| <code>style.label-text</code>      | dictionary | (fill: <code>rgb("#555555")</code> ) | Annotation text dictionary. See nested keys below.                                        |
| <code>style.edge-label-text</code> | dictionary | inherits <code>label-text</code>     | Edge-label typography.                                                                    |
| <code>style.operation-text</code>  | dictionary | (size: 8pt)                          | Operation-arrow caption typography.                                                       |
| <code>style.scale</code>           | float      | 1.0                                  | Uniform scale on <code>.diagram</code> , <code>.before</code> , and <code>.after</code> . |
| <code>style.diff-colors</code>     | bool       | true                                 | <code>false</code> keeps operation marks but uses normal fills.                           |

#### Nested keys for `style.node-text`

| Path                                  | Type        | Default  | Description                                    |
|---------------------------------------|-------------|----------|------------------------------------------------|
| <code>style.node-text.size</code>     | length      | inherits | Text size.                                     |
| <code>style.node-text.color</code>    | color       | inherits | Text color.                                    |
| <code>style.node-text.rotation</code> | angle       | 0deg     | Text rotation applied by the diagram renderer. |
| <code>style.node-text.font</code>     | str / array | inherits | Typst text font selection.                     |
| <code>style.node-text.weight</code>   | str / int   | inherits | Typst text weight.                             |

#### Nested keys for `style.label-text`

| Path                                   | Type        | Default                                  | Description                                                                                          |
|----------------------------------------|-------------|------------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>style.label-text.size</code>     | length      | 85% of <code>style.node-text.size</code> | Label text size.                                                                                     |
| <code>style.label-text.color</code>    | color       | <code>rgb("#555555")</code>              | Label text color.                                                                                    |
| <code>style.label-text.rotation</code> | angle / str | 0deg                                     | Label rotation. Tree and graph edge labels also accept <code>"edge"</code> to follow the edge angle. |
| <code>style.label-text.font</code>     | str / array | inherits                                 | Typst text font selection.                                                                           |
| <code>style.label-text.weight</code>   | str / int   | inherits                                 | Typst text weight.                                                                                   |

#### Nested keys for diff-highlight dictionaries

| Path                         | Type               | Default                     | Description                                                                             |
|------------------------------|--------------------|-----------------------------|-----------------------------------------------------------------------------------------|
| <code>style.new-style</code> | color / dictionary | <code>rgb("#C8E6C9")</code> | Node/cell added by an operation. A color is shorthand for (fill: <code>color</code> ) . |

|                                        |                       |                                |                                                                                                                                                                                     |
|----------------------------------------|-----------------------|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>style.path-style</code>          | color<br>dictionary / | <code>rgb("#FFE9A8")</code>    | Traversal or sift path.                                                                                                                                                             |
| <code>style.remove-style</code>        | color<br>dictionary / | <code>rgb("#FFCDD2")</code>    | Node/cell removed by an operation.                                                                                                                                                  |
| <code>style.rotate-style</code>        | color<br>dictionary / | <code>rgb("#BBDEFB")</code>    | AVL rotation nodes and visible subtree roots.                                                                                                                                       |
| <code>style.*-style.fill</code>        | color                 | mark default                   | Fill for the marked node/cell.                                                                                                                                                      |
| <code>style.*-style.shape</code>       | str                   | <code>style.node-shape</code>  | Marked tree/heap node shape: <code>"circle"</code> , <code>"square"</code> , <code>"rounded"</code> , <code>"capsule"</code> , <code>"diamond"</code> , or <code>"hexagon"</code> . |
| <code>style.*-style.stroke</code>      | stroke                | normal stroke                  | Marked node/cell outline.                                                                                                                                                           |
| <code>style.*-style.node-radius</code> | float                 | <code>style.node-radius</code> | Marked tree/heap node radius.                                                                                                                                                       |
| <code>style.*-style.text</code>        | dictionary            | <code>style.node-text</code>   | Marked node/cell text overrides. Accepts <code>size</code> , <code>color</code> , <code>font</code> , and <code>weight</code> .                                                     |

### Nested keys for `edge-customizations`

| Path                                               | Type                    | Default  | Description                                                                                                                                      |
|----------------------------------------------------|-------------------------|----------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>edge-customizations[]</code>                 | tuple                   | n/a      | One <code>(from, to, options)</code> tuple. For trees, <code>from</code> / <code>to</code> are parent and child values; for graphs, node labels. |
| <code>edge-customizations[].from</code>            | content                 | required | Source node identity.                                                                                                                            |
| <code>edge-customizations[].to</code>              | content                 | required | Target node identity.                                                                                                                            |
| <code>edge-customizations[].options</code>         | dictionary              | required | Per-edge options.                                                                                                                                |
| <code>edge-customizations[].options.stroke</code>  | stroke                  | none     | Full stroke override. Wins over <code>color</code> if both are set.                                                                              |
| <code>edge-customizations[].options.color</code>   | color                   | none     | Paint only, at the default thickness.                                                                                                            |
| <code>edge-customizations[].options.pattern</code> | str                     | "normal" | <code>"normal"</code> , <code>"dashed"</code> , <code>"dotted"</code> , or <code>"wavy"</code> .                                                 |
| <code>edge-customizations[].options.arrow</code>   | none / bool / str       | none     | Per-edge arrowheads: <code>none</code> , <code>"end"</code> / <code>true</code> , <code>"start"</code> , or <code>"both"</code> .                |
| <code>edge-customizations[].options.bend</code>    | str / bool              | false    | <code>"left"</code> or <code>"right"</code> curves the edge relative to its travel direction. <code>false</code> keeps it straight.              |
| <code>edge-customizations[].options.angle</code>   | angle                   | 25deg    | How sharply a bent edge curves. Ignored when <code>bend</code> is <code>false</code> .                                                           |
| <code>edge-customizations[].options.label</code>   | content<br>dictionary / | none     | Tree edge label content, or graph edge-label text overrides. A dictionary can include <code>content</code> for trees.                            |

### Nested keys for `edge-customizations[].options.label`

| Path                                         | Type        | Default                   | Description                                                                                 |
|----------------------------------------------|-------------|---------------------------|---------------------------------------------------------------------------------------------|
| edge-customizations[].options.label.size     | length      | style.label-text.size     | Edge-label size.                                                                            |
| edge-customizations[].options.label.content  | content     | tree only                 | Tree edge label content. Graph edge labels come from adjacency entries such as ("B", [4]) . |
| edge-customizations[].options.label.color    | color       | style.label-text.color    | Edge-label text color.                                                                      |
| edge-customizations[].options.label.rotation | angle / str | style.label-text.rotation | An angle, or "edge" to follow the tree/graph edge angle.                                    |
| edge-customizations[].options.label.font     | str / array | style.label-text.font     | Typst text font selection.                                                                  |
| edge-customizations[].options.label.weight   | str / int   | style.label-text.weight   | Typst text weight.                                                                          |

### Nested keys for node-customizations

| Path                                      | Type       | Default           | Description                                                                                 |
|-------------------------------------------|------------|-------------------|---------------------------------------------------------------------------------------------|
| node-customizations[]                     | tuple      | n/a               | One (node, options) tuple.                                                                  |
| node-customizations[].node                | content    | required          | Tree node value.                                                                            |
| node-customizations[].options             | dictionary | required          | Per-node drawing options.                                                                   |
| node-customizations[].options.fill        | color      | normal fill       | Node fill.                                                                                  |
| node-customizations[].options.stroke      | stroke     | normal stroke     | Node outline.                                                                               |
| node-customizations[].options.shape       | str        | style.node-shape  | Trees/graphs only: "circle" , "square" , "rounded" , "capsule" , "diamond" , or "hexagon" . |
| node-customizations[].options.node-radius | float      | style.node-radius | Trees/graphs only. Shape size.                                                              |
| node-customizations[].options.text        | dictionary | style.node-text   | Text style merged into the node text.                                                       |

### Nested keys for node-labels

| Path                         | Type                 | Default                  | Description                                                                                |
|------------------------------|----------------------|--------------------------|--------------------------------------------------------------------------------------------|
| node-labels[]                | tuple                | n/a                      | One (node, label) tuple.                                                                   |
| node-labels[].node           | content              | required                 | Tree node value.                                                                           |
| node-labels[].label          | content / dictionary | required                 | Bare content for quick labels, or a dictionary with content plus style and placement keys. |
| node-labels[].label.content  | content              | required for dict labels | Label body. body is also accepted.                                                         |
| node-labels[].label.position | str / angle          | "right"                  | "right" , "left" , "top" , "bottom" , or an angle.                                         |
| node-labels[].label.offset   | point                | (0, 0)                   | Extra (dx, dy) shift after the position is applied.                                        |

|                                           |             |                                        |                                    |
|-------------------------------------------|-------------|----------------------------------------|------------------------------------|
| <code>node-labels[].label.size</code>     | length      | <code>style.label-text.size</code>     | Node-label size.                   |
| <code>node-labels[].label.color</code>    | color       | <code>style.label-text.color</code>    | Node-label text color.             |
| <code>node-labels[].label.rotation</code> | angle       | <code>style.label-text.rotation</code> | Rotation of the label text itself. |
| <code>node-labels[].label.font</code>     | str / array | <code>style.label-text.font</code>     | Typst text font selection.         |
| <code>node-labels[].label.weight</code>   | str / int   | <code>style.label-text.weight</code>   | Typst text weight.                 |

#### Nested keys for `style.node-labels`

| Path                                    | Type        | Default                                | Description                                              |
|-----------------------------------------|-------------|----------------------------------------|----------------------------------------------------------|
| <code>style.node-labels.position</code> | str / angle | "right"                                | Default position for all node labels in this diagram.    |
| <code>style.node-labels.offset</code>   | point       | (0, 0)                                 | Default extra (dx, dy) shift for all node labels.        |
| <code>style.node-labels.gap</code>      | float       | 0.22                                   | Default distance from the node boundary in canvas units. |
| <code>style.node-labels.size</code>     | length      | <code>style.label-text.size</code>     | Default node-label text size.                            |
| <code>style.node-labels.color</code>    | color       | <code>style.label-text.color</code>    | Default node-label text color.                           |
| <code>style.node-labels.font</code>     | str / array | inherits                               | Typst text font selection.                               |
| <code>style.node-labels.weight</code>   | str / int   | inherits                               | Typst text weight.                                       |
| <code>style.node-labels.rotation</code> | angle       | <code>style.label-text.rotation</code> | Default rotation of the label text itself.               |

The highlight colors are: yellow for the traversal path, green for an added node, red for a removed node, and blue for the nodes and visible subtree roots in an AVL rotation schematic.

## 5.1 Tree operations: `tree-insert()`, `tree-delete()`, `tree-search()`

Each is a constructor. Call it with a key and it returns the closure `transition` applies. These aren't object methods. [Section 6](#) covers the `.insert` / `.delete` / `.search` fields on a `bst` / `avl` object, which wrap these same operations.

| Argument                | Type                                    | Default          | Description                                                                                                                         |
|-------------------------|-----------------------------------------|------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <code>key</code>        | <code>int</code> / <code>content</code> | (required)       | The key to insert, delete, or search for.                                                                                           |
| <code>rebalance</code>  | dictionary                              | <code>(:)</code> | <code>tree-insert</code> only. ( <code>enabled:</code> , <code>all-steps:</code> ) , both <code>false</code> by default. See below. |
| <code>step-label</code> | <code>content</code>                    | auto             | Overrides the arrow label for this operation step.                                                                                  |

### Nested keys for `rebalance`

| Path                             | Type              | Default            | Description                                                                     |
|----------------------------------|-------------------|--------------------|---------------------------------------------------------------------------------|
| <code>rebalance.enabled</code>   | <code>bool</code> | <code>false</code> | Show the unrotated AVL tree before a rotation.                                  |
| <code>rebalance.all-steps</code> | <code>bool</code> | <code>false</code> | For double rotations, split the inner and outer rotations into separate panels. |

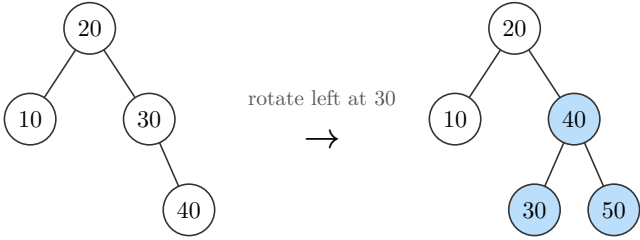
A BST insert highlights the search path on the before state and the new node on the after state.

```
1 #transition("bst", (50, 30, 70, 20, 40), tree-insert(45))
```

Typst

An AVL insert that unbalances the tree triggers a rotation. Every node that changed parent turns blue: the pivot **and** the child promoted above it, plus any subtree that swapped from one to the other. The new node is green.

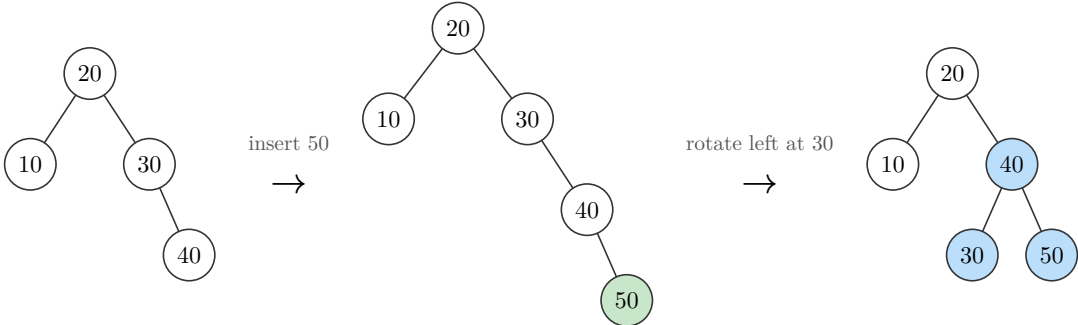
1 #transition("avl", (20, 10, 30, 40), tree-insert(50))



| Argument  | Type | Default | Description                                                                                                                                                                                                                                                            |
|-----------|------|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| enabled   | bool | false   | Adds a panel for the tree right after the plain insert, before any rotation straightens it out, when the AVL fixup actually rotates. A BST insert, or an AVL insert that never unbalances the tree, ignores this and stays 2-panel.                                    |
| all-steps | bool | false   | A <b>double</b> rotation (left-right or right-left) normally collapses straight from the unbalanced tree to the final one. <b>true</b> splits it into its own inner-then-outer panels instead. No effect on a single rotation, since there's only one step either way. |

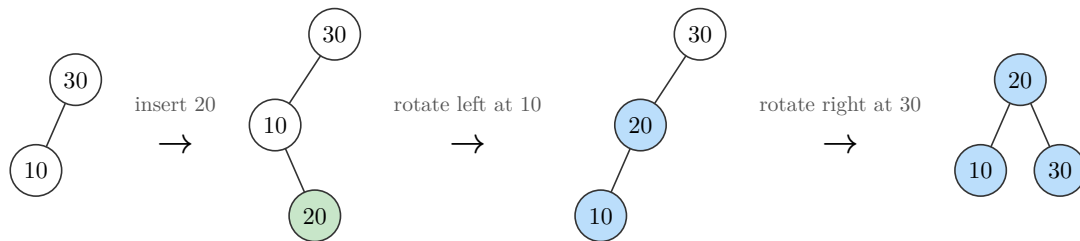
`rebalance: (enabled: true)` splits that same rotation into three panels: the tree right after the plain insert (unbalanced, new node green), then the rotation that fixes it (every reparented node blue).

1 #transition("avl", (20, 10, 30, 40), tree-insert(50, rebalance: (enabled: true)))



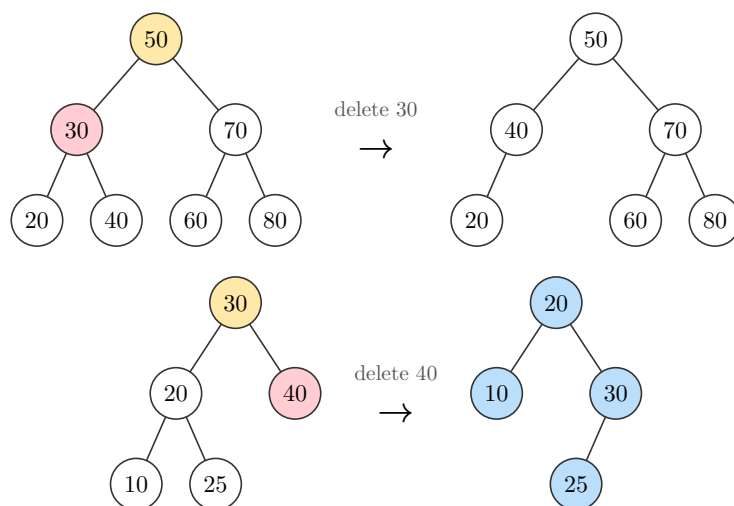
A double rotation, left-right or right-left, needs two rotations to fix. By default `enabled: true` still collapses it to 3 panels, marking every node either rotation touched; `all-steps: true` shows the inner rotation as its own step, for 4 panels total.

```
1 #transition("avl", (30, 10), tree-insert(20, rebalance: (enabled: true, all-
  steps: true)))
```

 Typst


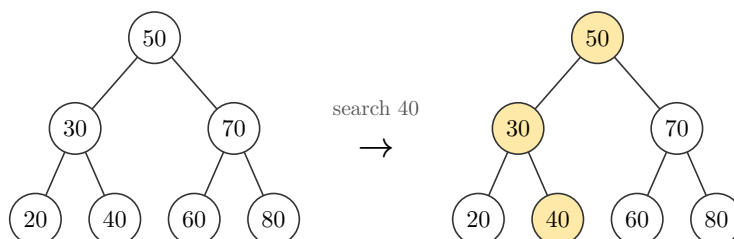
A delete marks the search path on the before state and gives the removed node the remove style. Two-child nodes are replaced by their in-order successor; AVL deletes also rebalance when removal makes a subtree too heavy, marking rotation nodes blue.

```
1 #transition("bst", (50, 30, 70, 20, 40, 60, 80), tree-delete(30))
2 #transition("avl", (30, 20, 40, 10, 25), tree-delete(40))
```

 Typst


A search leaves the before state plain and highlights the traversal path on the after state.

```
1 #transition("bst", (50, 30, 70, 20, 40, 60, 80), tree-search(40))
```

 Typst


## 5.2 Heap operations: `heap-insert()` and `heap-extract`

`heap-insert` is a constructor, like `tree-insert`. `heap-extract` is different: it takes no key (a heap can only pull its root), so it **is** the operation directly, passed without calling it. Object methods such as `(heap-extract)(step-label: [...])` can still override a step label.

| Argument                | Type                                    | Default    | Description                                                                                                                           |
|-------------------------|-----------------------------------------|------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <code>key</code>        | <code>int</code> / <code>content</code> | (required) | The key to insert, <code>heap-insert</code> only. <code>heap-extract</code> takes nothing.                                            |
| <code>step-label</code> | <code>content</code>                    | auto       | Overrides the arrow label. For <code>heap-extract</code> , use it on the object method: <code>(h-extract)(step-label: [...])</code> . |

`heap-insert` appends the key, marks the inserted value green, and highlights the existing sift-up path in yellow:

```
1 #transition("min-heap", (10, 20, 15, 40, 50, 30), heap-insert(5))
```

`heap-extract` removes the root (marked red), moves the last leaf into its place, and highlights the sift-down path in yellow:

```
1 #transition("max-heap", (50, 30, 70, 20, 40, 60, 80), heap-extract)
```

**Note.** Heap operation marks are keyed by array position while rendering, so inserting a value already present in the heap can still highlight only the new node. Tree operation marks remain value-keyed; see [Section 10](#).

## 6 Objects and operations

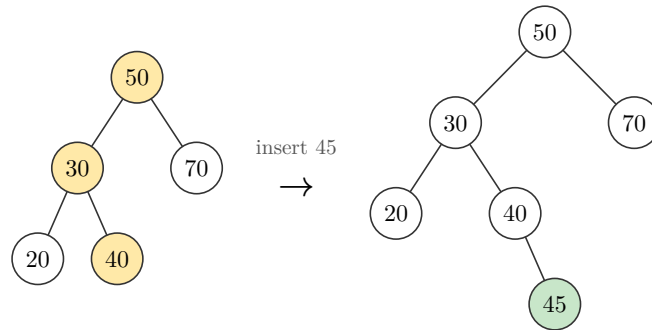
### 6.1 The object model

Every builder in [Section 3](#) returns a dictionary that is both the drawing and the structure you operate on. `.diagram` holds the rendering. An operation field derives the next state: `.insert` / `.delete` / `.search` on trees, `.insert` / `.extract` on heaps, `.push` / `.pop` on stacks, `.enqueue` / `.dequeue` on queues, `.insert` / `.delete` on linked lists and doubly linked lists.

Typst requires parentheses around a callable dictionary field, so calling an operation looks like `(obj.insert)(45)`, not `obj.insert(45)`.

```
1 #let b = bst(50, 30, 70, 20, 40)
2 #(b.insert)(45).diagram
```

**t** Typst



Calling an operation field returns a **step**: a dictionary with five fields.

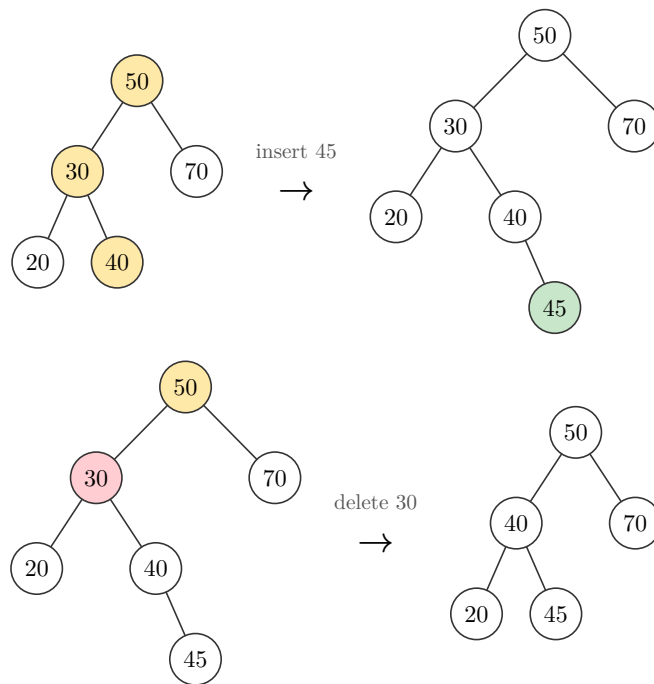
| Field                 | Description                                                                                                                                                                |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>.label</code>   | The arrow's text, e.g. <code>"insert 45"</code> or <code>"extract"</code> .                                                                                                |
| <code>.before</code>  | Rendered picture of the state <b>before</b> the operation, with that side's highlight marks.                                                                               |
| <code>.after</code>   | Rendered picture of the state <b>after</b> the operation, with that side's highlight marks. A picture only: no <code>.insert</code> / <code>.push</code> /etc. of its own. |
| <code>.diagram</code> | The packaged <code>.before</code> → arrow → <code>.after</code> row, what you show by default.                                                                             |
| <code>.result</code>  | The <b>next live object</b> , same shape as what the builder returned: <code>.diagram</code> plus operation fields, ready for the next call.                               |

**Note.** `.result` is fully displayable via `result.diagram`, but plain, with no highlights. Once you have `.result`, it is a fresh object built from the post-operation state, exactly as if you had called `bst(...)` (or `min-heap(...)`, etc.) on those exact keys directly. The diff-marking machinery belongs to the one step that produced it. It doesn't travel with the resulting object, and it doesn't accumulate across a chain of `.result` calls.

## 6.2 Chaining operations

Keep calling operations on `.result` to chain an arbitrary sequence, each with its own fresh highlight:

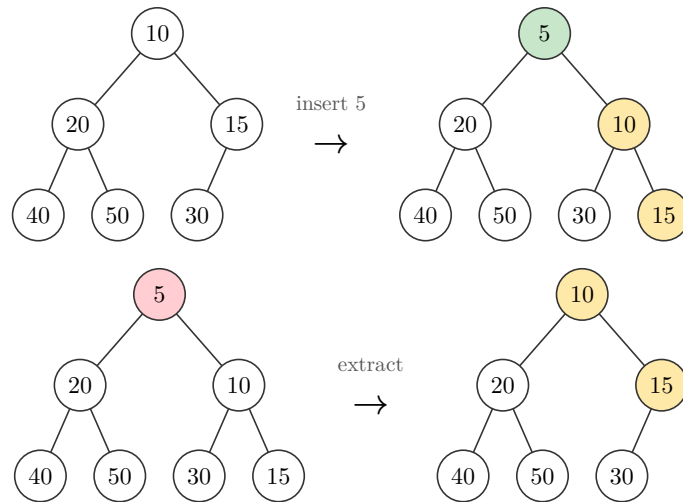
```
1 #let b = bst(50, 30, 70, 20, 40)
2 #let step = (b.insert)(45)
3 #std.stack(spacing: 1em,
4   step.diagram,
5   ((step.result).delete)(30).diagram,
6   )
```

 Typst

Every structure's natural operations chain the same way: heaps ( `.insert` / `.extract` ), stacks ( `.push` / `.pop` ), queues ( `.enqueue` / `.dequeue` ), linked lists ( `.insert` / `.delete` ):

```
1 #let h = min-heap(10, 20, 15, 40, 50, 30)
2 #let step = (h.insert)(5)
3 #std.stack(spacing: 1em,
4   step.diagram,
5   ((step.result).extract()).diagram,
6 )
```

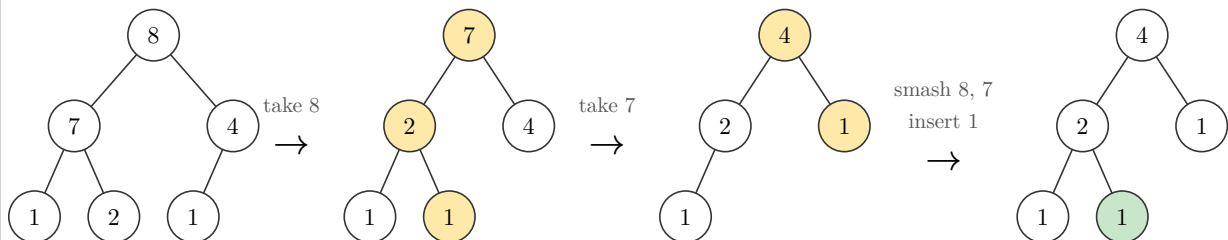
 Typst



Pass `step-label`: to an operation method when a sequence needs a problem-specific arrow label:

```
1 #let h = max-heap(2, 7, 4, 1, 8, 1)
2 #let s1 = (h.extract)(step-label: [take 8])
3 #let s2 = ((s1.result).extract)(step-label: [take 7])
4 #let s3 = ((s2.result).insert)(1, step-label: [smash 8, 7 \ insert 1])
5
6 #sequence(h, s1, s2, s3, mode: "after", columns: 7)
```

 Typst



## 6.3 Wrapped operation sequences

`sequence(..., columns:, mode:)` lays out step objects in a grid. It accepts operation steps directly, so you do not need to write `.diagram` for each one. Use `columns: 1` for a vertical trace, or a larger number to wrap across rows. The default `mode: "all"` shows each full transition row. Compact modes keep operation arrows between states: `mode: "after"` shows highlighted after panels, while `mode: "result"` shows the plain live object after each step, without operation highlights. Put the starting object first when you want the trace to begin with the original structure.

| Argument             | Type           | Default    | Description                                                                                                                                                    |
|----------------------|----------------|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>..steps</code> | step / content | (required) | Operation steps, live objects, or already-rendered content.                                                                                                    |
| <code>columns</code> | int            | 1          | Number of grid columns before wrapping. Compact modes count arrows as columns too.                                                                             |
| <code>gap</code>     | length         | 1em        | Horizontal gap between columns.                                                                                                                                |
| <code>row-gap</code> | length         | 1em        | Vertical gap between rows.                                                                                                                                     |
| <code>mode</code>    | str            | "all"      | "all" / "diagram" renders <code>.diagram</code> compact modes "before" , "after" , and "result" render selected states with labeled arrows between operations. |
| <code>style</code>   | dictionary     | (:)        | Uses <code>style.operation-text</code> for compact-mode arrow captions.                                                                                        |

```
1 #let b = bst(50, 30, 70, 20, 40)
2 #let s1 = (b.insert)(60)
3 #let s2 = ((s1.result).search)(20)
4 #let s3 = ((s2.result).delete)(30)
5
6 #sequence(s1, s2, s3, columns: 1, row-gap: 1.2em)
7 #sequence(b, s1, s2, s3, mode: "result", columns: 7)
```

Typst

The diagram shows a sequence of six binary search trees (BSTs) illustrating the operations performed in the code. The trees are arranged in two rows of three. The first tree is the initial BST with root 50, left child 30, and right child 70. Node 70 has children 20 and 40. The second tree shows the result of inserting 60, with 60 as the right child of 40. The third tree shows the result of searching for 20, with 20 highlighted in yellow. The fourth tree shows the result of deleting 30, with 30 highlighted in red. The fifth tree shows the result of deleting 30, with 30 highlighted in red. The sixth tree shows the final result after deleting 30, with 30 highlighted in red. The trees are connected by arrows indicating the sequence of operations.

`operation-sequence` computes the chain as well as laying it out. Each operation closure receives the current live object and returns one ordinary step. The result exposes `.steps`, the final live `.result`, and `.diagram`.

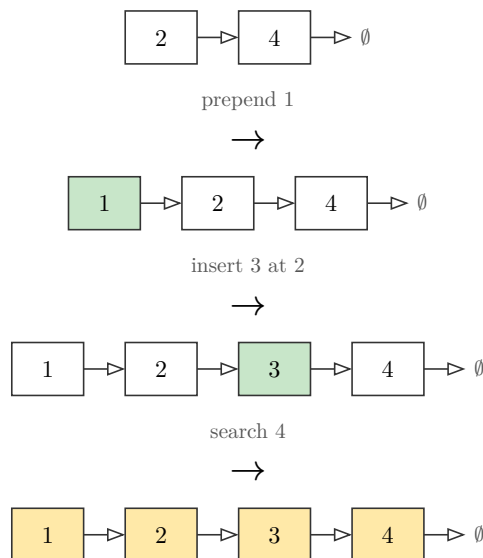
| Argument                                    | Type                          | Default                    | Description                                                           |
|---------------------------------------------|-------------------------------|----------------------------|-----------------------------------------------------------------------|
| <code>initial</code>                        | live object                   | (required)                 | Starting tree, heap, list, stack, queue, or hash table.               |
| <code>..operations</code>                   | function                      | (required)                 | Closures of the form <code>obj =&gt; (obj.operation)(..args)</code> . |
| <code>columns / gap / row-gap / mode</code> | same as <code>sequence</code> | <code>mode: "after"</code> | Layout settings passed to <code>sequence</code> .                     |
| <code>style</code>                          | dictionary                    | <code>(:)</code>           | Typography for operation-arrow captions.                              |

```

1 #operation-sequence(
2   linked-list(2, 4),
3   list => (list.prepend)(1),
4   list => (list.insert)(3, index: 2),
5   list => (list.search)(4),
6   columns: 1,
7 ).diagram

```

**t** Typst



## 6.4 Custom arrows with `op-arrow()`

To compose the arrow yourself instead of using `.diagram`, combine the step's `.before` / `.after` with `op-arrow(label, symbol:, style:)`.

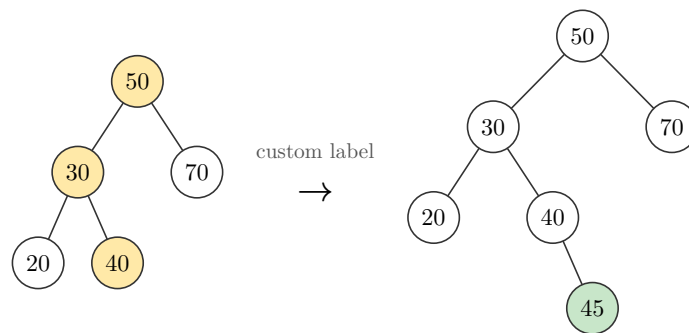
| Argument            | Type       | Default                  | Description                                             |
|---------------------|------------|--------------------------|---------------------------------------------------------|
| <code>label</code>  | content    | (required)               | Text shown above the arrow.                             |
| <code>symbol</code> | content    | <code>\$arrow.r\$</code> | The arrow glyph itself.                                 |
| <code>style</code>  | dictionary | <code>(:)</code>         | Uses <code>style.operation-text</code> for the caption. |

```

1 #let step = (bst(50, 30, 70, 20, 40).insert)(45)
2 #std.stack(dir: ltr, spacing: 1.2em,
3   align(horizon, step.before),
4   op-arrow[custom label],
5   align(horizon, step.after),
6 )

```

**t** Typst



## 7 Localization

Every caption a diagram generates — the operation wording above a transition arrow, the labels in a BFS/DFS/Dijkstra trace, the phase names in a sorting tree, the `Head` / `Front` / `Rear` / `top` annotations — is drawn from a central message catalog. You can switch that catalog to another language with `language:`, override individual phrases with `messages:`, and still override any single operation call with `step-label:`. Nothing about the geometry or styling changes; only the words do.

A document that passes none of these arguments renders exactly as before, in English.

### 7.1 The three resolution layers

A caption is resolved by walking four sources, each one winning over the ones before it:

| Layer                            | What it is                                                                                     |
|----------------------------------|------------------------------------------------------------------------------------------------|
| English / default catalog        | The built-in source of truth. Contains every message key.                                      |
| Selected language catalog        | Chosen with <code>language:</code> . A complete translation that replaces the English strings. |
| <code>messages:</code> overrides | A partial dictionary layered over the selected language, per builder.                          |
| <code>step-label:</code>         | The highest-priority override, for one operation call only.                                    |

### 7.2 `language:`

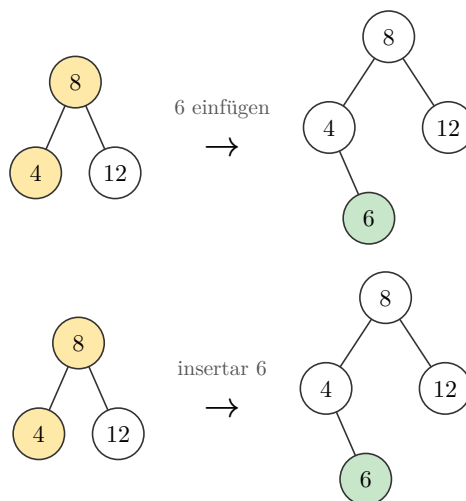
Every builder that can generate a caption accepts `language:`. The supported codes are `"en"` (default), `"de"`, and `"es"`. Any other value fails immediately with a clear assertion message, so a typo never silently falls back to English.

```

1 #std.stack(spacing: 1em,
2   (bst(8, 4, 12, language: "de").insert)(6).diagram,
3   (bst(8, 4, 12, language: "es").insert)(6).diagram,
4 )

```

**t** Typst

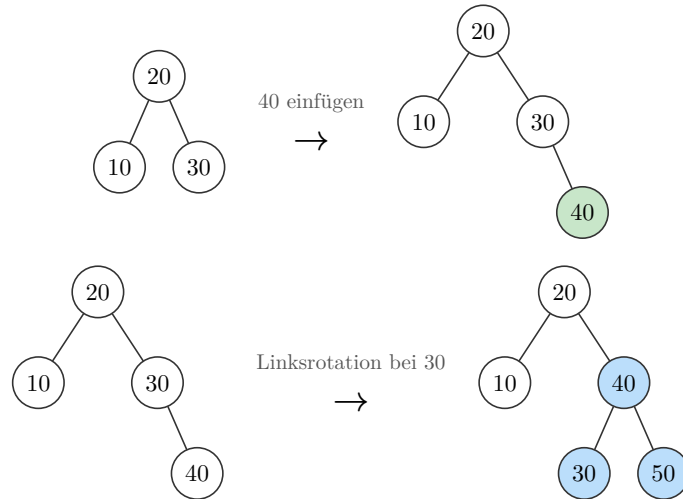


The chosen language is stored on the returned object and travels with it. Every operation you call, and every `.result` you chain from, keeps the same language without repeating the argument.

```

1 #let tree = avl(20, 10, 30, language: "de")
2 #let step = (tree.insert)(40, rebalance: (enabled: true))
3 #let next = (step.result.insert)(50)
4 #std.stack(spacing: 1em, step.diagram, next.diagram)

```

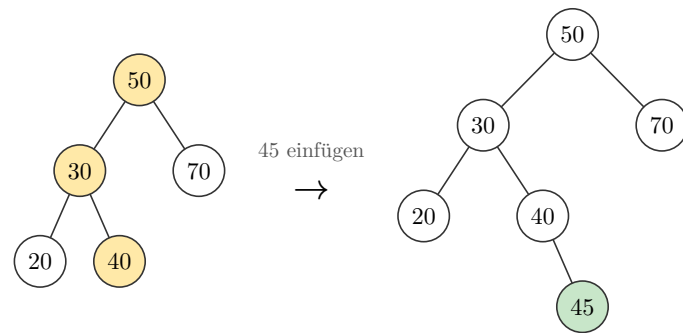
 Typst


The standalone operation functions ( `tree-insert` , `heap-insert` , and the graph and sorting builders) also take `language:` directly, so `transition(...)` and one-off traces localize the same way:

```

1 #transition("bst", (50, 30, 70, 20, 40), tree-insert(45, language: "de"))

```

 Typst


### 7.3 messages: and the messages(...) helper

`messages:` layers a partial dictionary of overrides over whatever `language:` selected. Use it to change specific wording — house style, a different verb, a discipline's notation — without shipping a whole catalog, or to add a language the package doesn't include yet by overriding the English base.

Build the dictionary with the `messages(...)` helper, which takes one named argument per structure group and returns a flat, reusable dictionary you can share across many builders:

```

1 #let spanish-tweaks = messages(
2   tree: (
3     insert: key => [insertar #key],
4     delete: key => [eliminar #key],
5   ),
6   graph: (
7     visit: node => [visitar #node],
8   ),
9 )

```

t Typst

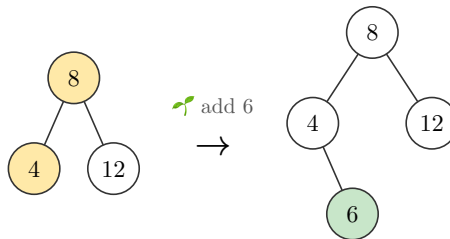
Each value is either static `content` or a callback returning `content`. Interpolating captions use a callback; fixed captions can be plain content. Callbacks receive whatever Typst value the caller passes — integers, strings, math, arrows, arbitrary content — so you are free to build a caption out of anything, not just a formatted number.

```

1 #let mine = messages(
2   tree: (insert: k => [🌱 add #k]),
3 )
4 #(bst(8, 4, 12, messages: mine).insert)(6).diagram

```

t Typst



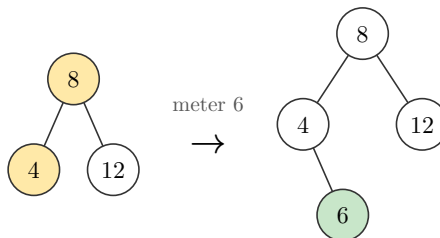
Overrides compose with a language. Here the Spanish catalog supplies every caption except `tree.insert`, which the override replaces:

```

1 #let mine = messages(tree: (insert: k => [meter #k]))
2 #(bst(8, 4, 12, language: "es", messages: mine).insert)(6).diagram

```

t Typst



An override key that does not name a real message fails with an error naming the offending key, so a misspelled group or key is caught at compile time instead of being silently ignored. (`messages:` also accepts a plain grouped dictionary like `(tree: (insert: ...))` directly, but `messages(...)` is the recommended, validated form.)

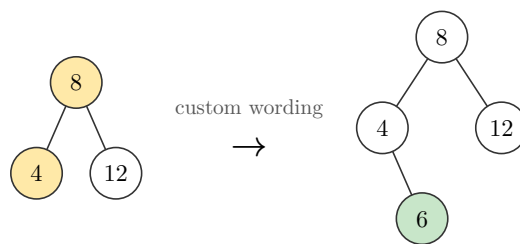
**Note.** Localization is deliberately **not** part of `style:`. Styling controls how a diagram looks (color, shape, size); `messages:` controls the words. They are independent, and a caption can contain math, arrows, or styled content, so wording is always `content`, never a plain string to be concatenated.

## 7.4 `step-label:` has the last word

The existing `step-label:` argument on each operation is unchanged and still wins over everything: language, messages, and the default. It sets both the rendered caption and the step's `.label` field for one call only.

```
1 #let tree = bst(8, 4, 12, language: "de")
2 #(tree.insert)(6, step-label: [custom wording]).diagram
```

 Typst

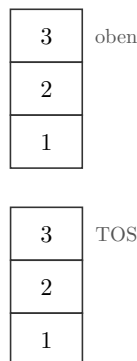


## 7.5 Structural default labels

`stack`'s `top-label`, and `queue`'s `front-label` and `rear-label`, default to `auto`, which means “use the localized default”. Passing explicit content still overrides the localized word for that one builder, so an explicit label always wins over the language:

```
1 #std.stack(spacing: 1.4em,
2   stack(3, 2, 1, language: "de").diagram,
3   stack(3, 2, 1, top-label: [TOS]).diagram,
4 )
```

 Typst



## 7.6 Message-key reference

Every key below exists in all three catalogs. In `messages(...)`, the part before the dot is the group name and the part after is the key. The **Arguments** column lists what a callback receives; keys with no arguments are static content.

| Key                 | Arguments    | English default                                 |
|---------------------|--------------|-------------------------------------------------|
| tree.insert         | key          | insert <code>key</code>                         |
| tree.delete         | key          | delete <code>key</code>                         |
| tree.search         | key          | search <code>key</code>                         |
| tree.rotate-left    | pivot        | rotate left at <code>pivot</code>               |
| tree.rotate-right   | pivot        | rotate right at <code>pivot</code>              |
| heap.insert         | key          | insert <code>key</code>                         |
| heap.extract        | —            | extract                                         |
| list.insert         | value        | insert <code>value</code>                       |
| list.insert-at      | value, index | insert <code>value</code> at <code>index</code> |
| list.prepend        | value        | prepend <code>value</code>                      |
| list.delete         | value        | delete <code>value</code>                       |
| list.delete-at      | index        | delete index <code>index</code>                 |
| list.search         | value        | search <code>value</code>                       |
| list.head           | —            | Head                                            |
| stack.push          | value        | push <code>value</code>                         |
| stack.pop           | —            | pop                                             |
| stack.top           | —            | top                                             |
| queue.enqueue       | value        | enqueue <code>value</code>                      |
| queue.dequeue       | —            | dequeue                                         |
| queue.enqueue-label | —            | Enqueue (single-frame builder annotation)       |
| queue.dequeue-label | —            | Dequeue (single-frame builder annotation)       |
| queue.front         | —            | Front                                           |
| queue.rear          | —            | Rear                                            |
| skip.search         | key          | search <code>key</code>                         |
| skip.insert         | value        | insert <code>value</code>                       |
| skip.delete         | value        | delete <code>value</code>                       |
| hash.insert         | key          | insert <code>key</code>                         |
| hash.delete         | key          | delete <code>key</code>                         |
| hash.search         | key          | search <code>key</code>                         |
| graph.queue         | node         | queue <code>node</code> (BFS start)             |
| graph.stack         | node         | stack <code>node</code> (DFS start)             |
| graph.visit         | node         | visit <code>node</code>                         |
| graph.inspect       | from, to     | inspect <code>from</code> → <code>to</code>     |
| graph.finish        | node         | finish <code>node</code>                        |
| graph.reached       | node         | reached <code>node</code>                       |
| graph.settle        | node         | settle <code>node</code> (Dijkstra)             |
| graph.relax         | from, to     | relax <code>from</code> → <code>to</code>       |

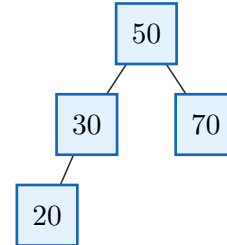
|                        |                    |                                        |
|------------------------|--------------------|----------------------------------------|
| graph.distance-init    | node               | distance( node ) = 0                   |
| graph.shortest-path    | node               | shortest path to node found            |
| graph.distance         | value              | d = value (node distance label)        |
| sort.original          | —                  | original array                         |
| sort.sorted-array      | —                  | sorted array                           |
| sort.divide            | —                  | divide (merge-sort step)               |
| sort.divide-phase      | —                  | Divide (merge-sort phase brace)        |
| sort.merge             | —                  | merge (merge-sort step)                |
| sort.merge-phase       | —                  | Merge (merge-sort phase brace)         |
| sort.partition-phase   | —                  | Partition (quick-sort phase brace)     |
| sort.left              | —                  | left (merge-operation column)          |
| sort.right             | —                  | right (merge-operation column)         |
| sort.result            | —                  | result (merge-operation column)        |
| sort.start             | —                  | start                                  |
| sort.sorted            | —                  | sorted                                 |
| sort.compare           | a, b               | compare a and b                        |
| sort.swap              | a, b               | swap a and b                           |
| sort.settled           | value              | value settled                          |
| sort.start-merge       | —                  | start merge                            |
| sort.merged            | —                  | merged                                 |
| sort.take              | value              | take value                             |
| sort.take-remaining    | value              | take remaining value                   |
| sort.compare-pivot     | a, pivot           | compare a and pivot pivot              |
| sort.select-pivot      | value              | select pivot value                     |
| sort.select-last-pivot | value              | select last pivot value                |
| sort.place-pivot       | value              | place pivot value                      |
| sort.advance-i         | i                  | advance i to i                         |
| sort.partitioned       | —                  | partitioned                            |
| sort.partition-pivot   | value              | partition pivot value                  |
| sort.partition-around  | value              | partition around pivot value           |
| sort.pivot-info        | value, index, i, j | pivot: value at index i: i j: j        |
| sort.i-satisfies       | value              | i value satisfies the pivot test       |
| sort.j-satisfies       | value              | j value satisfies the pivot test       |
| sort.selection-status  | pos, min, item     | position pos , minimum min , item item |

## 8 Styling

Every builder takes a `style`: dictionary that is merged over the package defaults, so colors, size, and shape can be overridden per call. `theme` is the default dictionary itself and `resolve(style)` is the merge helper ( `theme + style` ); both are exported if you want to build your own styling utilities on top. Raw dictionaries remain supported.

```
1 #bst(50, 30, 70, 20, style: tree-
  style(
2   node-shape: "square",
3   node-radius: 0.4,
4   node-fill: rgb("#E3F2FD"),
5   node-stroke: 1pt + rgb("#1565C0"),
6   node-text: text-style(size: 11pt),
7 ).diagram
```

 Typst

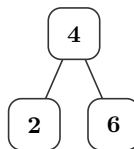


### 8.1 Named themes and typography roles

`theme-preset(name)` returns a sparse style dictionary. Available names are "default" , "dark" , "print" , "colorblind" , and "chalkboard" . Add a structure-specific style dictionary to override part of a preset.

```
1 #bst(
2   4, 2, 6,
3   style: theme-preset("colorblind") + tree-style(
4     node-shape: "rounded",
5     value-text: text-style(weight: "bold"),
6   ),
7 ).diagram
```

 Typst



| Argument                          | Type       | Default                          | Description                                 |
|-----------------------------------|------------|----------------------------------|---------------------------------------------|
| <code>value-text</code>           | dictionary | inherits <code>node-text</code>  | Values inside nodes and cells.              |
| <code>index-text</code>           | dictionary | inherits <code>label-text</code> | Array indices and matrix row/column labels. |
| <code>pointer-text</code>         | dictionary | inherits <code>label-text</code> | Pointers and linear-structure annotations.  |
| <code>operation-text</code>       | dictionary | (size: 8pt)                      | Transition and sequence arrow captions.     |
| <code>edge-label-text</code>      | dictionary | inherits <code>label-text</code> | Tree and graph edge labels.                 |
| <code>algorithm-label-text</code> | dictionary | (size: 8pt)                      | Sorting and graph-algorithm captions.       |

Each typography role accepts `size` , `color` / `fill` , `font` , `weight` , and `rotation` , the same nested keys produced by `text-style(...)` .

**Note.** `node-text` and `label-text` remain the broad compatibility defaults. The specialized text dictionaries inherit from them, so an existing style keeps applying until a more specific role overrides it.

**Note.** To set a document-wide default instead of repeating `style:` at every call, rebind the builder once: `#let bst = bst.with(style: tree-style(..))` .

**Note.** Editor tip: Typst dictionaries do not expose a schema to autocomplete. Use the style-builder functions when you want argument suggestions while writing `style:` . Raw dictionaries still work when you prefer them.

## 8.2 Autocomplete-friendly style builders

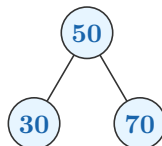
Structure-specific style builders return sparse dictionaries. Because their keys are named function arguments, Typst editors suggest only relevant keys while you type `style: tree-style(...)` , `style: stack-style(...)` , and so on. Omitted arguments do not overwrite package or structure-specific defaults. Nested builders provide the same completion for dictionary-valued settings.

```

1  #bst(
2    50, 30, 70,
3    style: tree-style(
4      x-gap: 1.4,
5      node-fill: rgb("#E7F5FF"),
6      node-text: text-style(
7        size: 10pt,
8        color: rgb("#1864AB"),
9        weight: "bold",
10     ),
11     new-style: node-mark-style(
12       fill: rgb("#D3F9D8"),
13       stroke: 1pt + rgb("#2B8A3E"),
14     ),
15   ),
16 ).diagram

```

 Typst



| Argument                     | Type     | Default | Description                                                 |
|------------------------------|----------|---------|-------------------------------------------------------------|
| <code>tree-style(...)</code> | function | n/a     | Styles BSTs, AVL trees, manual trees, and tree transitions. |
| <code>heap-style(...)</code> | function | n/a     | Styles min/max heaps and heap transitions.                  |

|                                                                                                   |          |     |                                                                                            |
|---------------------------------------------------------------------------------------------------|----------|-----|--------------------------------------------------------------------------------------------|
| <code>graph-style(...)</code>                                                                     | function | n/a | Styles graphs.                                                                             |
| <code>list-style(...)</code>                                                                      | function | n/a | Styles linked and doubly linked lists.                                                     |
| <code>stack-style(...)</code>                                                                     | function | n/a | Styles stacks.                                                                             |
| <code>queue-style(...)</code>                                                                     | function | n/a | Styles queues.                                                                             |
| <code>array-style(...)</code>                                                                     | function | n/a | Styles arrays, including <code>indices:</code> .                                           |
| <code>matrix-style(...)</code>                                                                    | function | n/a | Styles matrices.                                                                           |
| <code>text-style(size:, color:, fill:, font:, weight:, rotation:)</code>                          | function | n/a | Builds <code>style.node-text</code> or mark text overrides.                                |
| <code>label-style(size:, color:, fill:, font:, weight:, rotation:)</code>                         | function | n/a | Builds <code>style.label-text</code> .                                                     |
| <code>node-mark-style(fill:, shape:, stroke:, node-radius:, text:)</code>                         | function | n/a | Builds tree/heap diff styles.                                                              |
| <code>cell-mark-style(fill:, stroke:, text:)</code>                                               | function | n/a | Builds linear-structure diff styles without unsupported node geometry.                     |
| <code>node-label-style(position:, offset:, gap:, size:, color:, font:, weight:, rotation:)</code> | function | n/a | Builds <code>style.node-labels</code> .                                                    |
| <code>indices-style(enabled:, labels:, offset:, size:, color:, font:, weight:)</code>             | function | n/a | Builds <code>style.indices</code> . Pass <code>labels: auto</code> for zero-based indices. |

### 8.3 Where keys are documented

The detailed key lists now live where the argument appears, including nested dictionaries such as `style.node-text` , `style.label-text` , and `edge-customizations[].options.label` .

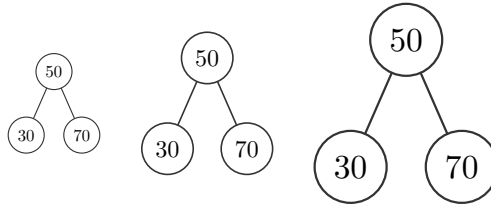
| Area              | Detailed reference                                                                                                                                                                                                                                                                            |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Trees             | <code>bst</code> , <code>avl</code> , <code>tree</code> , and <code>transition</code> list their full <code>style.*</code> , <code>edge-customizations.*</code> , <code>node-customizations.*</code> , and <code>node-labels.*</code> keys in their argument reference.                       |
| Heaps             | <code>min-heap</code> , <code>max-heap</code> , and heap transitions list the tree-renderer <code>style.*</code> keys they use.                                                                                                                                                               |
| Graphs            | <code>graph</code> lists <code>adjacency.*</code> , <code>labels.*</code> , <code>positions.*</code> , <code>edge-customizations.*</code> , <code>node-customizations.*</code> , <code>node-labels.*</code> , <code>edge-customizations[].options.label.*</code> , and <code>style.*</code> . |
| Linear structures | <code>linked-list</code> , <code>doubly-linked-list</code> , <code>stack</code> , and <code>queue</code> list their cell, pointer, label, scale, and diff-highlight <code>style.*</code> keys.                                                                                                |

```

1 #std.stack(dir: ltr, spacing: 1.5em,
2   bst(50, 30, 70, style: (scale: 0.7)).diagram,
3   bst(50, 30, 70).diagram,
4   bst(50, 30, 70, style: (scale: 1.4)).diagram,
5 )

```

**t** Typst



## 8.4 Diff-highlight styling

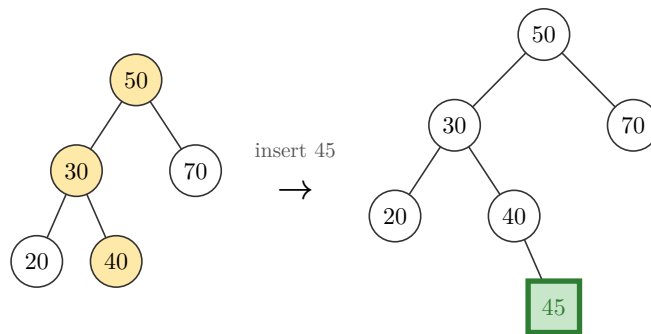
A plain color is shorthand for `(fill: color)` every example so far used that shorthand. Pass a dictionary instead to also override `shape`, `stroke`, `node-radius`, or `text` on just the marked node/cell. Fields the dictionary leaves out fall back to the surrounding `node-shape` / `node-stroke` / `node-radius`, **except** `fill`, which falls back to the key's own default highlight color, not `node-fill`. `text` merges over `node-text`, so each mark kind can have its own label color, size, font, or weight.

```

1 #transition("bst", (50, 30, 70, 20, 40), tree-insert(45), style: (
2   // square instead of circle, thicker stroke; fill stays the default green
3   new-style: (shape: "square", stroke: 2pt + rgb("#2E7D32"), text: (color:
4     rgb("#2E7D32"))),
5 ))

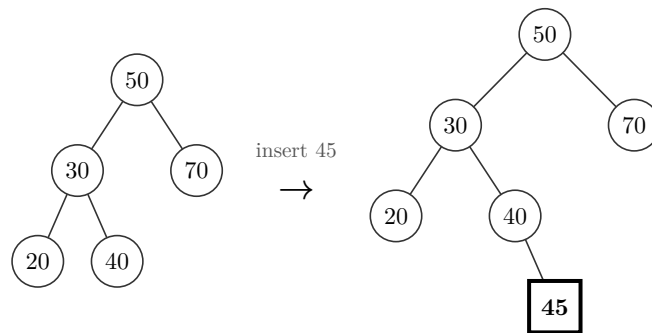
```

**t** Typst



Set `diff-colors: false` to remove highlight fill colors while keeping operation marks. Shape, stroke, and radius overrides still apply.

```
1 #transition("bst", (50, 30, 70, 20, 40), tree-insert(45), style: (  
2   diff-colors: false,  
3   new-style: (shape: "square", stroke: 1.5pt + black, text: (weight: "bold")),  
4 ))
```

[t Typst](#)

**Note.** This applies to every structure that highlights marks: `bst` / `avl` (node `shape` / `stroke` / `node-radius`), `min-heap` / `max-heap` (same, since they reuse the tree renderer), and `stack` / `queue` / `linked-list` / `doubly-linked-list` (cell `fill` / `stroke` cell marks keep the surrounding `box-shape` and do not use `node-radius`).

## 9 Worked example: Last Stone Weight

A complete example pairing a real solution with the diagram it produces: [LeetCode 1046, Last Stone Weight](#) (Easy). Repeatedly smash the two heaviest stones together; if they're equal both are destroyed, otherwise the difference goes back in. A max-heap hands you the two heaviest in  $O(\log n)$  each round instead of a full rescan.

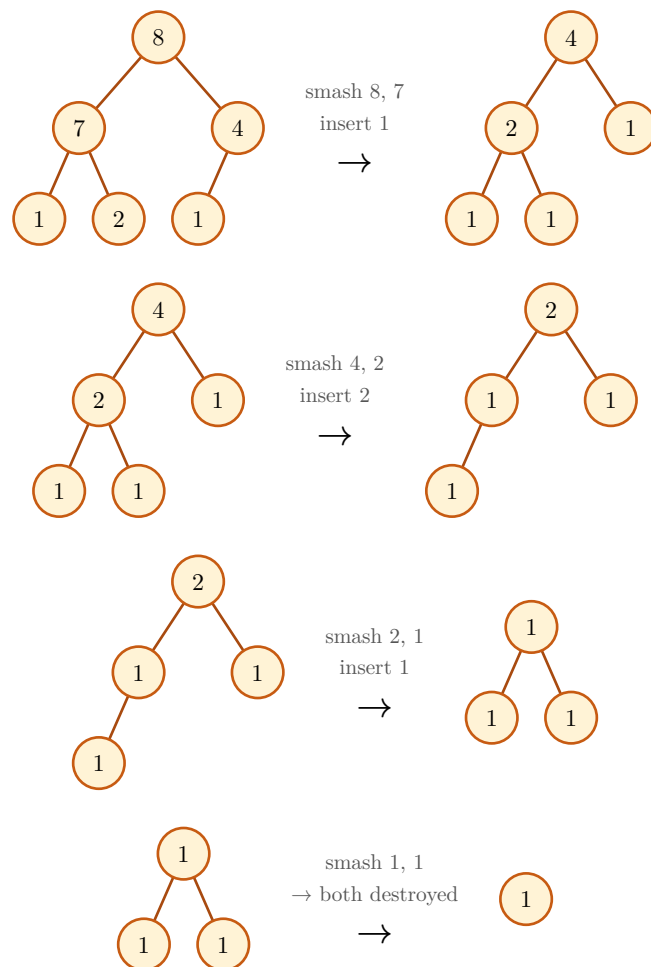
```

1  import heapq
2
3  def lastStoneWeight(stones):
4      heap = [-s for s in stones]
5      heapq.heapify(heap)
6      while len(heap) > 1:
7          y, x = -heapq.heappop(heap), -heapq.heappop(heap)
8          if y != x:
9              heapq.heappush(heap, -(y - x))
10     return -heap[0] if heap else 0

```

 Python

The diagram below runs the **same** algorithm through `max-heap`, chained with `.extract()` / `.insert()` and a custom `op-arrow` label per round. The diagram comes from running the algorithm, not from hand-drawing pictures to match the code:



One stone remains, weight **1**, matching `lastStoneWeight([2, 7, 4, 1, 8, 1]) == 1`.

## 10 Limitations

---

- Tree operation diff marks are keyed by node **value**, not position, so a tree with duplicate keys can highlight more nodes than actually moved. Heap operation marks are position-keyed. For static tree/graph diagrams, use [node-customizations](#) for explicit per-node control. For arrays and matrices, use [cell-customizations](#) .
- [graph](#) lays nodes out on a circle. A layout that pulls connected nodes closer together isn't built yet.
- [graph](#) has no operations yet: no add-node, add-edge, or a transition to match trees and heaps.
- Graph algorithm traces show frontier state on the graph itself; they do not draw a separate queue, stack, or priority queue panel.
- The default hash function is intended for teaching examples, not cryptographic or adversarial input. Pass `hash: key => int` when bucket placement must follow a course-specific function.

## 11 Quick reference

### 11.1 Structure builders

| Call                                                                                                                                                    | Result                                                                                                                                  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <code>bst(..keys, style:, edge-customizations:, node-customizations:, node-labels:)</code>                                                              | Binary search tree built by inserting keys in order; ops <code>insert</code> , <code>delete</code> , <code>search</code>                |
| <code>avl(..keys, style:, edge-customizations:, node-customizations:, node-labels:)</code>                                                              | AVL tree, rebalanced on every insertion and deletion; same ops as <code>bst</code>                                                      |
| <code>min-heap(..keys, style:)</code>                                                                                                                   | Array-backed complete binary tree, smallest key at the root; ops <code>insert</code> , <code>extract</code>                             |
| <code>max-heap(..keys, style:)</code>                                                                                                                   | Array-backed complete binary tree, largest key at the root; ops <code>insert</code> , <code>extract</code>                              |
| <code>linked-list(..vals, style:, pointer:, addresses:, head:)</code>                                                                                   | Linked list; ops <code>prepend</code> , <code>insert</code> , <code>delete</code> , <code>delete-at</code> , and <code>search</code>    |
| <code>doubly-linked-list(..vals, style:, pointer:, addresses:, head:)</code>                                                                            | Doubly linked list; same positional and value operations as <code>linked-list</code>                                                    |
| <code>stack(..vals, style:, top-label:)</code>                                                                                                          | Stack, first argument is the top; ops <code>push</code> , <code>pop</code>                                                              |
| <code>queue(..vals, style:, enqueue:, dequeue:, front-label:, rear-label:)</code>                                                                       | Queue, first argument is the front; ops <code>enqueue</code> , <code>dequeue</code>                                                     |
| <code>skip-list(..vals, style:, decision-fn:, level-spacing:, max-level:)</code>                                                                        | Sorted skip list with express-lane levels; ops <code>search</code> , <code>insert</code> , <code>delete</code>                          |
| <code>array-view(..vals, style:, cell-customizations:, pointers:)</code>                                                                                | Static array cells with optional <code>style.indices</code> , per-cell overrides, and labelled <code>pointers</code> arrows above cells |
| <code>matrix(rows, style:, cell-customizations:, row-labels:, column-labels:)</code>                                                                    | Static matrix/grid cells with optional row/column labels and per-cell overrides                                                         |
| <code>graph(adjacency, directed:, labels:, positions:, layout:, radius:, gap:, edge-customizations:, node-customizations:, node-labels:, style:)</code> | Graph from an adjacency dict; circular, linear, or manual layout, no ops yet                                                            |
| <code>hash-table(..entries, size:, collision:, hash:, style:)</code>                                                                                    | Separate-chaining or linear-probing table; ops <code>insert</code> , <code>delete</code> , and <code>search</code>                      |

### 11.2 Graph algorithms

| Call                                                               | Result                                                                                             |
|--------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| <code>bfs(adjacency, source, target:, ..graph-options)</code>      | Breadth-first trace; result has <code>order</code> , <code>found</code> , and <code>path</code>    |
| <code>dfs(adjacency, source, target:, ..graph-options)</code>      | Depth-first trace with the same result fields                                                      |
| <code>dijkstra(adjacency, source, target:, ..graph-options)</code> | Non-negative shortest-path trace; result also has <code>distances</code> and <code>previous</code> |

## 11.3 Sorting algorithms

| Call                                                                                                | Result                                                                                                                                                                                                                                              |
|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>merge-sort(array, order:, labels:)</code>                                                     | One breadth-by-depth divide-and-merge tree from a single input array; returns <code>.diagram</code> , <code>.steps</code> , and <code>.result</code>                                                                                                |
| <code>merge-operation(left, right, order:, pointers:, labels:)</code>                               | Step-by-step merge of two sorted arrays, with default <code>i</code> , <code>j</code> , and <code>i+j</code> cursor pointers and a progressively filled result array                                                                                |
| <code>partition-step(array, order:, pivot:, pointers:, labels:)</code>                              | Partition trace with a middle pivot, or a detailed last-pivot quicksort trace with <code>i</code> , <code>j</code> , and <code>pivot</code> markers                                                                                                 |
| <code>quick-sort(array, order:, pivot:, labels:)</code>                                             | Breadth-by-depth partition tree: every active subarray partitions on the same level; <code>pivot</code> is "first" , "last" , or an index                                                                                                           |
| <code>bubble-sort(array, order:, pointers:, labels:, compare:, swap:)</code>                        | Comparison and swap trace; pointers mark positions with index arrows by default                                                                                                                                                                     |
| <code>insertion-sort(array, order:, pointers:, labels:, compare:, swap:)</code>                     | Sorted-prefix insertion trace using adjacent swaps and default pointer markers                                                                                                                                                                      |
| <code>selection-sort(array, order:, pointers:, labels:, compare:, current:, minimum:, swap:)</code> | Selection and swap trace with default pointer markers; red marks <code>i</code> , purple marks minimum index <code>m</code> , and yellow marks inspected index <code>j</code> . When <code>i = m</code> , the cell combines red with purple stripes |
| <code>sort-sequence(steps, columns:, gap:, row-gap:)</code>                                         | Custom layout for generated sorting steps                                                                                                                                                                                                           |

## 11.4 Hand-composed trees

| Call                                                                                      | Result                                                            |
|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| <code>tree(root, style:, edge-customizations:, node-customizations:, node-labels:)</code> | Render a tree built from <code>node</code> / <code>subtree</code> |
| <code>node(label, left:, right:, children:, fill:)</code>                                 | A binary or multiway node with any content label                  |
| <code>subtree(label, fill:, height:, scale:)</code>                                       | Triangle leaf with an optional height bracket                     |

## 11.5 Transitions and operations

| Call                                                                                                         | Result                           |
|--------------------------------------------------------------------------------------------------------------|----------------------------------|
| <code>transition(variant, keys, op, style:, edge-customizations:, node-customizations:, node-labels:)</code> | Before<br>→<br>arrow<br>→<br>de- |

|                                                      |                                                                                                                                                                                                                                                                                                                                                                              |
|------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                      | <div>rived<br/>after<br/>for<br/>bst /<br/>avl /<br/>min-heap /<br/>max-heap ,<br/>diff<br/>highlighted</div>                                                                                                                                                                                                                                                                |
| <div>tree-insert(key, rebalance:, step-label:)</div> | <div>Op-<br/>era-<br/>tion:<br/>in-<br/>sert<br/>key .<br/>re-<br/>bal-<br/>ance:<br/>(en-<br/>abled:<br/>true)<br/>adds<br/>a panel<br/>for<br/>the<br/>un-<br/>ro-<br/>tated<br/>tree,<br/>when<br/>AVL<br/>rotates;<br/>all-<br/>steps:<br/>true<br/>splits<br/>a<br/>dou-<br/>ble<br/>rota-<br/>tion<br/>into<br/>its<br/>own<br/>in-<br/>ner/<br/>outer<br/>steps</div> |

|                                                                         |                                                                                    |
|-------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <code>tree-delete(key, step-label:)</code>                              | Operation: delete <code>key</code><br>AVL re-balances when needed                  |
| <code>tree-search(key, step-label:)</code>                              | Operation: highlight the path to <code>key</code>                                  |
| <code>heap-insert(key, step-label:)</code>                              | Operation: insert <code>key</code> , marks the inserted value and the sift-up path |
| <code>heap-extract</code>                                               | Operation: remove the root, marks it and the sift-down path (no <code>key</code> ) |
| <code>sequence(..steps, columns:, gap:, row-gap:, mode:, style:)</code> | Wrap operation steps                                                               |

|                                                                         |                                                                                                                           |
|-------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
|                                                                         | or diagrams into a grid; <code>mode</code> chooses full steps, before/after panels, or plain results                      |
| <code>operation-sequence(initial, ..operations, columns:, mode:)</code> | Apply operation closures in order and return <code>.steps</code> , final <code>.result</code> , and <code>.diagram</code> |
| <code>op-arrow(label, symbol:, style:)</code>                           | Reusable labeled arrow for custom operation layouts                                                                       |

11.6 Localization

| Call / argument | Result |
|-----------------|--------|
|-----------------|--------|

|                                  |                                                                                                                                                                                                 |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>language:</code>           | Caption language on every generating builder: <code>"en"</code> (default), <code>"de"</code> , <code>"es"</code> . Persists through operations and <code>.result</code> . Invalid codes assert. |
| <code>messages:</code>           | Partial overrides layered over the selected language. Build with <code>messages(...)</code> . Unknown keys assert.                                                                              |
| <code>messages(...groups)</code> | Turn structure-grouped definitions ( <code>tree: (insert: k =&gt; ...)</code> ) into a flat, reusable overrides dictionary.                                                                     |
| <code>step-label:</code>         | Highest-priority caption override for one operation call; also sets the step's <code>label</code> .                                                                                             |
| <code>supported-languages</code> | The array of accepted language codes, currently <code>("en", "de", "es")</code> .                                                                                                               |

See [Section 7](#) for the full message-key catalog and the resolution order.

## 11.7 Styling keys

| Key                              | Default      | Effect                                                                              |
|----------------------------------|--------------|-------------------------------------------------------------------------------------|
| <code>node-radius</code>         | 0.34         | Circle radius / half shape size.                                                    |
| <code>node-shape</code>          | "circle"     | "circle" , "square" , "rounded" , "capsule" , "diamond" , or "hexagon" .            |
| <code>x-gap</code>               | 1.05         | Horizontal spacing between columns.                                                 |
| <code>y-gap</code>               | 1.2          | Vertical spacing between depths.                                                    |
| <code>node-stroke</code>         | 0.6pt + #333 | Node outline.                                                                       |
| <code>node-fill</code>           | white        | Default node fill.                                                                  |
| <code>edge-stroke</code>         | 0.6pt + #333 | Edge stroke.                                                                        |
| <code>edge-arrow</code>          | none         | Edge arrowheads: <code>none</code> , "end" / <code>true</code> , "start" , "both" . |
| <code>edge-arrow-fill</code>     | none         | <code>graph</code> arrowheads: <code>none</code> (open) or a color (solid).         |
| <code>edge-pattern</code>        | "normal"     | Global tree/ <code>graph</code> edge pattern.                                       |
| <code>edge-wave-amplitude</code> | 0.07         | Wave height in canvas units.                                                        |
| <code>edge-wave-step</code>      | 0.14         | Approximate wavelength in canvas units.                                             |
| <code>tri-w</code>               | 1.2          | Subtree triangle base width.                                                        |
| <code>tri-h</code>               | 1.4          | Subtree triangle height.                                                            |
| <code>box-w</code>               | 0.95         | Linear-structure cell width.                                                        |
| <code>box-h</code>               | 0.7          | Linear-structure cell height.                                                       |
| <code>box-shape</code>           | "square"     | "square" , "rounded" , or "capsule" cells.                                          |

|                                                                                     |                       |                                                                            |
|-------------------------------------------------------------------------------------|-----------------------|----------------------------------------------------------------------------|
| <code>box-gap</code>                                                                | 0.55                  | Gap between cells/rows.                                                    |
| <code>box-stroke</code>                                                             | 0.6pt + #333          | Cell outline.                                                              |
| <code>box-fill</code>                                                               | white                 | Default cell fill.                                                         |
| <code>ptr-fill</code>                                                               | #D7ECC9               | Linked-list pointer-cell fill.                                             |
| <code>prev-ptr-fill</code>                                                          | #D7ECC9               | Doubly linked list previous-pointer fill.                                  |
| <code>next-ptr-fill</code>                                                          | #D7ECC9               | Doubly linked list next-pointer fill.                                      |
| <code>node-text</code>                                                              | (size: 9pt)           | Node/cell text styling.                                                    |
| <code>label-text</code>                                                             | (fill: #555)          | Annotation/edge text styling.                                              |
| <code>value-text</code>                                                             | (:)                   | Specialized node/cell value typography.                                    |
| <code>index-text</code>                                                             | (size: 7.5pt)         | Index and row/column typography.                                           |
| <code>pointer-text</code>                                                           | (:)                   | Pointer/linear annotation typography.                                      |
| <code>operation-text</code>                                                         | (size: 8pt)           | Operation caption typography.                                              |
| <code>edge-label-text</code>                                                        | (:)                   | Edge-label typography.                                                     |
| <code>algorithm-label-text</code>                                                   | (size: 8pt)           | Algorithm trace caption typography.                                        |
| <code>scale</code>                                                                  | 1.0                   | Uniform scale on the whole rendered diagram.                               |
| <code>diff-colors</code>                                                            | true                  | Set <code>false</code> to keep mark shapes/strokes but use normal fills.   |
| <code>new-style</code>                                                              | #C8E6C9               | Added node/cell; color or (fill: , shape:, stroke:, node-radius:, text:) . |
| <code>path-style</code>                                                             | #FFE9A8               | Traversal / sift path; color or dict, as above.                            |
| <code>remove-style</code>                                                           | #FFCDD2               | Removed node/cell; color or dict, as above.                                |
| <code>rotate-style</code>                                                           | #BBDEFB               | AVL rotation nodes and visible subtree roots; color or dict, as above.     |
| <code>visited-style</code> / <code>current-style</code> / <code>queued-style</code> | green / blue / yellow | Graph-algorithm node states.                                               |
| <code>active-edge-style</code>                                                      | blue 2pt              | Graph-algorithm inspected edge.                                            |

## 11.8 Styling helpers

| Call                                                                                              | Result                                                                                                                                                             |
|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>tree-style(..tree keys)</code>                                                              | Sparse BST/<br>AVL/manual-<br>tree style dictio-<br>nary                                                                                                           |
| <code>heap-style(..heap keys)</code>                                                              | Sparse min/<br>max-heap style<br>dictionary                                                                                                                        |
| <code>graph-style(..graph keys)</code>                                                            | Sparse graph<br>style dictionary                                                                                                                                   |
| <code>list-style(..list keys)</code>                                                              | Sparse linked-<br>list style dictio-<br>nary                                                                                                                       |
| <code>stack-style(..stack keys)</code>                                                            | Sparse stack<br>style dictionary                                                                                                                                   |
| <code>queue-style(..queue keys)</code>                                                            | Sparse queue<br>style dictionary                                                                                                                                   |
| <code>array-style(..array keys)</code>                                                            | Sparse array<br>style dictionary                                                                                                                                   |
| <code>matrix-style(..matrix keys)</code>                                                          | Sparse matrix<br>style dictionary                                                                                                                                  |
| <code>theme-preset(name)</code>                                                                   | Named <code>de-<br/>fault</code> / <code>dark</code> /<br><code>print</code> / <code>color-<br/>blind</code> / <code>chalk-<br/>board</code> style dic-<br>tionary |
| <code>text-style(size:, color:, fill:, font:, weight:, rotation:)</code>                          | Node/cell or<br>mark text dic-<br>tionary                                                                                                                          |
| <code>label-style(size:, color:, fill:, font:, weight:, rotation:)</code>                         | Annotation/<br>edge text dictio-<br>nary                                                                                                                           |
| <code>node-mark-style(fill:, shape:, stroke:, node-radius:, text:)</code>                         | Tree/heap diff-<br>highlight dictio-<br>nary                                                                                                                       |
| <code>cell-mark-style(fill:, stroke:, text:)</code>                                               | Linear-cell diff-<br>highlight dictio-<br>nary                                                                                                                     |
| <code>node-label-style(position:, offset:, gap:, size:, color:, font:, weight:, rotation:)</code> | Default exter-<br>nal node-label<br>dictionary                                                                                                                     |

|                                                                                       |                        |
|---------------------------------------------------------------------------------------|------------------------|
| <code>indices-style(enabled:, labels:, offset:, size:, color:, font:, weight:)</code> | Array-index dictionary |
|---------------------------------------------------------------------------------------|------------------------|